



**Politecnico
di Torino**

Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2025/2026

Programmazione di sistema

Docenti:

Gianpiero Cabodi
Giovanni Malnati

Studente:

Cristiano Berardo

Indice

Elenco delle figure

VII

I	API Programming	1
1	Architettura e memoria	2
1.1	Obiettivi	2
1.2	Perché studiare come è organizzata la memoria?	2
1.3	Programmi e loro esecuzione	2
1.4	Spazio di indirizzamento	3
1.5	Effetti pratici	3
1.6	Gerarchia di memoria	4
1.7	Località	4
1.8	Esecuzione e programmi di alto livello	4
1.9	Modello di esecuzione di un programma	4
1.10	Livelli di astrazione	5
1.11	Librerie di esecuzione	5
1.12	Modello di esecuzione nei linguaggi C e C++	5
1.13	Esecuzione sequenziale sincrona, senza limitazioni	5
1.14	Flusso di esecuzione	6
1.15	Stack	6
1.16	Heap	7
1.17	Organizzazione dello spazio di indirizzamento	7
1.18	Ciclo di vita delle variabili	8
1.19	Variabili globali e locali	8
1.20	Variabili dinamiche	8
1.21	Allocazione della memoria	8
1.22	Rilascio della memoria	9
1.23	Puntatori	9
1.24	Il costo del rilascio	9
1.25	Puntatori in C/C++	9
1.26	Le ambiguità dei puntatori in C/C++	10
1.27	Problemi legati ai puntatori	10
1.28	Responsabilità del programmatore	10
1.29	Rischi	10
1.30	Problemi dei puntatori in C/C++	11
1.31	Le risposte di Rust	11
1.32	Gestire i puntatori	11
1.33	Possesso della memoria	11
1.34	Dipendenze	12
1.35	Ma non basta scrivere un programma "giusto"?	12
1.36	Perché questo problema non si verifica in altri linguaggi?	12
1.37	Confronto	13

2	Possesso	14
2.1	Possesso	14
2.2	Movimento	15
2.3	Copia	16
2.3.1	Copia e Movimento	16
2.4	Clonazione	17
2.5	Box<T> - Puntatore con possesso	18
2.6	Riferimenti	19
2.6.1	Riferimenti e prestiti	19
2.6.2	Riferimenti mut - mutual exclusion	19
2.6.3	Riferimenti: disposizione in memoria	21
2.7	Tempo di vita dei riferimenti	21
2.7.1	Esistenza in vita	22
2.8	Riferimenti e funzioni	22
2.9	Riferimenti e strutture dati	24
2.10	Elisione dei tempi di vita	24
2.11	Possesso - riassunto delle regole	25
2.12	Puntatori in Rust	25
2.13	Slice	25
2.14	Vantaggi introdotti dal concetto di possesso	27
2.15	Disposizione in memoria	27
3	Tipi composti	28
3.1	Struct in Rust	28
3.1.1	Come utilizzare una struct	28
3.1.2	Accesso alla struct	29
3.1.3	Struct fatte a tupla	29
3.1.4	Probabile rappresentazione in memoria	29
3.1.5	Rappresentazione in memoria	30
3.2	Visibilità	30
3.3	Metodi	30
3.3.1	Definizione di un metodo	31
3.4	Costruttori	32
3.5	Gestione delle risorse	32
3.6	RAII - Resource Acquisition Is Initialization	32
3.7	Distruttori	33
3.8	Metodi statici	33
3.9	Enum	33
3.9.1	Rappresentazione in memoria	34
3.9.2	Enumerazioni e clausole match	34
3.10	Destruutturazione	34
4	Gestione degli errori	36
4.1	Errori ed eccezioni	36
4.1.1	Spazio di scelta	36
4.1.2	Supporto sintattico alla gestione degli errori	37
4.1.3	I limiti della gestione delle eccezioni	37
4.1.4	Gestioni delle eccezioni in Rust	37
4.1.5	Elaborare i risultati	38
4.1.6	Gestire gli errori	38
4.1.7	Panic	38
4.1.8	Ignorare gli errori	39
4.1.9	Propagare gli errori	39
4.1.10	Altri modi di esprimere il fallimento	39
4.1.11	Propagare errori eterogenei	40

5 Moduli	43
6 Polimorfismo	44
II OS Programming	45
7 Main memory	46
7.1 Memoria principale	46
7.1.1 Località spaziale e località temporale	46
7.2 Relocation problem	47
7.3 Protezione	47
7.3.1 Protezione degli indirizzi hardware	47
7.4 Address binding	47
7.5 Memory Managment Unit - MMU	48
7.5.1 Spazio di indirizzamento logico vs. fisico	48
7.6 MMU	48
7.6.1 Relocatin register	49
7.6.2 Dynamic loading	49
7.6.3 Static vs Dynamic Linking	49
7.7 Allocazione dei programmi	50
7.7.1 Allocazione contigua	50
7.7.2 Partizione variabile	50
7.7.3 Problema dell'allocazione dinamica della memoria	51
7.7.4 Frammentazione	51
7.7.5 Compattazione	51
7.8 Paginazione	52
7.8.1 Schema di traduzione degli indirizzi	52
7.9 Hardware di paginazione	52
7.9.1 Quante pagine?	54
7.9.2 Implementazione della page table	54
7.9.3 Soluzione hardware - Translation Look-Aside Buffer	54
7.9.4 Tempo di accesso effettivo	55
7.10 Codice C e page fault	55
7.11 Page fault	56
7.11.1 Come rilevare i page fault?	57
7.11.2 Passi nella gestione dei page fault	58
7.11.3 Costo di un page fault	58
7.12 Aspetti nel richiedere paging	58
7.13 Instruction Restart	59
7.14 Strategia di sostituzione delle pagine	59
7.14.1 Passi nel richiedere Paging - Worse Case	59
7.14.2 Demand Paging Optimizations	60
7.14.3 Copy-on-Write	60
7.14.4 Cosa succede se non ci sono frame liberi?	60
7.14.5 Sostituzione di pagina di base	61
7.15 Algoritmi di sostituzione di pagine e frame	61
7.15.1 Algoritmo FIFO	61
7.15.2 Algoritmo ottimale (?)	61
7.15.3 Algoritmo LRU (Least Recently Used)	63
7.15.4 Algoritmi LRU approssimati	64
7.15.5 Miglioramento del Second-Chance Algorithm	64
7.15.6 Algoritmi basati sul conteggio	65
7.15.7 Page-Buffering Algorithms	65
7.15.8 Applications and Page Replacement	65
7.16 Allocazione dei frame ai processi	65

7.16.1	Allocazione fissa	65
7.16.2	Allocazione proporzionale	65
7.16.3	Allocazione globale vs. locale	66
7.16.4	Recupero delle pagine	66
7.16.5	Non-Uniform Memory Access	66
7.16.6	Thrashing	66
7.16.7	Paging su richiesta e thrashing	67
7.16.8	Working-Set Model	67
7.16.9	Tenere traccia del Working Set	67
7.16.10	Frequenza dei page fault	67
8	Approfondimento sulla Memoria Virtuale	68
8.1	Swapping delle Pagine	68
8.1.1	Tempo di Context Switch con lo Swapping	69
8.1.2	Swapping sui Sistemi Mobili	69
8.2	Struttura della Tabella delle Pagine	69
8.2.1	Shared Pages	70
8.2.2	Paginazione gerarchica	70
8.2.3	Tabelle delle Pagine con Hashing	71
8.2.4	Tabella delle Pagine Invertita	71
8.3	Oracle SPARC Solaris	72
8.4	Segmentazione	72
8.4.1	Paginazione vs Segmentazione	73
8.5	Example: ARM Architecture	74
9	Sistemi di Archiviazione di Massa	75
9.1	Dispositivi di Memoria Non Volatile	76
9.2	Scheduling del Disco	76
9.2.1	FCFS	77
9.2.2	SCAN - algoritmo dell'ascensore	77
9.2.3	C-SCAN	77
9.2.4	Scelta dell'Algoritmo di Scheduling del Disco	78
9.3	Pianificazione per la NVM	78
9.4	Gestione dei Dispositivi	78
9.4.1	Gestione dei dispositivi di storage	78
9.4.2	Gestione dello Spazio di Swap	79
9.4.3	Collegamento dello Storage all'Host	80
9.4.4	Archiviazione Cloud	80
9.4.5	Array Ridondante di Dischi Indipendenti - RAID	80
10	Interfaccia del File System	82
10.1	Cos'è un File?	82
10.1.1	Attributi del File	83
10.1.2	Struttura del File	83
10.2	Accesso e Operazioni sui File	83
10.2.1	Blocco dei File	83
10.2.2	Metodi di Accesso	84
10.3	File Mappati in Memoria	85
10.4	Dischi e File System	86
10.4.1	Tipi di File System	86
10.5	Struttura delle Directory	87
10.5.1	Organizzazione delle Directory	87
10.5.2	Directory a Singolo Livello	87
10.5.3	Directory a Due Livelli	88
10.5.4	Directory ad Albero	89
10.5.5	Directory a Grafo Aciclico	89

10.5.6	Directory a Grafo Generale	90
10.6	Protezione	90
11	File System Implementation	91
11.1	Struttura del File System	91
11.2	File System a Livelli	91
11.2.1	Driver dei Dispositivi	92
11.2.2	File System di Base	92
11.2.3	Modulo di Organizzazione dei File	92
11.2.4	File System Logico	92
11.3	Tipi di File System	92
11.4	Dall'API all'Implementazione	93
11.4.1	Operazioni del File System	93
11.4.2	Blocco di Controllo dei File - FCB	93
11.4.3	Il Linux iNode	93
11.4.4	Strutture del File System in Memoria	94
11.5	Implementazione delle Directory	95
11.5.1	Lista Lineare	95
11.5.2	Tabella Hash	95
11.6	Metodo di Allocazione	95
11.6.1	Metodo di Allocazione Contigua	96
11.6.2	Allocazione Collegata	96
11.6.3	Metodo di Allocazione FAT	97
11.6.4	Metodo di Allocazione Indicizzata	98
11.6.5	Prestazioni	98
11.7	Gestione dello Spazio Libero	98
11.7.1	Lista Collegata dello Spazio Libero su Disco	99
11.7.2	Lista Collegata dello Spazio Libero	99
11.8	File System in altri OS	100
11.8.1	File System di Windows	100
11.8.2	Il File System di Apple	101

Elenco delle figure

2.1	Single pointer	21
2.2	Fat pointer	21
2.3	Double pointer	21
7.1	Elaborazione in più fasi di un programma utente	48
7.2	File .dll	49
7.3	Modello di paginazione della memoria logica e fisica	52
7.4	Frame liberi	54
7.5	Page fault	55
7.6	Page fault	56
7.7	Page fault	58
7.8	Esempio di recupero delle pagine	66
7.9	PFF	67
8.1	Processo di swapping	68
8.2	Swapping con paginazione	69
8.3	Tabella delle pagine con hashing	71
8.4	Comparativa tra tabella delle pagine tradizionale e tabella delle pagine invertita	71
8.5	Segmentazione: da logico a fisico	72
8.6	Conversione degli indirizzi di memoria	74
8.7	Architettura ARM	74
9.1	Avvio dallo storage secondario in Windows	79
9.2	Strutture dati per lo swapping nei sistemi Linux	79
9.3	NAS	80
9.4	Schemi RAID	81
10.1	Tipi di file	82
10.2	Accesso sequenziale	84
10.3	Accesso diretto	84
10.4	File mappati in memoria	86
10.5	Una singola directory per tutti gli utenti	87
10.6	Directory separata per ogni utente	88
10.7	Schemi RAID	89
11.1	Il Linux iNode	94
11.2	UNIX UFS - 4 KB per blocco, indirizzi a 32 bit	94
11.3	Struttura del file system FAT32	101

Parte I

API Programming

Capitolo 1

Architettura e memoria

1.1 Obiettivi

- Comprendere lo spazio di indirizzamento virtuale
- Conoscere la differenza tra stack e heap
- Saper spiegare perché i puntatori sono pericolosi
- Conoscere quali bug tipici nascono in C/C++
- Spiegare perché Rust introduce i concetti di possesso, prestito, smart pointer

1.2 Perché studiare come è organizzata la memoria?

90% dei bug di sistema → Accessi illeciti alla memoria o mancata sincronizzazione in programmi concorrenti.

Rust offre una risposta a questi problemi

```
int* p = (int*)malloc(sizeof(int));
free(p);
*p = 5; // cosa succede?
```

1.3 Programmi e loro esecuzione

Un programma eseguibile è costituito da un insieme di istruzioni macchina, dati, valori di configurazione e controllo rappresentati come sequenze di numeri binari. Il significato delle istruzioni macchina è cablato all'interno del processore specifico per cui il programma è stato creato. Il significato delle informazioni restanti è contenuto, in parte, nelle istruzioni macchina e, per la parte restante, dipende dal sistema operativo (se c'è) o da particolari caratteristiche del processore stesso.

Per poter essere eseguito, un programma deve essere caricato all'interno della memoria accessibile al processore.

- Nei sistemi più piccoli (es., Arduino) questo avviene grazie ad hardware specifico che memorizza in modo semi-permanente quanto necessario nella memoria flash.
- Nei sistemi più grandi, un modulo del sistema operativo (**loader**) si occupa di trasferire dal disco alla memoria RAM il contenuto del file eseguibile

Una volta che il programma è presente in memoria, può essere eseguito. Il processore preleva dalla memoria, una alla volta, le istruzioni macchina del programma e provvede ad eseguirle.

L'esecuzione di un'istruzione determina quali effetti collaterali debbano avvenire e quale debba essere la prossima istruzione da eseguire. Modifiche nelle informazioni contenute nel processore stesso e/o nella

memoria. Eventuali interazioni con altre periferiche collegate al processore stesso (I/O).

Il modello base di esecuzione prevede l'alternarsi continuo di tre fasi:

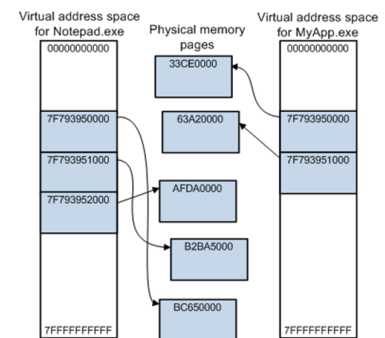
- Fetch - Un'istruzione viene trasferita dalla memoria all'interno del processore
- Decode - L'istruzione prelevata viene trasformata in comandi da eseguire
- Execute - I comandi vengono eseguiti

1.4 Spazio di indirizzamento

L'esecuzione di un programma avviene nel suo **spazio di indirizzamento**. Sottoinsieme delle celle indirizzabili, gestito dal sistema operativo. Dati, istruzioni, informazioni di controllo, ... sono memorizzati come valori al suo interno. Lo spazio di indirizzamento può essere immaginato come un enorme array di byte, che possono essere letti individualmente, a coppie, a gruppi di 4, 8 alla volta, in base al parallelismo del processore (8, 16, 32 o 64 bit).

Il numero di celle **potenzialmente** presenti nello spazio di indirizzamento dipende dal processore. Il numero di celle **effettivamente** presenti, è limitato dalla quantità di RAM disponibile ed è controllato dal sistema operativo.

Nel caso di CPU Intel x64, il processore opera a 64 bit. Ma lo spazio di indirizzamento si estende tra 0 e $2^{48} - 1$ (scelta di implementazione). All'interno di questo, però, solo una piccola parte di indirizzi è effettivamente presente.



Gli indirizzi che un programma utilizza non corrispondono - in generale - all'effettiva posizione che un dato assume in memoria

- Il codice fa riferimento ad **indirizzi virtuali**
- L'hardware utilizza **indirizzi fisici**

Un apposito blocco hardware presente nel processore (**MMU - Memory Management Unit**) si occupa di tradurre l'indirizzo virtuale in indirizzo fisico.

Tale traduzione garantisce che:

- Programmi diversi contemporaneamente in esecuzione non possano interferire tra loro
- Consente di controllare quali operazioni siano possibili (lettura, scrittura, esecuzione)
- Permette inoltre di scrivere programmi che manipolano dati più grossi della memoria fisica effettivamente presente

1.5 Effetti pratici

- Segmentation fault
 - Tentativo di accedere ad indirizzo non mappato o senza gli adeguati permessi
 - Il processore blocca l'esecuzione e rimanda al sistema operativo (che termina brutalmente il programma)
- Page fault
 - Tentativo di accedere ad un indirizzo appartenente ad un blocco il cui contenuto è stato salvato su disco

- Il processore richiede l'intervento del sistema operativo che ricerca una pagina fisica libera e vi trasferisce il contenuto del blocco presente nel file di paginazione
- Al termine il processore ritenta l'accesso e procede normalmente
- Trashing
 - Se il programma cerca di accedere rapidamente ad una grande quantità di dati paginati, il sistema passa più tempo a spostare pagine tra disco e memoria fisica di quanto ne possa dedicare all'esecuzione vera e propria

1.6 Gerarchia di memoria

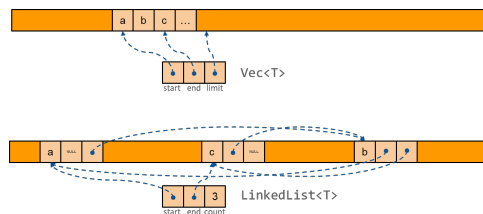
Nei sistemi non elementari, il processore **non legge/scrive** direttamente dalla memoria RAM. Tra i due elementi viene interposta la memoria **cache**. Questa ha il compito di velocizzare l'accesso alle informazioni cercando di anticipare il bisogno di informazioni da parte della CPU.

Si basa sul **principio di località** dei programmi: se è stato appena fatto accesso all'indirizzo I , è molto probabile che venga fatto subito dopo accesso all'indirizzo $I + \Delta$ (con Δ molto piccolo).

La memoria cache si basa su dispositivi il cui tempo di accesso è molto più rapido di quello offerto dalla memoria RAM tradizionale, ma dotati di una capacità molto inferiore (pochi KB/MB).

Per questo $\text{Vec}\langle T \rangle$ è quasi sempre meglio di $\text{LinkedList}\langle T \rangle$

1.7 Località



1.8 Esecuzione e programmi di alto livello

Utilizzando linguaggi di programmazione ad alto livello, i dettagli legati al meccanismo di esecuzione sono normalmente invisibili.

Tuttavia, per poter garantire il livello di astrazione necessario, essi devono introdurre delle sovrastrutture che impattano sul codice che viene scritto dal programmatore... e introducono un insieme di **vincoli** e **restrizioni** di cui è necessario essere pienamente coscienti.

1.9 Modello di esecuzione di un programma

Ogni linguaggio di programmazione propone uno specifico modello di esecuzione, un insieme di comportamenti che attuati dall'elaboratore a fronte dei costrutti di alto livello del linguaggio.

Tale modello per lo più non corrisponde a quello di un dispositivo reale, come fa il processore ad eseguire solo Fetch, Decode ed Execute? Utilizzando strutture dati intermedie e da un insieme di run-time library. È compito del compilatore introdurre uno strato di adattamento che implementi il modello nei termini offerti dal dispositivo sottostante.

Questo strato è composto, da un lato, da una specifica modalità di traduzione dei costrutti di alto livello in istruzioni macchina e, dall'altro, da un insieme di funzionalità di supporto offerte sotto forma di libreria di supporto all'esecuzione (run-time library).

1.10 Livelli di astrazione

La compilazione trasforma il programma sorgente in un nuovo programma dotato di un modello di esecuzione ed un insieme di istruzioni più semplice. Che può essere eseguito da una macchina fisica (CPU) o subire un'ulteriore trasformazione (esecuzione virtuale)



1.11 Librerie di esecuzione

Offrono agli applicativi meccanismi di base per il loro funzionamento

- Supportano le astrazioni del linguaggio di programmazione
- Forniscono un'interfaccia uniforme tra i diversi S.O. per le funzioni ad essi demandate

Costituite da due tipi di funzioni

- Alcune, invisibili al programmatore, sono inserite in fase di compilazione per supportare l'esecuzione (controllo dello stack, copia di aree di memoria ...)
- Altre offrono al programmatore funzionalità standard, gestendo opportune strutture dati ausiliarie e/o richiedendo al S.O. quelle non altrimenti realizzabili (malloc o fopen)

1.12 Modello di esecuzione nei linguaggi C e C++

In C/C++ sono pensati come se fossero gli unici utilizzatori di un elaboratore, completamente dedicato loro. Le uniche interazioni si riducono al più all'uso di risorse persistenti come il file system o le connessioni di rete .

L'isolamento è reso possibile dal meccanismo di indirizzamento virtuale.

Un programma C/C++ assume di poter accedere a qualsiasi indirizzo di memoria presente nello spazio di indirizzamento, all'interno del quale può leggere o scrivere dati o dal quale può eseguire codice macchina. Ma sappiamo che invece lo spazio di indirizzamento è:

- Limitato
- Non contiguo
- Non tutto lo spazio consente ogni tipo di operazione

1.13 Esecuzione sequenziale sincrona, senza limitazioni

Un programma C/C++ è formato da un insieme di istruzioni eseguite, una per volta, nell'ordine indicato dal programmatore. Non ci sono limiti sul numero di istruzioni da eseguire, sul tempo richiesto alla loro esecuzione, né sulla memoria necessaria .

In realtà, tali limiti esistono e condizionano la scrittura dei programmi. Da qui nasce la necessità di un modello di gestione della memoria sicuro .

All'interno del flusso principale di esecuzione è possibile attivare flussi di elaborazione secondari, detti **thread**. Questi abilitano l'esecuzione concorrente (effettivamente parallela se la CPU è multicore) e introducono un ordine di grandezza nel livello di complessità della scrittura dei programmi. È il S.O. che si occupa di gestire tutti questi flussi di esecuzione indirizzandoli nei vari core fisici.

1.14 Flusso di esecuzione

Il programma è costituito, come minimo, da un flusso di esecuzione il cui punto di partenza è predefinito:

- La funzione `main()` nel linguaggio C
- I costruttori delle variabili globali in C++, seguiti dalla chiamata alla funzione `main()`

Vi sono inoltre due strutture dati:

- Lo stack che permette di gestire chiamate annidate tra funzioni. Essa **supporta la ricorsione** e l'uso di **variabili locali**. In C++, supporta anche la gestione strutturata delle eccezioni
- Lo heap offre supporto per gestire dinamicamente l'utilizzo della memoria. Secondo logiche non strettamente correlate al procedere dell'esecuzione

1.15 Stack

Blocco di memoria allocato automaticamente all'avvio di un programma, circa 1 MB, al cui interno sarà possibile salvare:

- Variabili locali e temporanee
- Gli argomenti passati ad ogni invocazione di funzione
- Valori di ritorno
- Traccia la storia delle chiamate a funzione

Viene utilizzato a partire da un estremo, espandendosi verso l'estremo opposto ogni volta che avviene una chiamata e contraendosi verso l'inizio quando una funzione ritorna.

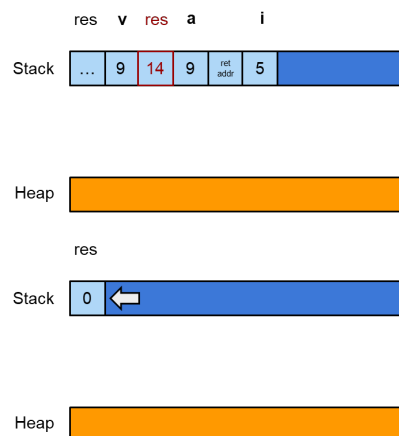
Poiché ha dimensione finita, limita la profondità massima di ricorsione. Così come la dimensione totale delle variabili locali complessivamente utilizzabili.

Se, a seguito di un numero troppo elevato di chiamate o all'allocazione di variabili troppo grandi, si espande oltre il proprio limite, il sistema operativo (o il processore) interviene terminando l'esecuzione del programma generando un Stack overflow.

Nota: Stack = automatico, veloce, tempo di vita deterministico

```
int f(int a) {
    int i = 5;
    if (a>0) {
        return a+i;
    } else {
        return a-1;
    }
}

int main() {
    int v = 9;
    v = f(v);
    return 0;
}
```



L'allocazione ed il rilascio sullo stack sono estremamente efficienti, grazie alla particolare politica di espansione / contrazione adottata.

Per contro, la durata dei valori memorizzati al suo interno è strettamente correlata alla durata dell'invocazione di una funzione. Non è possibile memorizzare nello stack un dato che duri più a lungo della funzione in cui è stato allocato. Per questo motivo, il chiamante di una funzione ha la responsabilità di pre-allocare lo spazio in cui dovrà essere memorizzato il risultato prodotto dalla funzione chiamata.

Inoltre, poiché lo spazio complessivo dello stack è definito a priori, non è possibile memorizzare un dato la cui dimensione superi tale spazio. In generale, occorre limitare la dimensione massima del dato allocato ad una grandezza compatibile con il livello di profondità di chiamate e la dimensione degli altri dati che devono essere memorizzati complessivamente nello stack.

1.16 Heap

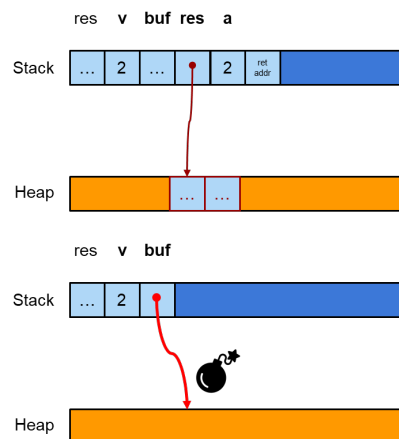
Tutte le volte in cui un dato ha un ciclo di vita **non collegato al tempo di esecuzione** della funzione in cui è stato creato o la cui dimensione è **cospicua** oppure **non nota** in fase di compilazione, occorre richiederne la memorizzazione in una struttura a parte. I linguaggi C e C++ offrono un'apposita area, chiamata Heap o Free Store, all'interno della quale è possibile allocare e rilasciare blocchi di dimensione arbitraria (entro il limite della memoria disponibile).

A differenza di quanto avviene per lo stack, le aree allocate sullo heap non sono associate, a livello di linguaggio sorgente, ad un nome di variabile bensì si accede al loro contenuto esclusivamente tramite **puntatori**. Utilizzando l'operatore di dereferenziazione, *, posso accedere al contenuto memorizzato in memoria.

Questo spazio concesso dal S.O. è un prestito, dunque è responsabilità del programmatore rilasciare, prima o poi, la memoria allocata. Altrimenti si verificherà una perdita di memoria (memory leak) che, se reiterata, porta il sistema operativo a distruggere il processo.

```
int* f(int a) {
    if (a>0) {
        return new int[a];
    } else {
        return nullptr;
    }
}

int main() {
    int v = 2;
    int* buf = f(v);    //process buf
    delete[] buf;
    return 0;
}
```



Il puntatore è probabile che sia buono, ma non lo è, non gli è stato assegnato null, il linguaggio non te lo impone.

1.17 Organizzazione dello spazio di indirizzamento

- Codice eseguibile
 - Contiene le istruzioni in codice macchina
 - Accesso in lettura ed esecuzione
- Costanti
 - Accesso in sola lettura
- Variabili globali
 - Accesso lettura/scrittura
- Stack

- Contiene indirizzi e valori di ritorno, parametri e variabili locali
- Accesso lettura/scrittura
- Heap
 - Insieme di blocchi di memoria disponibili per l'allocazione dinamica
 - Gestiti tramite funzioni di libreria (`malloc`, `new`, `free`, ...) che li frammentano e ricompattano in base
 - alle richieste del programma

Nel sistema operativo Linux, è facile verificare come sia organizzato lo spazio di indirizzamento di un programma in esecuzione (processo). Ogni processo è identificato univocamente da un numero intero (PID) e con il comando `ps -ef`, il sistema operativo crea, per ciascun processo attivo, un file virtuale denominato `/proc/<pid>/maps` che descrive lo spazio di indirizzamento con informazioni sui diversi segmenti al suo interno.

1.18 Ciclo di vita delle variabili

Il modello di esecuzione dei linguaggi C/C++ distingue diverse tipologie di variabili

Globali locali dinamiche

Ciascuna classe ha un proprio ciclo di vita: Intervallo di tempo in cui è garantito l'accesso alle informazioni contenute al loro interno.

1.19 Variabili globali e locali

- Le variabili globali hanno un indirizzo fisso, determinato dal compilatore e dal linker. Sono accessibili in ogni momento e all'avvio del programma, contengono l'eventuale **valore di inizializzazione**.
- Le variabili locali hanno un indirizzo relativo alla cima dello stack. Il ciclo di vita coincidente con quello della funzione/blocco in cui sono dichiarate. Il valore **iniziale è casuale**.

1.20 Variabili dinamiche

Le variabili dinamiche hanno un indirizzo assoluto, determinato in fase di esecuzione. Sono accessibili solo tramite puntatori, è il programmatore a controllarne il ciclo di vita e il valore iniziale può essere inizializzato o meno.

L'uso di questo tipo di variabili presuppone un'infrastruttura di supporto che offra meccanismi di allocazione e di rilascio fornita dalla libreria di esecuzione e dal S.O.

1.21 Allocazione della memoria

La libreria standard C offre vari meccanismi per ottenere l'indirizzo di un blocco allocato dinamicamente

```
void *malloc(size_t s)
void *calloc(int n, size_t s)    //per gli array
void *realloc(void* p, size_t s) //rialloca aggiungendo memoria
```

In caso di fallimento, viene restituito `NULL`, tranne nella `realloc(...)` dove il blocco originale non viene toccato e il puntatore `p` resta valido.

In C++ viene definito il costrutto: `new NomeTipo\{argomenti...\}`

1.22 Rilascio della memoria

Opportune funzioni di libreria permettono di restituire i blocchi precedentemente allocati. Poiché ogni funzione di allocazione mantiene le proprie strutture dati, occorre che un blocco sia rilasciato dalla funzione duale di quella con cui è stato allocato:

- `malloc(...)` / `free(...)`
- `new` / `delete`
- `new ...[num\_elements]` / `delete[ ]`

Se il blocco non viene rilasciato si crea una perdita di memoria, che, a lungo andare, provoca l'esaurimento dello spazio di indirizzamento.

Se il blocco viene rilasciato più volte o viene rilasciato con la funzione sbagliata si corrompono le strutture dati degli allocatori, con conseguenze imprevedibili.

1.23 Puntatori

Le variabili sullo heap possono essere gestite solo tramite puntatori perché la loro posizione non è nota a priori.

Vi è un problema, potentissimi per implementare di una vasta gamma di algoritmi come liste, grafo, etc., ma sono estremamente pericolosi. I puntatori sono intrinsecamente ambigui implica che il programmatore gestisca esplicitamente l'allocazione ed il rilascio dei blocchi al loro interno e ne manipoli il contenuto attraverso l'uso dei puntatori.

Il loro utilizzo espone infatti il programmatore ad una vasta gamma di errori, le cui conseguenze portano - nella maggior parte dei casi - a **comportamenti non definiti** (undefined behavior), le cui conseguenze sono disastrose.

1.24 Il costo del rilascio

I linguaggi gestiti, come Java, JavaScript, Python, C#, etc. gestiscono automaticamente il rilascio dei blocchi di memoria utilizzando il Garbage Collector. Per farlo, tengono traccia di chi punta a chi altro. Se, a partire dalle variabili presenti nello stack, un blocco non risulta più raggiungibile, questo può essere liberato.

Per poter applicare questo algoritmo occorre "fermare il mondo". Ovvero **sospendere temporaneamente l'esecuzione del programma intero** per poter osservare una situazione statica e procedere con la compattazione dello heap. Questo può comportare lo spostamento di blocchi e il cambiamento del valore contenuto nei rispettivi puntatori.

L'operazione di rilascio può comportare pause non deterministiche. Talvolta incompatibili con contesti di esecuzione in tempo reale.

1.25 Puntatori in C/C++

Permettono l'accesso diretto ad un blocco di memoria, tale blocco può appartenere ad altri oggetti:

```
int A=10;
int* pA = &A;
```

o essere allocato allo scopo `int* pB = new int{24}`
Possono essere esplicitamente resi invalidi:

- Il valore `0`
- La macro `NULL` $\Rightarrow ((void *)0)$
- La parola-chiave `nullptr` (C++11 e superiori)

Quando si usano i puntatori, occorre stabilire quali responsabilità/permessi sono associati al loro uso.

1.26 Le ambiguità dei puntatori in C/C++

- Contiene un indirizzo valido?
- Quanto è grosso il blocco puntato?
- Fino a quando è garantito l'accesso?
- Se ne può modificare il contenuto?
- Occorre rilasciarlo?
- Lo si può rilasciare o altri conoscono lo stesso indirizzo?
- Viene usato come modo per esprimere l'opzionalità del dato?

Nota: In C/C++ il programmatore è responsabile; in Rust la responsabilità è codificata nel sistema dei tipi

1.27 Problemi legati ai puntatori

In C, la gestione della memoria è totalmente affidata al programmatore

- Non ci sono supporti sintattici per automatizzare il ciclo di vita del blocco puntato
- Né meccanismi per associare una specifica semantica al puntatore

In C++ sono state introdotte nuove astrazioni (riferimenti, smart pointer) che sollevano il programmatore da alcune responsabilità, facilitando la creazione di programmi più robusti. Ma richiedono in ogni caso una comprensione approfondita del problema e l'uso attento e coerente dei puntatori.

1.28 Responsabilità del programmatore

- Limitare gli accessi ad un blocco
 - Nello spazio: non accedere alle locazioni che lo precedono o che lo seguono
 - Nel tempo: non accedere al blocco al di fuori del suo ciclo di vita
- Non assegnare a puntatori valori che corrispondono ad indirizzi non mappati
 - Possibile nel caso di cast o di assegnazione improprie
- Rilasciare tutta la memoria dinamica allocata
 - Una e una sola volta, usando la funzione duale di quella usata per la sua allocazione

1.29 Rischi

Accedere ad un indirizzo quando il corrispondente ciclo di vita è terminato, ha **effetti imprevedibili**:

- Dangling pointer
- La memoria indirizzata può essere inutilizzata, in uso ad altre parti del programma o non mappata

Non rilasciare la memoria non più in uso, **spreca risorse** del sistema

- Memory leakage
- Se si continua ad allocare senza mai rilasciare, si può saturare lo spazio di indirizzamento

Rilasciare la memoria più volte **corrompe le strutture** usate dallo heap, double free.

Se si assegna ad un puntatore un indirizzo non mappato nello spazio di indirizzamento e si usa quel puntatore, viene generata una interruzione del processore. Il S.O. intercetta tale interruzione e termina il processo.

Se non si inizializza un puntatore e lo si usa, *Wild pointer*, il suo contenuto potrebbe puntare ovunque. Può causare una violazione di accesso oppure corrompere un'area di memoria in uso ad altre parti del programma.

1.30 Problemi dei puntatori in C/C++

Problema	Conseguenza
dangling / wild pointer	crash / UB
double free	corruzione heap
leak	esaurimento della memoria
data race	risultati casuali

1.31 Le risposte di Rust

Problema C / C++	Strumento Rust
dangling / wild pointer	borrow checker + tempo di vita
double free	gestione del possesso
leak	pattern RAII
nullptr	tipo Option<T>
gestione del rilascio	tipi Box<T> / RC<T> / Arc<T>

1.32 Gestire i puntatori

Chi alloca un blocco di memoria è **responsabile** di mettere in atto un meccanismo che ne garantisca il successivo **rilascio**. Viene detto "possessore" del blocco.

Cosa capita se si effettua una copia di un puntatore?

Il secondo puntatore si trova coinvolto, suo malgrado, nel ciclo di vita del dato. Occorre introdurre un meccanismo per gestire efficacemente la semantica di un puntatore.

1.33 Possesso della memoria

Il vincolo di rilascio risulta problematico per via dell'**ambiguità** del concetto di puntatore all'interno del linguaggio. Dato un indirizzo non nullo, non è possibile distinguere se sia **valido o meno**, né **a quale area appartenga**, né se occorra o meno **rilasciarlo**.

Ogni volta in cui si alloca un blocco dinamico e se ne salva l'indirizzo in una variabile puntatore, quella variabile diventa proprietaria del blocco: ha la responsabilità di liberarlo.

Non tutti i puntatori posseggono il blocco a cui puntano. Se ad un puntatore viene assegnato l'indirizzo di un'altra variabile, la proprietà è della libreria di esecuzione.

Il problema si complica se un puntatore che possiede il proprio blocco viene copiato. Quale delle due copie è responsabile del rilascio?

1.34 Dipendenze

Quando si introducono strutture dati complesse, è comune che tali strutture debbano appoggiarsi a blocchi di memoria ausiliari in cui memorizzare le informazioni che esse gestiscono.

Un oggetto di tipo `std::vector<T>`, in C++, incapsula un array dinamico di dati di tipo generico `T`: tali dati possono essere aggiunti, rimossi, sostituiti, ... tramite opportuni metodi sull'oggetto stesso. Tali blocchi ausiliari sono di proprietà dell'oggetto: vengono gestiti dai suoi metodi e devono essere rilasciati quando l'oggetto viene distrutto.

Analogamente, altre strutture dati possono mantenere riferimenti a oggetti del sistema operativo (handle a file, socket, timer, ...). Anche in questo caso le risorse corrispondenti risultano di proprietà dell'oggetto che le gestisce.

Collettivamente, questi tipi di dato prendono il nome di **dipendenze**.

Il linguaggio C non offre nessun supporto sintattico per la gestione delle dipendenze e lascia al programmatore la completa responsabilità della loro gestione.

Al contrario, il linguaggio C++ offre, per gli oggetti di tipo *class*, *struct* e *union* (!), la possibilità di specificare particolari funzioni membro dette rispettivamente costruttori e distruttori.

- Un costruttore è una funzione membro che ha il compito di inizializzare un oggetto, provvedendo ad acquisire - se necessario - le sue dipendenze
- Il distruttore è una funzione membro che viene invocata automaticamente dal compilatore quando l'oggetto raggiunge la fine della propria esistenza ed ha il compito di rilasciare correttamente le dipendenze possedute dall'oggetto stesso

1.35 Ma non basta scrivere un programma "giusto"?

Sì, basterebbe se fosse possibile essere certi che lo è! In realtà le cose sono molto più complesse.

Raramente i programmi sono scritti da una singola persona. Per lo più si appoggiano a librerie scritte da altri che hanno fatto opportune assunzioni su come debbano essere utilizzate, ma non è detto che tali assunzioni siano rispettate né, talvolta, conosciute. Spesso si lavora in gruppo e la comunicazione è imperfetta.

Al crescere della dimensione del programma, la quantità di particolari a cui occorre badare cresce molto più in fretta. Ed è estremamente probabile dimenticarsi delle proprie assunzioni.

1.36 Perché questo problema non si verifica in altri linguaggi?

Nella maggior parte dei linguaggi di alto livello il problema della gestione della memoria non si pone, pur offrendo, a vario titolo, il concetto di puntatore.

Java, C#, Python, JavaScript, ... , basano il proprio meccanismo di rilascio su un algoritmo di Garbage Collection. Libera il programmatore dalla responsabilità di rilasciare esplicitamente la memoria dinamica allocata, in cambio della perdita di controllo su **quando** e **come** tale rilascio avvenga.

La maggior parte di tali linguaggi basa la propria strategia di gestione della memoria su una combinazione dei seguenti algoritmi:

- Reference counting
- Mark and Sweep

1.37 Confronto

Gestione manuale in C/C++	Gestione automatica (Java/C#/Python/Js)
Il rilascio avviene invocando la funzione free() o l'operatore delete	Il rilascio è eseguito automaticamente dal garbage collector
Il momento in cui avviene il rilascio è controllato dal programmatore	Il momento in cui avviene il rilascio è controllato dal garbage collector, che viene eseguito periodicamente
In C++, gli oggetti dispongono di un distruttore che gestisce il rilascio esplicito delle dipendenze	Il concetto di distruttore non è parte del linguaggio (ecc. Java, con il metodo <i>finalize()</i> ora deprecato)
Il rilascio non comporta tempi supplementari di attesa, che sarebbero incompatibili con i sistemi in tempo reale	Quando si attiva la garbage collection, il programma si arresta fino al termine dell'algoritmo, mettendo a rischio il funzionamento di un sistema in tempo reale
Si possono verificare memory leak, doppi rilasci, dangling pointer, wild pointer	Non possono verificarsi errori sulla gestione della memoria

Capitolo 2

Possesso

2.1 Possesso

In Rust ogni valore introdotto nel programma è **posseduto** da una ed una sola variabile e il suo valore salvato in memoria.

- Il **Borrow Checker** verifica formalmente questo fatto per ogni punto dell'esecuzione del programma
- Ogni violazione porta ad un **fallimento** della compilazione del programma con conseguente errore e spiegazione di quest'ultimo.

Possedere un valore significa essere responsabile del suo **rilascio**.

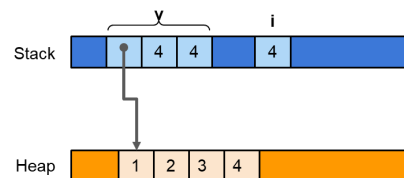
In particolare se il valore contiene una risorsa quale un puntatore a memoria dinamica, handle di file o socket, etc, questa deve essere liberata. Una volta liberata deve essere restituita al proprietario cui molto spesso è il Sistema Operativo.

Il rilascio avviene quando la variabile che possiede esce dal proprio scope sintattico o quando viene **assegnata** ad un nuovo valore.

Rust offre un meccanismo di drop mediante il quale è possibile associare azioni arbitrarie da eseguire prima che sia liberata la memoria. Il rilascio può essere rimandato ad un secondo momento se il contenuto della variabile viene trasferito (mosso) in un'altra variabile, la quale diventa responsabile del suo rilascio.

Nota: Questo meccanismo rende l'utilizzo del Garbage collector, presente in linguaggi di programmazione come Java o Python, non necessario.

```
fn main() {  
    let mut v = Vec::with_capacity(4);  
    // v possiede il vec  
  
    for i in 1..=5 {  
        v.push(i);  
    }  
  
    println!("{:?}", v);  
}
```



Inserendo valori all'interno del vettore, lo spazio precedentemente allocato viene via via riempito. Se necessario il blocco viene riallocato per fare spazio ad un maggior numero di elementi.

Finché le variabili sono in scope, *i* e *v*, le risorse che posseggono sono accessibili. Una volta usciti dal proprio scope la variabile si occupa dello smaltimento della porzione di stack, ed eventualmente heap, precedentemente allocato seguendo le informazioni contenute in Drop.

2.2 Movimento

Quando una variabile viene inizializzata, essa prende il possesso del relativo valore. Se ad una variabile mutable viene assegnato un nuovo valore, quello precedentemente posseduto viene rilasciato e la variabile diventa proprietaria del nuovo valore.

```
let mut b = a      f(a)      return a
```

In questi tre casi, assegnazione, passaggio ad una funzione e ritorno di una variabile, in Rust, succede qualcosa che in altri linguaggi di programmazione non succede: il valore viene **mosso**, il possesso viene passato dalla variabile originale alla destinazione.

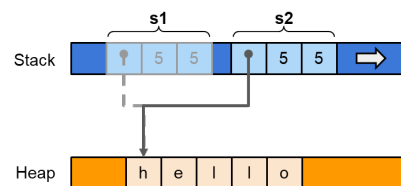
Questo porta ad alcune particolarità:

- La variabile originale cessa di possedere il valore, ovvero non è più responsabile della sua distruzione ed il possesso passa alla variabile di destinazione, o parametro della funzione invocata
- La variabile originale rimane allocata fino a quando non termina la sua visibilità, ovvero al chiusura del blocco in cui è stata definita
- Accessi in **lettura** porteranno ad **errori di compilazione**
- Accessi in **scrittura** alla variabile originale avranno successo e ne riabiliteranno la lettura (si crea un nuovo oggetto)
- La variabile destinazione conterrà una copia bit a bit del valore originale, ammesso che il compilatore non riesca a riusare i dati originali al loro posto

```
let mut s1 = "hello".to_string();
println!("s1: {}", s1);

let s2 = s1;
println!("s2: {}", s2);

//s1 non è più accessibile
...
s1 = "world".to_string();  \\s1 ritorna
↪ accessibile
```



Il movimento in Rust è la soluzione al problema del doppio possesso di parti di aree di memoria. Fisicamente nessuno ha cancellato l'area di memoria ma il compilatore marca *s1* come inaccessibile.

Il movimento previene l'ambiguità che introduce C/C++: un puntatore è valido/non valido, dobbiamo rilasciare noi la memoria o ci pensa qualcun altro? In Rust è solo il possessore, univoco, della variabile ad occuparsene, i precedenti possessori altro non fanno che contrarre lo stack per liberare lo spazio occupato.

```
let mut s1 = "hello".to_string();
let mut s2 = s1 \\s2 possiede "hello"
...
let mut s1 ...  \\Questo s1 nasconde il precedente s1
```

2.3 Copia

Alcuni tipo, tra cui quelli numerici, sono definiti **copiabili**.

Questa proprietà è definita attraverso il tratto **Copy**. Quando una valore assegnato ad un'altra variabile o usato come argomento in una chiamata a funzione, il valore originale rimane accessibile in **sola lettura**.

Questo è possibile quando il valore contenuto non costituisce una risorsa che richiede altre azioni di rilascio come ad esempio possono essere File o Socket.

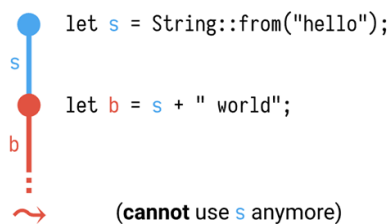
Se *Vec* e *String* posseggono il tratto *Drop*, etichetta per lo smaltimento specifico dato che sono puntatori a zone di heap, *Int*, *Bool*, etc. assumono il tratto *Copy*.

Nota: Drop e Copy sono mutualmente esclusivi. Posso esistere anche tipi né Copy né Drop come per esempio gli oggetti Thread.

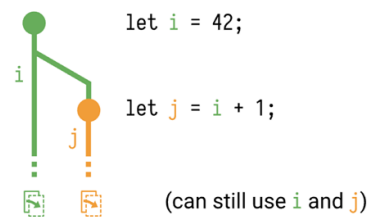
I tipo semplici e le loro combinazioni, tuple e array di numeri, sono copiabili. Sono copiabili anche i riferimenti a valori non mutabili.

Viceversa i riferimenti a valori mutabili NON sono copiabili.

→ **move** (for types that do not implement Copy)



→ **copy** (for types that do implement Copy)

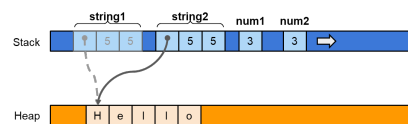


Nota: Dal punto di vista del codice genero, non cambia nulla tra copia e movimento. L'istruzione di assegnazione o passaggio come argomento comporta la duplicazione, bit a bit, del valore originale. In caso di copia il Borrow Checker non impedisce l'ulteriore accesso in sola lettura al dato originale.

2.3.1 Copia e Movimento

Copia e Movimento diventano esattamente la medesima istruzione assembly *mov*, quello che cambia sono le tabelle e le decisioni prese dal compilatore.

```
let string1 = "Hello".to_string();
let string2 = string1;
//da qui in poi, string1 è inaccessibile in
↳ lettura
//a meno che non venga riassegnato
let num1: i32 = 3;
let num2 = num1; //nessun vincolo su num1!
```



2.4 Clonazione

Alcune volte non vogliamo cedere il possesso ma solamente avere una copia e se un tipo implementa il tratto *Clone* può essere duplicato invocando il metodo *clone()*.

A differenza di copia e movimento, la clonazione può implementare una **copia profonda** dei valori. Questo implica un elevato costo di clonazione se il tipo risulta occupare molta memoria.

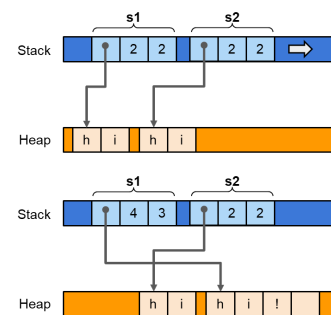
L'implementazione dell'operazione di clonazione, a differenza di copia e movimento che sono sotto il controllo esclusivo del compilatore ed è basata sull'invocazione della funzione *memcpy(...)*, è modificabile dal programmatore.

Nota: Affinché un tipo possa implementare il tratto *Copy*, occorre che implementi anche il tratto *Clone*. Tuttavia, non occorre che le due implementazioni coincidano.

```
let mut s1 = "hi".to_string();
let s2 = s1.clone();

s1.push('!');

println!("s1: {}", s1); //hi!
println!("s2: {}", s2); //hi
```



2.5 Box<T> - Puntatore con possesso

Una variabile locale è SEMPRE allocata sullo stack. Questo si estende di una quantità pari alla dimensione del valore che deve essere memorizzato nella variabile. Quando la variabile esce dal proprio scope sintattico, lo stack si contrae ed il valore viene rilasciato.

In alcune situazioni:

- Non è nota la dimensione a priori
- Occorre prolungare il tempo di vita del valore oltre al blocco sintattico in cui è definito
- La dimensione è maggiore dello stack stesso

l'oggetto deve essere allocato sullo heap, utilizzando il tipo generico **Box<T>**. Una variabile di questo tipo contiene un puntatore al valore.

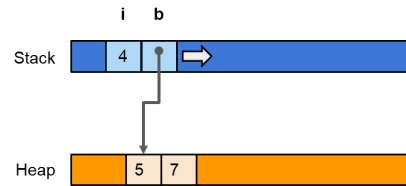
Per allocare un valore con il tipo Box si scrive:

```
let b = Box::new(v);
```

dove v è un qualsiasi valore. L'istruzione precedente definisce una variabile b la quale è un puntatore ad un area di memoria sullo heap contenente il valore v .

Si accede al valore contenuto con l'espressione $*b^1$, e se la variabile è definita come mutabile è possibile modificare il contenuto a cui si punta.

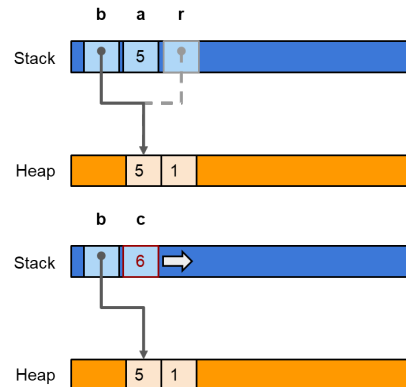
```
fn useBox() {
  let i = 4;
  let mut b = Box::new( (5, 2) );
  (*b).1 = 7;
  println!("{}", *b); // (5,7)
}
```



Quando l'esecuzione del programma raggiungerà la fine del blocco di codice in cui la variabile b è stata definita, il blocco verrà rilasciato a patto che il contenuto di b , il puntatore al blocco sia stato **mosso** in un'altra variabile.

```
fn makeBox(a: i32) -> Box<(i32,i32)> {
  let r = Box::new( (a, 1) );
  return r;
}
```

```
fn main() {
  let b = makeBox(5);
  let c = b.0 + b.1;
}
```



¹Si può scrivere pure $b.1$, il compilatore lo de-referenzia in autonomia $(*b).1$

2.6 Riferimenti

Per risolvere i problemi di accesso, Rust introduce diverse forme di puntatori, volte a rendere espliciti responsabilità e diritti di chi le maneggia. Il borrow checker si occupa di garantire che il codice sia scritto in conformità con tali regole e impedisce la compilazione in caso contrario.

Un **riferimento** è un puntatore in sola lettura ad un blocco di memoria **posseduto da un'altra variabile**.

- Permette di accedere ad un valore senza trasferirne la proprietà
- Il riferimento può esistere solo mentre esiste la variabile che possiede il dato
- non serve utilizzare *, quando si accede al valore tramite l'operatore .

```
let point = (1.0, 0.0); //point possiede il valore
let reference = &point; //reference PUÒ accedere al valore in lettura
//finché esiste point
println!("{}", reference.0, reference.1);

point = (2.0, 3.0); //tentativi di accesso a reference generano errore
```

2.6.1 Riferimenti e prestiti

Un riferimento prende in prestito, borrows, in sola lettura, l'indirizzo di memoria in cui esiste il valore.

Fino a che il riferimento è accessibile, non è possibile modificare il valore, né tramite il riferimento, il quale è in sola lettura, né tramite la variabile che possiede il valore.

È possibile creare ulteriori riferimenti a partire dal dato originale o da altri riferimenti ad esso.

- I riferimenti sono **copiabili**: viene duplicato il puntatore
- Il compilatore, attraverso il BC, garantisce che un riferimento punti sempre ad un dato valido
- Finché esiste almeno un riferimento, il dato originale **non potrà essere né modificato né distrutto**

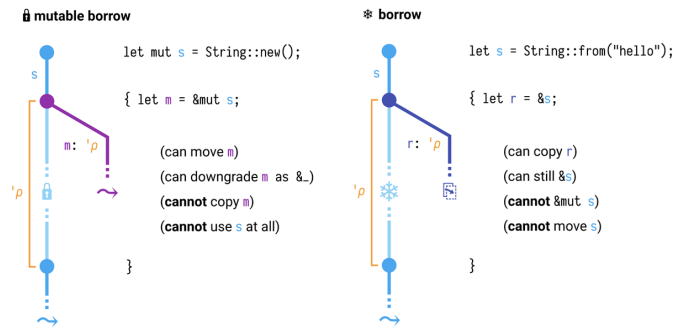
2.6.2 Riferimenti mut - mutual exclusion

A partire da una variabile che possiede un valore è possibile estrarre **un solo riferimento mutabile** per volta. `let r = &mut v;`

Mentre esiste un riferimento mutabile

- Non possono esistere riferimenti semplici, in sola lettura
- Non è possibile modificare né muovere la variabile che possiede il valore

È possibile creare un riferimento mutabile solo se la variabile che possiede il dato è a sua volta dichiarata come mutabile.



2.6.3 Riferimenti: disposizione in memoria

I riferimenti sono implementati diversamente in base al tipo puntato

- Puntatori semplici, se il compilatore conosce la dimensione del dato puntato
- Puntatore + dimensione (fat pointer), se la dimensione del dato è solo nota in fase di esecuzione. Nelle stringhe tolgo il problema di inserire il terminatore.
- Puntatore doppio, se il tipo di dato puntato è noto solo per l'insieme di tratti che implementa. Esempi File o Socket.

```
let i: i32 = 10;
let r1: &i32 = &i;
```

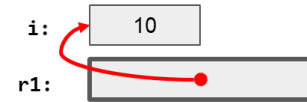


Figura 2.1: Single pointer

```
let a: [i32;3] = [1,2,3];
let r2: &[i32] = &a;
```

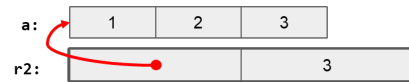


Figura 2.2: Fat pointer

```
let f: File = File::create("test.txt");
let r3: &dyn Write = &f;
```

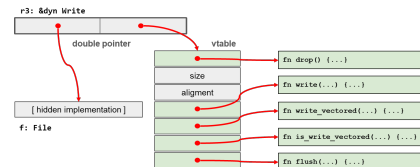


Figura 2.3: Double pointer

2.7 Tempo di vita dei riferimenti

Il BC garantisce che tutti gli accessi ad un riferimento avvengano solo in un intervallo di tempo, o righe, compreso in quello in cui il dato esiste. Serve ad impedire il fenomeno dei **dangling pointer**.

L'insieme delle righe in cui si fa accesso al riferimento costituisce il suo **lifetime**. Tale info è mantenuta dal compilatore assieme ad altre informazioni, quale il tipo, e non ha alcuna rappresentazione in fase di esecuzione.

Sebbene il tempo di vita in molte situazioni possa essere omesso, il tempo di vita di un riferimento può essere espresso nella firma del tipo. Questo aiuta sia il programmatore sia il compilatore per il mantenimento dell'informazione.

- `&'a NomeTipo`, dove `a` è un identificativo qualsiasi e il simbolo `'` serve a quantificarlo come tempo di vita
- Tale notazione è utile in quelle situazioni in cui occorre imporre vincoli sulla durata relativa dei tempo di vita di due o più riferimenti

Se un riferimento deve durare l'intera durata del programma viene indicato con `&'static NomeTipo`.

Nota: Una stringa espressa in formato letterale ("Some string") ha come tipo `&'static str`, in quanto la sequenza di caratteri viene allocata dal compilatore nella sezione delle costanti e non viene mai rilasciata.

Per essere lecito, il tempo di vita di un riferimento deve essere contenuto, minore, nel tempo di vita del valore a cui punta. Sempre il BC si fa carico di garantire il vincolo. Questo richiede di esplicitare il tempo di vita quando si memorizza un riferimento all'interno di una struttura dati, come per esempio una tupla, o si usa un riferimento come valore di ritorno di una funzione.

Il vincolo (apparentemente ovvio) di esistenza in vita dei vincoli dovrebbe essere alla base anche di tutti gli usi dei puntatori in C e C++.

Poiché, però, in questi linguaggi non viene tracciato il tempo di vita, il compilatore non è in grado di identificare la presenza di dangling pointers, né identificare eventuali problemi di non rilascio o doppio rilascio, come invece avviene in Rust.

```
{
    let r;
    {
        let x = 1;
        r = &x;
    }
    assert_eq!(*r, 1);
}

//error: `x` does not live long enough
```

2.7.1 Esistenza in vita

Le regole, ovviamente, valgono anche quando si crea un riferimento ad una parte di una struttura dati più grande.

L'esistenza in vita del riferimento deve essere inclusa in quella della struttura a cui punta

```
let v = vec![1, 2, 3];
let r = &v[1]; // v deve durare più a lungo di r
```

Se, al contrario si memorizzano dei riferimenti in una struttura, tutti questi riferimenti devono avere una durata di vita maggiore della struttura dati in cui sono memorizzati.

```
{
    let mut v = Vec::new();
    {
        let a = 1;
        v.push(&a);
    }
    println!("{:?}", v);
}

\\error[E0597]: `a` does not live long enough
```

2.8 Riferimenti e funzioni

Se una funzione riceve un parametro di tipo riferimento (mutabile o meno), il tempo di vita del riferimento diventa parte integrante della firma della funzione. È necessario infatti che tutte le operazioni eseguite dalla funzione sul riferimento siano compatibili con la validità del dato memorizzato al suo interno

$$\text{fn } f(p: \&i32) \{...\} \Rightarrow \text{fn } f\langle'a\rangle(p: \&'a i32) \{...\}$$

Il compilatore provvede in molti casi a effettuare autonomamente la riscrittura indicata (lifetime elision).

Se la funzione riceve più riferimenti, può essere necessario indicare se il loro tempo di vita sia vincolato al più breve o se siano disgiunti

- Nel primo caso si usa un solo identificatore per identificare l'intersezione dei loro tempi di vita


```
fn f<'a>(p1: &'a i32, p2:&'a i32) {...}
```

- Nel secondo si usano etichette diverse `fn f<'a, 'b>(p1: &'a i32, p2:&'b i32) {...}`

Uno dei casi più frequenti in cui il compilatore non riesce a dedurre il corretto tempo di vita è legato a funzioni che restituiscono un riferimento, estratto da una delle strutture dati ricevute in ingresso.

In tale caso occorre annotare il tipo restituito con la corretta etichetta

```
fn f<'a, 'b>(p1: &'a Foo, p2:&'b Bar) -> &'b i32 { /*...*/ return &p2.y; }
```

Se la funzione memorizza il riferimento ricevuto in ingresso in una struttura dati, il compilatore deduce che il tempo di vita della struttura in cui il riferimento è memorizzato deve essere incluso o coincidente con il tempo di vita del riferimento.

Se questo non avviene, il compilatore identifica l'errore e impedisce alla compilazione di avere successo.

```
fn f(s: &str, v: &mut Vec<&str>) {
    v.push(s); //error: lifetime mismatch
}

fn f<'a>(s: &'a str, v: &'a mut Vec<&str>) {
    v.push(s);
}
//error: explicit lifetime required in the type of `v`

fn f<'a>(s: &'a str, v: &'a mut Vec<&'a str>) {
    v.push(s);
}

fn main() {
    let mut v: Vec<&str> = Vec::new();
    {
        let s= String::from("abc");
        v.push(&s); //error: `s` does not live long enough. Il tempo di vita di v è
        ↪ limitata alla vita di vita di s.
    }
    println!("{:?}",v);
}
```

Se una funzione ha due o più riferimenti in ingresso e un riferimento in uscita, il compilatore non è in grado di decidere (senza eseguire il codice) da quale parametro provenga il riferimento in uscita. L'inserimento degli identificatori nella firma della funzione risolve questa ambiguità.

```
fn f<'b, 'c>(x: &'b S, y: &'c S) -> &'c u8 {...}

let v1 = S(1);
let mut v2 = S(2);

let r = f(&v1, &v2); // Sappiamo che r è basato su v2,
                    // che rimane nello stato di prestito
                    // mentre v1 viene rilasciato e può evolvere
v2 = v1; // Questa operazione crea un problema!
print_byte(r); // Qui finisce il prestito di r (e di v2)
```

Lo scopo degli identificatori relativi al ciclo di vita è duplice:

- Per il codice che invoca la funzione (chiamante), essi indicano su quale, tra gli indirizzi in ingresso, è basato il risultato in uscita
- Per il codice all'interno della funzione (chiamato), essi garantiscono che vengano restituiti solo indirizzi cui è lecito accedere per (almeno) il tempo di vita indicato

All'atto dell'invocazione, gli identificatori forniti dal programmatore (o inseriti automaticamente dal compilatore, quando possibile) sono legati all'effettivo intervallo minimo (espresso come insieme di linee di codice) nel quale il valore da cui il prestito è stato preso debba restare bloccato per non violare le assunzioni su cui la funzione è basata.

Tentativi di modificare il valore originale da cui il prestito è preso prima che il tempo di vita sia trascorso portano ad errori di compilazione che costituiscono la base della robustezza del sistema di possesso e prestito offerto da Rust

2.9 Riferimenti e strutture dati

Lo stesso tipo di ragionamento vale se il dato viene salvato all'interno di una qualsiasi struttura dati

- Rust verifica che il valore a cui si punta abbia un tempo di vita maggiore o uguale del tempo di vita della struttura dati
- Questo richiede di esplicitare il tempo di vita della struttura rispetto al tempo di vita dei riferimenti in essa contenuti

Se una struttura che contiene riferimenti è contenuta, a sua volta, in un'altra struttura, anche quest'ultima deve avere il tempo di vita specificato in modo esplicito.

```
struct User<'a> {
    id: u32,
    name: &'a str,
}
```

```
struct Data<'a> {
    user: User<'a>,
    password: String,
}
```

2.10 Elisione dei tempi di vita

Rust, per default, provvede ad assegnare, ad ogni riferimento presente in una struttura dati o tra i parametri di una funzione, un tempo di vita distinto. Tali tempi di vita si propagano al codice che fa uso di tale struttura dati e, in assenza di ambiguità, non richiede di esplicitare gli identificatori dei tempi di vita.

```
struct Point {
    x: i32,
    y: i32,
}
```

```
fun scale(r: i32, p: Point) -> i32 {
    r * (p.x * p.x + p.y * p.y)
}
```

```
struct Point {
    x: &'a i32,
    y: &'b i32,
}
```

```
fun scale<'a,'b,'c> (r: &'a i32, p: Point<'b,
    ↪ 'c>) -> i32 {
    r * (p.x * p.x + p.y * p.y)
}
```

Se una funzione restituisce un riferimento o un tipo che contiene - direttamente o indirettamente - un riferimento, occorre disambiguare il tempo di vita del valore ritornato

- Se c'è un solo parametro in ingresso dotato di tempo di vita, Rust assume che quello debba essere il tempo di vita del risultato
- Se sono presenti più parametri in ingresso con tempo di vita, tocca al programmatore esplicitare quale debba essere il tempo di vita da associare al risultato

Nel caso di metodi che accedono a self tramite un riferimento, Rust assume che il tempo di vita da associare al risultato sia quello del riferimento a self.

Questo parte dal presupposto che se un metodo di un oggetto restituisce un dato preso a prestito (borrow), questo sia stato preso dai dati posseduti dall'oggetto stesso.

2.11 Possesso - riassunto delle regole

- Ciascun valore ha un unico possessore (variabile o campo di una struttura). Il valore viene rilasciato (drop) quando il possessore esce dal proprio scope o quando al possessore viene assegnato un nuovo valore.
- Può esistere, al più, un singolo riferimento mutabile ad un dato valore
- **Oppure**, possono esistere molti riferimenti immutabili al medesimo valore. Ma fintanto che ne esiste almeno uno, il valore non può essere mutato. Principio del *multiple reader and single writer* presente in molti ambiti dell'informatica.
- Tutti i riferimenti, che sono pestiti del valore, devono avere una durata di vita inferiore a quella del valore a cui fanno riferimento

2.12 Puntatori in Rust

Proprietà	&T	Box<T>
Possesso	No: è un prestito	SI: Possessore unico, risolve il doppio rilascio del C/C++
Rilascio	A carico del possessore	Automatico al termine dello scope
Validità	Garantita dal compilatore, il borrow checker	Garantita in fase di esecuzione
Nullabilità	Mai nullo	Mai nullo
Aliasing e mutabilità	Controllata: o molti riferimenti in sola lettura o un solo riferimento mutabile	Soggetta alla regole del possesso (no aliasing)

2.13 Slice

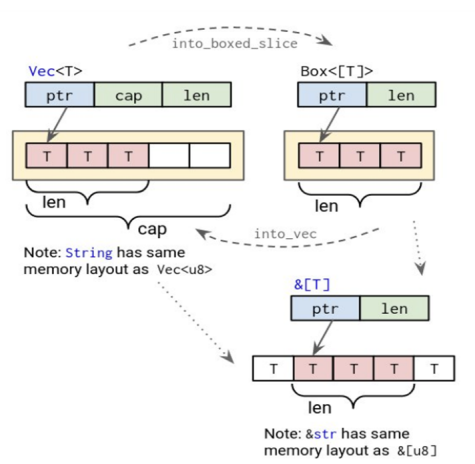
Una slice (fetta) è una VISTA di una sequenza contigua di elementi, la cui lunghezza NON è nota in fase di compilazione, ma disponibile durante l'esecuzione.

Internamente viene rappresentata come **fat pointer** una tupla di due elementi, i primi 64 bits puntano al valore della sequenza, mentre i secondi 64 bits indicano il numero di elementi consecutivi. Una slice di elementi T ha tipo [T].

Una slice **non possiede**, è un prestito a tutti gli effetti, i dati cui fa riferimento. Questi appartengono sempre ad un'altra variabile, da cui la slice viene derivata. Tutti i valori contenuti sono garantiti essere inizializzati, gli accessi sono verificati in fase di esecuzione.

È possibile ricavare una slice a partire da un array, ma anche da altri tipi di contenitori quali `std::Vec<T>`, `String`, `Box<[T]>`, etc.

```
let a: [i32; 5] = [1, 2, 3, 4, 5]; // a è un array di 5 interi
let s = &a[1..3]; // s è una slice formata da 2 elementi [2,3]
let two = s[0]; // two contiene il valore 2
```



2.14 Vantaggi introdotti dal concetto di possesso

Un programma Rust, pur non avendo un garbage collector, offre molteplici garanzie relative alla correttezza degli accessi in memoria e del rilascio delle risorse.

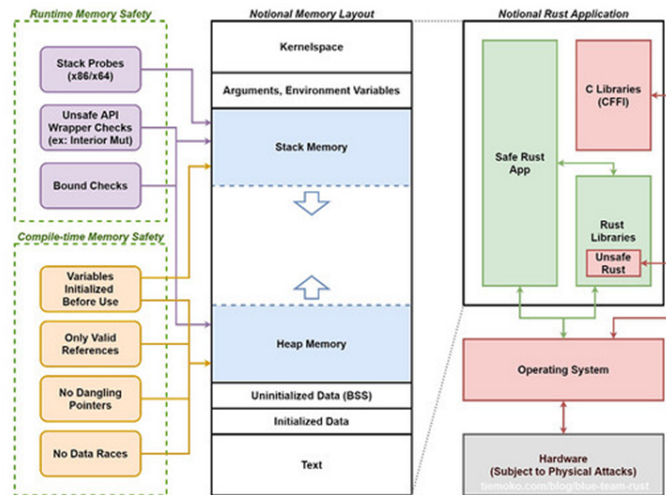
- Non esiste il concetto di **riferimento nullo** e, di conseguenza, non c'è il rischio di dereferenziarlo
- Poiché il borrow checker vigila sulle assegnazioni dei riferimenti e sugli intervalli temporali in cui essi sono effettivamente usati impedendo accessi illeciti, non c'è il rischio di originare errori di segmentazione o accesso illegale ad aree ristrette di memoria, né la possibilità di avere riferimenti ad aree già rilasciate (dangling pointer)
- Poiché in ogni istante il borrow checker è in grado di determinare la dimensione di un blocco di memoria cui un riferimento punta, vedi le slice, non possono verificarsi **buffer overflow** né **buffer underflow**
- Per lo stesso motivo, gli iteratori offerti da Rust **non eccedono** mai i loro limiti

Tutte le variabili sono **immutabili** per default e occorre una dichiarazione esplicita per renderle mutabili. Questo obbliga il programmatore a riflettere attentamente sul come e dove i dati debbano essere modificati e su quale sia il ciclo di vita di ciascun valore.

Il modello di possesso non riguarda solo la gestione della memoria, ma anche la **gestione delle risorse** contenute in un valore come socket di rete, handle di file e database, descrittori dei dispositivi.

L'assenza di un garbage collector impedisce **comportamenti non deterministici**. In particolare, la sospensione totale del funzionamento ogni qual volta occorra ricompattare la memoria.

2.15 Disposizione in memoria



Capitolo 3

Tipi composti

3.1 Struct in Rust

Spesso occorre mantenere unite informazioni tra loro eterogenee. Sebbene sia possibile utilizzare una tupla, questa tende a nascondere la semantica del dato: una tupla (i32, i32) potrebbe rappresentare una frazione così come le coordinate di un pixel, rendendo il codice ambiguo.

```
struct Player {  
    name: String, // nickname  
    health: i32, // punti vita  
    level: u8, // livello corr.  
}
```

Quando si intende associare una semantica particolare o legare ad una struttura dati un insieme di comportamenti, è possibile introdurre una **struct**.

Una struct è un costrutto che permette di rappresentare un blocco di memoria in cui sono disposti, consecutivamente, una serie di campi il cui nome e tipo sono indicati dal programmatore.

Il compilatore capisce quanti byte deve allocare in memoria. Essendo che la memoria è organizzata a parole (word), per un processore a 64 bit, gli x86, è molto più efficiente leggere dati che iniziano a indirizzi multipli di 8 byte. Per questo motivo il compilatore Rust utilizza la tecnica di allineamento e padding.

Se abbiamo una struct composta da un u8 (1 byte) e un u32 (4 byte), il processore preferirebbe che il u32 iniziasse su un indirizzo divisibile per 4. Per ottenere questo, il compilatore inserisce dei “byte morti” (padding) tra i due campi.

3.1.1 Come utilizzare una struct

Si istanzia una struct tramite un blocco preceduto dal nome della struttura, contenente un valore per ciascun campo.

```
let mut s = Player { name: "Mario".to_string(), health: 25, level: 1 };
```

Quando il nome del valore coincide con quello del campo è possibile abbreviare la notazione stile TypeScript:

```
let p = Player { name, health, level };
```

Nota: A differenza del TypeScript, il quale utilizza l’equivalenza strutturale, Rust si basa sull’equivalenza nominale. Il tipo Player con 3 valori non ha nulla a che vedere con Giocatore con i medesimi campi.

Operatore di spreading

Esiste la notazione di *spreading*, parziale o totale, anche in Rust.

Se è parziale la si può utilizzare solo alla fine:

```
let s1 = Player {name: "Paolo".to_string(), ..s}
```

Se è totale:

```
let s1 = Player {...s}
```

Nota: La copia dipende dai tipi di dati, se posseggono il tratto `copy`, come `interi`, `bool`, etc, vengo copiati altrimenti per gli altri dati come le stringhe il valore viene mosso e il possesso del dato viene passato alla struct e la struct originale `s` non è più utilizzabile nel suo insieme perché è stata parzialmente svuotata.

3.1.2 Accesso alla struct

Si accede ai singoli campi con la notazione puntata.

```
println!("Player {} has health {}", s.name, s.health );
```

Ogni struct introduce un nuovo tipo, il cui nome coincide con il nome della struct, basato sui tipi che la compongono. Questo permette di disambiguare l'uso che si intende fare delle singole informazioni contenute.

3.1.3 Struct fatte a tupla

Si possono definire delle struct simili a delle tuple, indicando solo il tipo del campo, senza attribuire un nome.

Le struct di questo tipo si istanziano come una tupla con l'aggiunta del nome della struct. Si accede tramite la notazione posizionale.

```
struct Playground ( String, u32, u32 );
struct Empty; // non viene allocata memoria per questo tipo di valore

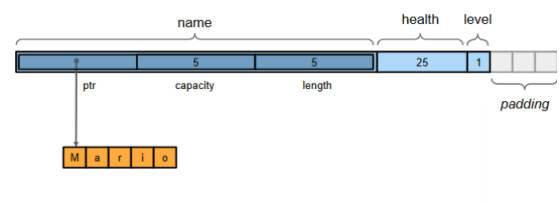
let mut f = Playground( "football".to_string(), 90, 45 );
//f.1 trovo football
//f.2 trovo 90
//f.3 trovo 45
let e = Empty;
```

La semantica a tupla è adatta in situazioni in cui la semantica del dato è ovvia e la notazione posizionale risulta più comoda, come ad esempio le coordinate di un punto. Per il resto, non vi è differenza se non la leggibilità del codice.

In Rust è possibile definire anche struct vuote, senza campi. Questi tipi di struct non occupano spazio in memoria e vengono utilizzati principalmente come marcatori o per implementare comportamenti specifici tramite il tratto `asseganto`.

3.1.4 Probabile rappresentazione in memoria

```
fn main() {
    let p1 = Player {
        name: "Mario".to_string(),
        health: 25,
        level: 1u8,
    };
    println!("{:?}", p1);
}
```



Nota: Non abbiamo garanzia che in memoria sia ordinato nel modo in cui è dichiarato perché il compilatore Rust, a differenza del compilatore del C/C++, può riordinare i campi per ottimizzare l'uso della memoria.

3.1.5 Rappresentazione in memoria

La disposizione in memoria dei singoli campi è conseguenza di vincoli ed ottimizzazioni e può essere controllata attraverso opportuni meccanismi. Ogni singolo campo, in base al proprio tipo, richiede che l'indirizzo a cui viene collocato sia multiplo di una data potenza di 2 (allineamento).

Rust mette a disposizione la notazione

```
std::mem::align_of_val(...)
```

che permette di conoscere l'allineamento richiesto da un particolare valore. Normalmente è utilizzato internamente per il dialogo con il sistema operativo.

Mentre la funzione

```
std::mem::size_of_val(...)
```

ne indica la dimensione in byte occupata in memoria.

Ovviamente i vincoli di allineamento dipendono dalla piattaforma di esecuzione, dalla CPU.

Rappresentazione stile C/C++

In alcuni casi particolare, come l'interoperabilità con codice C ed altri linguaggi, interoperabilità con l'hardware, non vogliamo che il compilatore riordini i campi.

A tal proposito viene anteposto alla definizione della struct l'attributo `#[repr(C)]`.

3.2 Visibilità

Sia la struct nel suo complesso che i singoli campi che la formano possono essere preceduti da un modificatore di visibilità. Per default, i campi sono considerati privati e quindi accessibili nel modulo corrente e nei sottomoduli, eventuali fratelli o genitori non vedono i campi.

Possono però essere resi pubblici facendo precedere il nome dalla parola chiave `pub`.

Nota: Posso definire `pub` solo la struct, ma non i campi, in questo caso i campi sono privati e non accessibili, come il `Vec` la cui `capacity` e `size` non sono accessibili.

Questo permette di implementare il meccanismo di incapsulamento (information hiding).

Per essere efficace, occorre però poter associare un insieme di comportamenti (metodi) alla struct.

A differenza di quanto avviene in altri linguaggi, in cui struttura e comportamento sono definiti contestualmente in un unico blocco, classe, in Rust la definizione dei metodi associati ad una struct avviene separatamente, in un blocco di tipo `impl` ...

3.3 Metodi

Rust non è un linguaggio ad oggetti nel senso tradizionale del termine, anche se implementa incapsulamento e polimorfismo, conseguentemente, non ha il concetto di classe. Dato che questo concetto tende a creare vincoli non voluti.

Sebbene le struct possano apparire simili alle classi di altri linguaggi, il parallelismo è limitato, in particolare, le struct NON sono organizzate in una gerarchia di ereditarietà. Il concetto di metodo si applica invece a tutti i tipi, compresi quelli primitivi.

Il metodo ha accesso ha a TUTTI i campi della struct, anche quelli privati, questo ci permette di implementare l'incapsulamento.

3.3.1 Definizione di un metodo

Si definiscono i metodi collegati ad un tipo in un blocco racchiuso tra parentesi graffe, preceduto dalla parola chiave `impl` seguita dal nome del tipo.

Le funzioni presenti in tale blocco il cui primo parametro sia `self`, `&self` o `&mut self` diventano metodi.

`Self`¹ è una parola chiave che rappresenta l'istanza del tipo di cui si sta facendo l'implementazione.

Le funzioni che non hanno come primo parametro `self` sono dette funzioni associate e svolgono il ruolo giocato dai costruttori e dai metodi statici nei linguaggi ad oggetti.

```
class Something {
    int i;
    String s;

    void process() {...}
    int increment() {...}
};

struct Something {
    i: i32,
    s: String
}
impl Something {
    fn process(&self) {...}
    fn increment(&mut self) {...}
}
```

Se in JavaScript il `this` non compare nella lista dei parametri, dato che è implicito, in Rust viene esplicitato perché dobbiamo dire che operazione facciamo su quel `self`.

- `self`: voglio prendere possesso del ricevitore il quale non sarà più accessibile poi, movimento.
- `&self`: chiedo un prestito un sola lettura del ricevitore, posso leggere ma non posso modificarli, riferimento condiviso.
- `&mut self`: chiedo l'accesso in lettura e scrittura, riferimento esclusivo.

```
pub struct Point{
    x: i32, //privati
    y: i32,

    impl Point {
        pub fn new(x: i32, y: i32) -> Self {
            Point { x, y }
        }

        pub fn length(&self) -> i32{
            (self.x * self.x + self.y *
             ↪ self.y).isqrt()
        }
    }
}

let p1 = Point::new(3, 4);

// possibile solo nello stesso modulo
// o in un sottomodulo di quello in cui
// è stato definito Point
let p2 = Point::new(x:3, y:4);
↪ //potenzialmente non scrivibile dato che
↪ se non sono nello stesso modulo non va
↪ bene

let m1 = p1.length(); //5 quando scrivo
↪ questo il compilatore traduce in
↪ Point::length(&p1)

let m2 = Point::length(&p1); // 5
```

I metodi sono funzioni legate ad un'istanza di un dato tipo. Il legame si manifesta sia a livello sintattico, che a livello semantico.

Sintatticamente un metodo viene invocato a partire da un'istanza del tipo a cui è legato utilizzando la notazione `instance.method(...)`

Semanticamente il codice del metodo ha accesso al contenuto (pubblico e privato) del ricevitore attraverso la parola chiave `self`.

¹Corrisponde grosso modo a quello che in altri linguaggi ad oggetti viene chiamato `this`

```

struct Point {
    x: i32,
    y: i32,
}
impl Point {
    fn mirror(self) -> Self {
        Self{ x: self.y, y: self.x }
    }
    fn length(&self) -> i32 {
        (self.x*self.x + self.y*self.y).isqrt()
    }
    fn scale(&mut self, s: i32) {
        self.x *= s;
        self.y *= s;
    }
}

fn main() {
    let p1 = Point{ x: 3, y: 4 };
    let mut p2 = p1.mirror();
    let l1 = p2.length(); // l1: 5
    p2.scale(2);
    let l2 = p2.length();
    // l2: 10
}

```

3.4 Costruttori

A differenza di quanto capita in altri linguaggi, in Rust non esiste il concetto di costruttore.

In Rust, invece, qualunque metodo nel blocco *impl*, che ritorna *Self*, può fare le veci del costruttore. Questo garantisce che il programmatore sia consapevole delle informazioni contenute al suo interno.

Per convenzione, si sceglie di chiamarlo *new*, ma non è obbligatorio.

```
pub fn new() -> Self {...}
```

Poiché Rust non supporta l'overloading delle funzioni, se servono più funzioni di inizializzazione, ciascuna di esse avrà un nome differente: in questo caso la convenzione è utilizzare un pattern come

```
pub fn with_details(...) -> Self {...}
```

3.5 Gestione delle risorse

Se una struttura contiene al proprio interno risorse la cui semantica richiede una esplicita gestione del ciclo di vita, occorre fare in modo che, quando la struttura cessa di esistere, tali risorse siano liberate in modo appropriato. (E.g. file handle, socket, etc.)

Questo compito è affidato ad una funzione speciale detta *Drop*, il distruttore, il quale se presente viene chiamato in automatico dal compilatore quando l'oggetto cessa di esistere per liberare le risorse che contiene.

Rust segue il paradigma *RAII* - *Resource Acquisition Is Initialization* - l'acquisizione delle risorse avviene alla creazione, il rilascio alla distruzione.

3.6 RAII - Resource Acquisition Is Initialization

Le risorse sono incapsulate in una classe, struttura, in cui:

- il costruttore acquisisce le risorse e stabilisce eventuali invarianti, oppure lancia un'eccezione se non può essere fatto
- il distruttore rilascia le risorse e NON lancia mai eccezioni

Si usano le risorse attraverso l'istanza di una classe *RAII*-compatibile che ha una gestione automatica delle durata di tutte le risorse, **oppure**, ha un ciclo di vita connesso al ciclo di vita di un altro oggetto (ad es., è parte di esso).

In questo contesto, la presenza della semantica del movimento, garantisce il corretto trasferimento delle risorse, mantenendo la sicurezza del rilascio.

3.7 Distruttori

Rust gestisce il rilascio di risorse contenute in un'istanza attraverso il tratto `Drop`, costituito dalla funzione

```
fn drop(&mut self) -> ()
```

Chiamato in automatico dal compilatore. Si può forzare il rilascio delle risorse contenute in un oggetto usando la funzione `drop(some_object)`; che ne acquisisce il contenuto e determina l'uscita dallo scope.

```
pub struct Shape {
    pub position: (f64, f64),
    pub size: (f64, f64),
    pub type: String
}

impl Drop for Shape {
    fn drop(&mut self) {
        println!("Dropping shape!");
    }
}
```

Nota: Il tratto `Drop` è mutuamente esclusivo con il tratto `Copy`. Se un tipo implementa il primo non può implementare l'altro, e viceversa.

3.8 Metodi statici

In C++/Java è lecito inserire all'interno del costrutto `class ...{ }` la dichiarazione di metodi preceduti dalla parola chiave `static`. Essi non sono legati ad una specifica istanza, ma possono operare sulle istanze della classe (se ne conoscono l'indirizzo) avendo accesso anche alle componenti private.

In Rust, è possibile implementare metodi analoghi semplicemente non indicando, come primo parametro, né `self` né un suo derivato. Questo permette la creazione di funzioni per la costruzione di un'istanza, metodi per la conversione di istanze di altri tipi nel tipo corrente o, semplicemente, l'accesso a funzionalità statiche.

L'esempio più comune è il metodo `new`.

```
<Tipo>::metodo(...)
```

3.9 Enum

In Rust, è possibile introdurre tipi enumerativi composti da un semplice valore scalare. Ma anche incapsulare, in ciascuna alternativa, una tupla o una struct volta a fornire ulteriori informazioni relative allo specifico valore.

Anche in questo caso è possibile legare metodi ad un'enumerazione, aggiungendo il blocco `impl ...` come nel caso delle struct.

```
enum HttpResponse {
    Ok = 200,
    NotFound = 404,
    InternalError = 500
}

enum HttpResponse {
    Ok,
    NotFound(String),
    InternalError {
        desc: String,
        data: Vec<u8>
    },
}
```

Se le struct sono un tipo prodotto perché l'insieme dei valori che può contenere è il prodotto cartesiano degli insiemi legati ai singoli campi, le enum sono un tipo somma perché il loro dominio è la somma, l'unione, dei domini dei loro campi. (E.g. o c'è `Ok` o c'è `NotFound` o c'è `InternalError`)

Nelle enum posso legare, oltre che un'etichetta, anche degli altri valori etichetta e non solo valori numerici associati al valore.

3.9.1 Rappresentazione in memoria

In memoria gli enum occupano lo spazio di un intero da 1 byte più lo spazio necessario a contenere la variante più grande.



3.9.2 Enumerazioni e clausole match

Il costrutto *match* si presta particolarmente per gestire in modo differenziato valori enumerativi.

Il comportamento offerto dal pattern matching e la possibilità di legare variabili temporanee in base alla struttura del valore che viene analizzato offre una sintassi compatta ed efficiente per esprimere comportamenti alternativi (destructuring assignment).

Il fatto che i pattern confrontati debbano essere esaustivi, garantisce che il codice resti coerente anche se il numero di possibili alternative presenti nell'enumerazione cambia nel tempo.

```
enum Shape {
  Square { s: f64 },
  Circle { r: f64 },
  Rectangle { w: f64, h: f64 }
}

fn compute_area(shape: Shape) -> f64 {
  match shape {
    Square { s } => s * s,
    Circle { r } => r * r * std::f64::PI,
    Rectangle { w, h } => w * h,
  }
}
```

3.10 Destrutturazione

L'utilizzo del pattern matching e la possibilità di destrutturare un valore complesso non sono limitate al costrutto *match*. È possibile usare la stessa tecnica anche all'interno di costrutti di controllo del flusso come:

```
if let <pattern> = <value> ... e while let <pattern> = <value> ...
```

Tali costrutti verificano se il valore fornito corrisponda o meno al pattern indicato e, nel caso, eseguono le necessarie assegnazioni alle variabili contenute nel pattern.

```
enum Shape {
  Square { s: f64 },
  Circle { r: f64 },
  Rectangle { w: f64, h: f64 }
}

fn process(shape: Shape) {
  // stampa solo se shape è Square...
  if let Square { s } = shape {
    println!("Square side {}", s);
  }
}
```

La destrutturazione è anche utile per ottenere un “parsing” di una struct, così da gestirne più facilmente i campi, estraendoli in contenitori singoli da trattare separatamente. I campi della struct sono “estratti” con il loro nome, direttamente con l'assegnazione.

```
pub struct Point {
  x: i32,
  y: i32
}

...
let p = Point { x: 5, y: 10 };
...
// la destrutturazione deve rispettare i nomi dei campi
let Point { x, y } = p;

println!("The original point was: ({} , {})", x, y);
```

Il processo di destrutturazione utilizza la semantica delle assegnazioni. Se il valore implementa il tratto copy, le variabili introdotte nel pattern conterranno una copia dell'elemento corrispondente. In caso contrario, verrà eseguito un movimento, invalidando il valore originale.

```
enum Shape{
  Square{s:f64},
  Circle{r:f64},
  Rect{w:f64, h:f64}
}

impl Shape{
  fn area(&self) -> f64{
    match self{
      Shape::Square{s: &f64} => s * s,
      Shape::Circle{r: &f64} => std::f64::PI * r * r,
      Shape::Rect{w: &f64, h: &f64} => w * h,
    }
  }
}

fn main() {
  let s = Shape::Square { s: 3.0 };
  println!("Area: {}", s.area());
}

fn shrink_if_circle(shape: &mut Shape) {
  if let Shape::Circle { r } = shape { *r *= 0.5}; //r ha tipo &mut f64
}
```

Capitolo 4

Gestione degli errori

4.1 Errori ed eccezioni

Tutte le computazioni, le funzioni chiamate, possono fallire prematuramente impedendo che possa essere restituito il valore atteso o che vengano eseguiti tutti, o parzialmente, gli effetti collaterali richiesti come la scrittura su file.

Questi fallimenti possono essere dovuti a cause differenti

- Alcune facilmente prevedibili come argomenti illeciti, conversione da testo a numero, etc.
- Altre dovute a limiti del sistema di elaborazione come memoria/spazio su disco esauriti, malfunzionamento della rete o di altre periferiche, etc.

Indipendentemente dalle cause che li hanno prodotti, i fallimenti possono essere catalogati in due gruppi principali

- **Recuperabili:** non hanno compromesso lo stato del programma, per cui è possibile attivare una strategia di ripristino
- **Non recuperabili:** causano un'alterazione imprevedibile dello stato o che indicano l'impossibilità di procedere ulteriormente nella computazione.

Nei casi irrecuperabili, occorre terminare il processo, eventualmente eseguendo qualche operazione di pulizia sull'ambiente esterno.

Negli altri, occorre mettere in atto una strategia di ripristino dello stato come ritentare l'operazione, saltare l'operazione, chiedere l'intervento dell'utente o amministratore, utilizzare strategia alternativa, etc.

Non è detto che il punto in cui si verifica il fallimento possieda abbastanza contesto per discriminare come comportarsi. Occorre fare in modo che, in caso di fallimento della computazione all'interno di una funzione, il controllo torni al suo chiamante, corredato di una opportuna descrizione di quanto successo

Questo requisito tende ad aumentare la complessità del codice, introducendo rami diversi, if/else, switch/case, match, etc., lungo i quali la computazione può procedere, favorendo l'introduzione di errori logici legati alla quantità di dettagli cui occorre badare.

Per questo motivo i linguaggi moderni introducono dei costrutti sintattici a supporto della gestione degli errori.

4.1.1 Spazio di scelta

Quando un'operazione non ha successo si deve decidere come comportarsi nello spazio di scelta che abbiamo a disposizione.

È possibile / sensato ritentare l'esecuzione?

Se l'errore è frutto di condizioni transitorie come ad esempio congestione di rete, file in uso, potrebbe avere senso, a patto di evitare cicli infiniti e introdurre opportune pause tra un tentativo e l'altro, ritentare l'esecuzione.

Un'altra domanda sensata da porsi è se sia possibile continuare l'esecuzione utilizzando un valore di default o attivando una strategia alternativa per calcolare il valore mancante.

È possibile coinvolgere l'utente nella risoluzione del problema? Se l'applicazione è interattiva e l'errore deriva dal comportamento dell'utente questo potrebbe funzionare.

È possibile isolare l'errore e mantenere il resto del sistema in funzione? È un bug o è stato violato un invariante, per cui occorre terminare il programma?

4.1.2 Supporto sintattico alla gestione degli errori

Il linguaggio C non fornisce alcun supporto sintattico alla modellazione degli errori. Lasciando completamente al programmatore la responsabilità di gestire la situazione.

In altri linguaggi come Java, Python, C++, C#, etc., viene introdotto il concetto di eccezione, tipo di dato che descrive un errore che viene lanciato, *throw*, e viene restituito al chiamante.

L'eccezione risale fino al primo *try-catch* che lo gestirà o fino al main dove incontrerà la morte.

Questa idea crea molti problemi anche solo nella conoscenza di tutte le classi di eccezione che possono essere lanciate, l'esempio lampante è Java che descrive le classi `Error` ed `Exception` con le varie differenze.

Inoltre, si creano problemi per la gestione dello stack nel quale si deve tenere traccia del contesto in cui è stata lanciata l'eccezione.

Rust non utilizza il concetto di eccezione, ma offre i tipi algebrici `Result<T, E>` e `Option<T>` per esprimere gli esiti delle computazioni.

Offre inoltre la macro `panic!(...)` per forzare l'interruzione del thread corrente producendo una descrizione testuale di quanto successo.

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

4.1.3 I limiti della gestione delle eccezioni

Il compilatore non è in grado di identificare dove vengano restituite eccezioni, conseguentemente, non forza l'utilizzo di costrutti in grado di gestirle.

La generazione di un'eccezione blocca l'esecuzione del codice seguente e riporta tipicamente un singolo tipo di errore, rendendo onerosa la validazione contemporanea di più criteri.

La possibilità che un'eccezione dello stesso tipo possa essere generata in parti diverse della computazione, ma gestita in un unico punto a monte, rende complessa la scelta delle contromisure da applicare, richiedendo l'ispezione manuale della catena delle chiamate fallite per identificare il punto di rottura.

Per permettere la corretta contrazione dello stack e il ritorno al blocco *try-catch* più recente occorre imporre una certa sovrastruttura allo stack ed al contesto di esecuzione. Tale sovrastruttura non si adatta particolarmente alle assunzioni che vengono fatte nel kernel di Linux e questo è uno dei principali motivi per cui non è possibile scrivere moduli kernel in C++ per quel sistema operativo.

4.1.4 Gestioni delle eccezioni in Rust

Rust offre una risposta funzionale al problema della modellazione degli errori basata sul tipo algebrico generico `Result<T, E>`.

Questo costrutto modella l'unione di tutti i possibili risultati con successo `T` e tutti gli errori `E` che possono verificarsi nell'esecuzione di una funzione.

```
fn read_file(name: &str) -> Result<String, io::Error> {
    let r1 = File::open(name);
    let mut file = match r1 {
        Err(why) => return Err(why),
        Ok(file) => file,
    };
}
```

```
let mut s = String::new();
let r2 = file.read_to_string(&mut s);
match (r2) {
    Err(why) => Err(why),
    Ok(_) => Ok(s),
}
```

4.1.5 Elaborare i risultati

`Result<T, E>` mette a disposizione svariati metodi che consentono di accedere ai dati contenuti al suo interno:

- I metodi `is_ok(&self)` e `is_err(&self)` permettono, rispettivamente, di determinare se l'esito di un'operazione ha avuto successo o meno
- I metodi `ok(self)` e `err(self)` consumano il risultato trasformandolo in un oggetto di tipo `Option<T>` piuttosto che `Option<E>`
- Il metodo `map(self, op: F)->Result<U,E>` applica la funzione al valore contenuto nel risultato, se questo è ok, altrimenti lascia l'errore invariato
- Il metodo `contains(&self, x: &U)` restituisce vero se il risultato è valido e contiene un valore che equivale all'argomento
- Il metodo `unwrap(self)->T` restituisce il valore contenuto, se è valido, ma invoca la macro `panic!(...)` se il risultato contiene un errore

4.1.6 Gestire gli errori

Se l'invocazione di una funzione restituisce un valore di tipo *Result* contenente un errore, occorre mettere in atto una strategia di gestione:

- Ritentare l'esecuzione, evitando di entrare in un loop infinito
- Ricostruire uno stato precedente all'errore (rollback)
- Utilizzare una strategia alternativa per la computazione fallita
- Registrare un messaggio nel log e propagare l'errore al chiamante
- Propagare tout court l'errore al chiamante, delegando ad esso la corrispondente responsabilità
- Terminare, in modo ordinato, il programma

Sebbene terminare il programma possa apparire facile, è sufficiente invocare la funzione `std::process::exit(code: i` farlo in modo pulito può essere complesso.

Occorre infatti garantire che non vengano lasciati oggetti persistenti, come file o altre risorse di sistema, in stati non coerenti. Per questo motivo Rust mette a disposizione la macro `panic!(...)` che, oltre a terminare il thread corrente, esegue un processo di pulizia.

4.1.7 Panic

In alcune situazioni, lo stato del programma risulta così compromesso che non è pensabile eseguire azioni di ripristino: questo spesso è dovuto alla presenza di errori logici nel programma stesso, come l'accesso ad array al di fuori dei relativi limiti, divisioni intere per zero, fallimenti di assegnazione, etc.

In questi casi Rust offre la macro `panic!(...)` che accetta argomenti simili a quelli offerti dalla macro `println!(...)` per formulare un messaggio di errore.

L'effetto dell'invocazione di questa macro è la contrazione dello stack, come nel caso delle eccezioni C++, la distruzione via `.drop()` delle variabili che implementano il tratto Drop fino alla terminazione del thread corrente.

Se il thread in cui è stato invocato è il thread principale dell'applicazione, il processo termina con un codice di errore non nullo, altrimenti **continua**.

4.1.8 Ignorare gli errori

In alcune situazioni, può capitare che l'errore, che potenzialmente viene restituito da una funzione, non possa di fatto succedere, a conseguenza di quanto si è appena verificato nel corso dell'esecuzione.

Oppure che il programmatore assuma che non sia necessario mettere in atto una strategia di contenimento e gestione dell'errore, lasciando semplicemente terminare il processo.

Il tipo `Result<T, E>` mette a disposizione i metodi `unwrap(...)` ed `expect(...)` che restituisce il valore `T`, se presente, ed invocano la macro `panic!(...)` in caso di errore.

Nota: `expect(...)` permette di indicare una stringa che sarà usata al posto del messaggio standard generato da `unwrap(...)`, nel caso si sia verificato un errore

4.1.9 Propagare gli errori

Nella maggior parte delle funzioni in cui si verifica un errore, non si sa quale strategia mettere in atto per ripristinare lo stato del programma dato il contesto insufficiente in cui ci si trova.

Si può ovviare a questo problema restituendo un valore di tipo `Result`. Questo porta, tuttavia, a costrutti alquanto complessi, con espressioni di controllo difficili da decodificare.

Per semplificare la sintassi, Rust offre l'operatore `?` che può essere applicato a qualunque espressione che produce un valore di tipo `Result<T, E>`.

Se il risultato è di tipo `Ok(v)`, l'operatore restituisce il valore `v`, altrimenti, se il risultato è di tipo `Err(e)` la funzione corrente termina, ritornando il valore che incapsula l'errore.

Questo prevede che l'errore di compatibile con l'errore `E` che proviene dalla funzione chiamata.

```
fn read_file(name: &str) -> Result<String, io::Error> {
    let mut file = File::open(name)?;
    let mut s = String::new();
    file.read_to_string(&mut s)?;
    Ok(s)
}
```

4.1.10 Altri modi di esprimere il fallimento

In alcune situazioni, può essere sufficiente distinguere se la computazione ha prodotto il proprio risultato, oppure indicare che non è stato possibile completarla.

Per questi casi esiste il tipo `Option<T>` che racchiude due alternative:

- `Some(T)`: usata per descrivere il risultato atteso
- `None`: he indica l'assenza di risultato, senza descriverne le ragioni

L'operatore `?` può essere applicato anche al tipo `Option<T>`, a condizione che la funzione in cui viene usato restituisca un `Option<U>`, con `U` qualsiasi.

`Result<T,E>` e `Option<U>`

`Result<T,E>` e `Option<U>` sono correlati:

`Result` offre il metodo `ok(self)` che restituisce `Option<T>`, valorizzato con il dato in caso di successo (il dato risulta mosso da `Result`, che quindi diventa inaccessibile).

Offre anche il metodo `err(self)` che restituisce `Option<E>`, valorizzato con l'errore in caso di fallimento (anche in questo caso, con movimento)

4.1.11 Propagare errori eterogenei

Quando una funzione produce diverse tipologie di errore, è necessario propagare i diversi errori così da poter specializzare le contromisure. Rust offre diversi modi per propagare errori eterogenei, la scelta spetta al programmatore sulla base delle sue esigenze.

La libreria standard mette a disposizione l'implementazione generica del tratto *From*, che converte qualsiasi valore che implementi il tratto *Error* in `Box<dyn error::Error>`.

Gli oggetti-tratto richiedono l'utilizzo di fat pointer e vtable con il conseguente costo in termini di memoria. Durante la conversione vengono perse le informazioni sul tipo dell'errore.

Si può risalire allo specifico errore tramite l'utilizzo del `downcast_ref` a patto che si conosca l'implementazione della funzione che genera gli errori. La conversione può avvenire in maniera implicita attraverso l'utilizzo dell'operatore `?`.

```
impl<E: error::Error> From<E> for Box<dyn error::Error>;

fn sum_file(path: &Path) -> Result<i32, Box<dyn error::Error>> {
    let mut file = File::open(path) ? ; // io::Error -> Box<dyn error::Error>
    let mut contents = String::new();
    file.read_to_string(&mut contents) ? ; // io::Error -> Box<dyn error::Error>
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>() ? ; // ParseIntError -> Box<dyn error::Error>
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("sum is {}", sum),
        Err(err) => {
            if let Some(e) = err.downcast_ref::<io::Error>() {...} //tratto io::Error
            else if let Some(e) = err.downcast_ref::<ParseIntError>() {...} //tratto ParseIntError
            else { unreachable!(); } //non può capitare
        }
    }
}
```

Errori custom

Per propagare errori eterogenei senza forzare il sistema dei tipi è possibile implementare degli errori custom.

Tutti gli errori custom devono implementare il tratto *Error* e conseguentemente anche i tratti *Debug* e *Display*.

L'utilizzo di un *enum* permette di racchiudere i diversi tipi di errore da gestire successivamente tramite l'utilizzo del costrutto *match*. È necessario implementare il tratto *From* per convertire i diversi errori nel tipo custom da propagare.

```
#[derive(Debug)]
enum SumFileError {
    Io(io::Error),
    Parse(ParseIntError),
}
```

```

impl From<io::Error> for SumFileError {
    fn from(err: io::Error) -> Self {
        ↪ SumFileError::Io(err) }
}

impl From<ParseIntError> for SumFileError {
    fn from(err: ParseIntError) -> Self {
        ↪ SumFileError::Parse(err) }
}

impl fmt::Display for SumFileError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) ->
        ↪ fmt::Result {
        match self {
            SumFileError::Io(err) => write!(f, "IO
            ↪ error: {}", err),
            SumFileError::Parse(err) => write!(f,
            ↪ "Parse error: {}", err),
        }
    }
}

impl error::Error for SumFileError {
    fn source(&self) -> Option<&(dyn
    ↪ error::Error + 'static)> {
        Some(match self {
            SumFileError::Io(err) => err,
            SumFileError::Parse(err) => err,
        })
    }
}

fn sum_file(path: &Path) -> Result<i32,
    ↪ SumFileError> {
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>()?;
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("the sum is {} ", sum),
        Err(SumFileError::Io(err)) => {...},
        Err(SumFileError::Parse(err)) => {...},
    }
}

```

Create thiserror

Un grosso aiuto all'implementazione di tipi che implementano il tratto *Error* viene dal crate **thiserror**. Esso offre una implementazione della macro *derive* specializzata per il tratto *Error*.

Definendo un tipo (enum, struct, unit) preceduto dall'attributo `#[derive(Error, Debug)]` si ottiene l'implementazione automatica di questi due tratti:

- L'attributo `#[error("Messaggio con formato")]` posta di fronte alle singole voci enumerative o all'intera struct genera l'implementazione del tratto *Display*
- L'attributo `#[from]` posto di fronte ad un campo il cui tipo implementa il tratto *Error* genera l'implementazione del tratto *From* a partire dal tipo del campo

Questo approccio è particolarmente adatto nella creazione di librerie, dove si vogliono rendere i possibili errori parte della definizione del contratto (API).

```

[dependencies]
thiserror = "1.0"

#[derive(Error, Debug)]
enum SumFileError {

    #[error("IO error {0}")]
    Io(#[from] io::Error),

    #[error("Parse error {0}")]
    Parse(#[from] ParseIntError),
}

```

Create anyhow

Il crate **anyhow** definisce l'oggetto-tratto *anyhow::Error* che semplifica la gestione idiomatica degli errori. Si può usare il tipo *anyhow::Error* per incapsulare il valore di ritorno di una funzione che può fallire.

Questo tipo offre un'implementazione automatica del tratto `From<T: Error>`, il che permette di utilizzare la notazione basata sull'operatore `?` per propagare l'errore ritornato.

Quando viene generato un errore è possibile aggiungere una descrizione che contestualizza ciò che è successo tramite i metodi `context(...)` e `with_context(...)`.

Il messaggio di errore associato verrà sostituito dalla stringa indicata seguita dalla causa originale.

Questo crate interopera correttamente con *thiserror* e risulta adatto nella scrittura di codice applicativo, piuttosto che di librerie. In questo caso, la semplicità di scrittura del codice domina rispetto al controllo dell'informazione contenuta nell'errore generato, tenuto conto che sarà interpretato da persone.

```
fn sum_file(path: &Path) -> anyhow::Result<i32> {
    let mut file = File::open(path).with_context(|| format!("Missing path {}", path)) ? ;
    let mut contents = String::new();
    file.read_to_string(&mut contents).context("File read error") ? ;
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>().with_context(|| format!("Not a number: {}", line)) ? ;
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("sum is {}", sum),
        Err(err) => {
            if let Some(e) = err.downcast_ref::<io::Error>() {...} //tratto io::Error
            else if let Some(e) = err.downcast_ref::<ParseIntError>() {...} //tratto ParseIntError
            else { unreachable!(); } //non può capitare
        }
    }
}
```

Capitolo 5

Moduli

Capitolo 6

Polimorfismo

Parte II

OS Programming

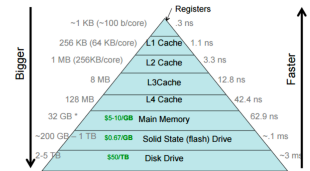
Capitolo 7

Main memory

7.1 Memoria principale

Questa sezione illustra un modo efficace per sfruttare al meglio le potenzialità dell'hardware acquistato; il focus principale è sulla **memoria principale**.

Abbiamo visto all'inizio del corso che un programma, memorizzato su disco, diventa un processo quando viene caricato dal disco nella memoria principale.



La memoria principale e i registri sono gli unici supporti di memorizzazione a cui la CPU può accedere direttamente. L'unità di memoria vede solo un flusso di:

- indirizzi + richieste di lettura, oppure
- indirizzo + dati e richieste di scrittura

La CPU può accedere ai registri in uno, o meno, cicli di clock. Tuttavia, quando la CPU preleva dati dalla memoria principale utilizza molti cicli di clock, causando uno **stall**.

La cache si trova tra la memoria principale e i registri e funge da buffer per i dati necessari; non vogliamo un collo di bottiglia.

Tutti questi livelli contengono una copia dello stesso sottoinsieme di dati.

7.1.1 Località spaziale e località temporale

Località spaziale: La località spaziale significa che se leggo una variabile o un'istruzione in memoria, tutti i dati vicini hanno un'alta probabilità di essere prelevati; pertanto quando leggo, leggo in blocchi e memorizzo tutti i dati in una memoria più veloce.

Località temporale: La località temporale significa che se leggo una variabile, la probabilità di riutilizzarla è alta; pertanto la mantengo nella memoria cache.

S.No.	Spatial Locality	Temporal Locality
1.	In Spatial Locality, nearby instructions to recently executed instruction are likely to be executed soon.	In Temporal Locality, a recently executed instruction is likely to be executed again very soon.
2.	It refers to the tendency of execution which involve a number of memory locations .	It refers to the tendency of execution where memory location that have been used recently have a access.
3.	It is also known as locality in space.	It is also known as locality in time.
4.	It only refers to data item which are closed together in memory.	It repeatedly refers to same data in short time span.
5.	Each time new data comes into execution.	Each time same useful data comes into execution.
6.	Example : Data elements accessed in array (where each time different (or just next) element is being accessing).	Example : Data elements accessed in loops (where same data elements are accessed multiple times).

7.2 Relocation problem

Si presenta il problem riallocation: l'indirizzo deve essere scalato a quello reale. Come si può risolvere?

- Si guardando i riferimenti e calcolo l'indirizzo. Difficile
- Carico gli indirizzi così come sono. Fattibile utilizzando l'hardware.

Utilizzando l'hardware, base e limit, prima di arrivare al bus inseriamo un sommatore e in assembly non cambia nulla.

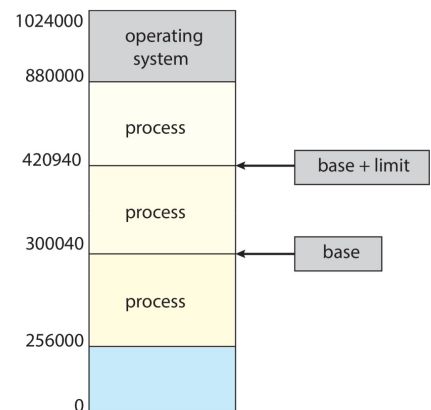
7.3 Protezione

Quando un programma è diventato un processo, tutti o gran parte dei suoi dati sono memorizzati nella memoria principale. Abbiamo bisogno di un modo per:

- Suddividere la memoria tra tutti i processi;
- Garantire che un processo possa accedere solo agli indirizzi del proprio spazio di indirizzamento^a.

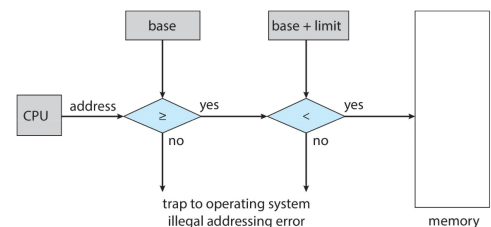
Possiamo risolvere questi problemi utilizzando una coppia di registri **base** e **limite** che definiscono lo spazio di indirizzamento logico di un processo.

^aLo spazio di indirizzamento, o address space, è una porzione di memoria cui un programma può utilizzare



7.3.1 Protezione degli indirizzi hardware

La CPU/OS deve verificare ogni accesso alla memoria generato in modalità utente per assicurarsi che sia compreso tra base e limite per quell'utente. Le istruzioni per caricare i registri base e limite sono privilegiate.



7.4 Address binding

Quando scriviamo un programma C, vogliamo utilizzare indirizzi simbolici e vogliamo che sia il compilatore a trasformarlo in indirizzi relativi. Il linker decide di riallocare a valori assoluti.

Inoltre non ci vogliamo preoccupare di dove si trovi la variabile in memoria, sappiamo solo che è in memoria.

- Gli indirizzi nel codice sorgente sono tipicamente simbolici;
- Gli indirizzi nel codice compilato vengono legati a indirizzi rilocabili, es. «14 byte dall'inizio di questo modulo» invece di «byte 14»;
- Il linker o il loader legherà gli indirizzi rilocabili ad indirizzi assoluti;
- Ogni binding mappa uno spazio di indirizzamento in un altro.

Nel codice compilato vediamo quanta memoria è necessaria e non l'indirizzo codificato in modo fisso. Vogliamo un binding dinamico e non statico.

Binding di istruzioni e dati in memoria

Quando un programma viene avviato, il SO assegna una quantità di spazio al programma.

Il binding degli indirizzi di istruzioni e dati agli indirizzi di memoria può avvenire in tre momenti diversi:

- **Tempo di compilazione:** Se la posizione in memoria è nota a priori, può essere generato codice assoluto; è necessario ricompilare il codice se la posizione iniziale cambia. Utilizzato nei sistemi embedded o nei sistemi in cui i task sono noti a priori e sono in numero ridotto. Sto generando l'eseguibile;
- **Tempo di caricamento:** Deve essere generato codice rilocabile se la posizione in memoria non è nota al momento della compilazione. Strategia ampiamente utilizzata nei vecchi SO o in sistemi semplici come le lavastoviglie...
- **Tempo di esecuzione:** Il binding viene ritardato al runtime se il processo può essere spostato durante l'esecuzione da un segmento di memoria a un altro; richiede supporto hardware (es. registri base e limite) ed è lo standard de facto per i SO moderni.

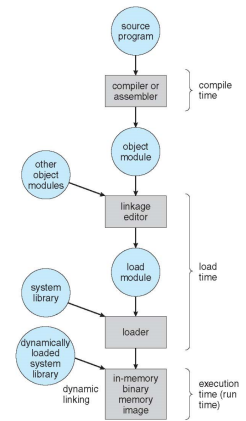


Figura 7.1: Elaborazione in più fasi di un programma utente

7.5 Memory Management Unit - MMU

Ho bisogno di un modo per passare dalla memoria virtuale alla memoria hardware e viceversa, soprattutto quando gli indirizzi sono diversi in fase di esecuzione. Introduciamo il concetto di **processo di binding**.

7.5.1 Spazio di indirizzamento logico vs. fisico

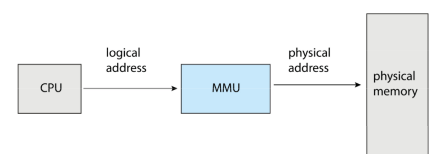
Il concetto di spazio di indirizzamento logico legato a un separato **spazio di indirizzamento fisico** è centrale per una corretta gestione della memoria.

- **Indirizzo logico** - generato dalla CPU; detto anche **indirizzo virtuale**;
- **Indirizzo fisico** - indirizzo visto dall'unità di memoria.

Gli indirizzi logici e fisici coincidono negli schemi di binding al tempo di compilazione e di caricamento, ma differiscono nello schema di binding al tempo di esecuzione.

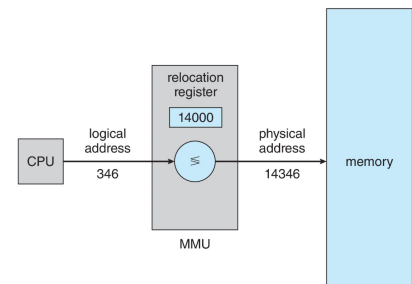
7.6 MMU

Dispositivo hardware, all'interno della CPU, che al runtime mappa gli indirizzi virtuali, che troviamo nel codice assembler, in indirizzi fisici. Vi sono diverse tipologie di MMU, anche in base a quanto vogliamo far pagare il chip.



Il registro base ora si chiama registro di rilocazione. Il valore nel registro di rilocazione viene aggiunto a ogni indirizzo generato da un processo utente nel momento in cui viene inviato alla memoria.

Questo componente è composto da un sommatore ed un comparatore per bloccare il programma se vuole utilizzare spazi non facenti parti della sua address space.



Il programma utente lavora con indirizzi logici; **non vede mai i veri indirizzi fisici**. Se si stampa l'indirizzo di una variabile in un programma C, viene stampato il valore logico e non il valore fisico reale!

7.6.1 Relocatin register

Se abbiamo una MMU basta su Relocation register, gli indirizzi logici sono calcolati al volo. Viene utilizzato quando si vuole ritardare il binding fino alla fase di esecuzione.

Oltre che al relocation register ci sono anche altri modi per caricare nuovi processi in memoria: **Dynamic loading** e **dynamic loading**

7.6.2 Dynamic loading

L'intero programma non deve necessariamente essere tutto in memoria per essere eseguito. Una routine, parte del programma, non vorremmo venisse caricata finché non viene chiamata. Questo migliora l'utilizzo dello spazio in memoria. Il loader perciò deve intervenire quando vogliamo caricare nuove porzioni di codice. Questo fa sì che il programma deve essere organizzato ed avere un formato specifico.

Il SO può aiutare fornendo librerie per implementare il caricamento dinamico.

7.6.3 Static vs Dynamic Linking

Linking statico - librerie di sistema e codice di programma sono tutti combinati dal linker nell'immagine del programma binario in fase di compilazione. Problema: grandi eseguibili.

Linking dinamico - il linking è rinviato fino alla fase di esecuzione del programma.

Nota: Le DLL possono avere le due varianti: load-time dynamic linking (caricamento all'avvio) o run-time dynamic linking (caricamento su richiesta durante l'esecuzione)

Non tutti gli indirizzi sono risolti prima dell'esecuzione, pensiamo ad esempio alla printf, possiamo assumere che molti programmi in esecuzione possono volerla utilizzare. Per questo motivo teniamo ancora indirizzi non risolti e inseriamo piccolo frammento di codice, chiamati *stub*, utilizzato per individuare la routine di libreria residente in memoria. A runtime lo stub, che faceva finta di essere la routine corretta, viene sostituita con l'indirizzo della routine corretta e la esegue a runtime.

Il sistema operativo verifica se la routine si trova nello spazio di indirizzamento del processo e, in caso contrario, la aggiunge.

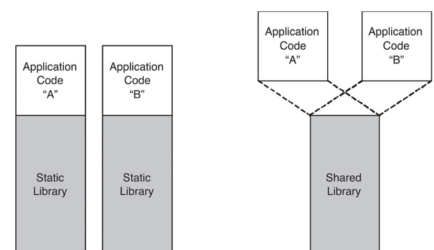


Figura 7.2: File .dll

Vantaggio: Il dynamic linking è particolarmente utile per le librerie, il sistema è anche visto come "shared libraries" (librerie condivise); questo consente di ottenere programmi di dimensioni ridotte poiché le librerie sono condivise tra più programmi. Tutti i programmi, tranne il primo che la caricherà, la troveranno già in memoria.

Questo metodo offre inoltre un ulteriore vantaggio: quando Windows aggiorna una libreria, è necessario ricompilare solamente quella e non tutte le altre applicazioni che la utilizzano.

Problemi: Spazio condiviso e versioning delle librerie, normalmente viene risolto tramite l'aggiornamento di queste ultime.

7.7 Allocazione dei programmi

A questo punto dobbiamo parlare di dove effettivamente vengono allocati i programmi in memoria, e di come viene creato lo spazio di indirizzamento. Nella memoria centrale va inserito tutto, dal sistema operativo ai processi utente. Solitamente il SO si prende una parte di RAM in alto o in basso dato che lui non parteciperà al gioco dell'allocazione e deallocazione della memoria

7.7.1 Allocazione contigua

Vogliamo gestire efficientemente la memoria principale perché deve supportare sia il SO che i processi utente ed è una risorsa limitata; deve essere allocata in modo efficiente.

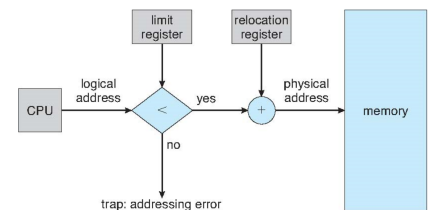
L'allocazione contigua è uno dei primi metodi: array e liste concatenate.

La memoria principale è solitamente suddivisa in due partizioni:

- Sistema operativo residente, solitamente collocato in memoria bassa con il vettore degli interrupt;
- I processi utente sono collocati in memoria alta

Ogni processo è contenuto in una singola sezione contigua di memoria.

Il relocation register viene utilizzato per proteggere i processi utente l'uno dall'altro e per impedire modifiche al codice e ai dati del sistema operativo.



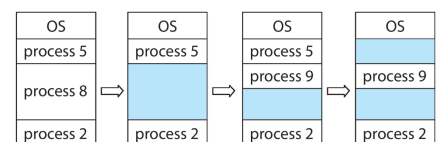
- Il base register contiene il valore del più piccolo indirizzo fisico
- Il limit register contiene l'intervallo degli indirizzi logici - ogni indirizzo logico deve essere inferiore al registro limite
- La MMU mappa dinamicamente gli indirizzi logici

7.7.2 Partizione variabile

Il SO può decidere l'allocazione a partizioni multiple. Il grado di multiprogrammazione è limitato dal numero di partizioni.

- **Partizione variabile** - dimensioni per l'efficienza (dimensionate in base alle necessità del processo).
- **Buco** - blocco di memoria disponibile; buchi di varie dimensioni sono dispersi nella memoria. Indesiderato; idealmente si vuole un blocco contiguo di memoria disponibile.

Quando un processo arriva, gli viene allocata memoria da un buco sufficientemente grande da contenerlo; quando un processo termina e libera la sua partizione, le partizioni libere adiacenti vengono unite. Tutto ciò è gestito dal sistema operativo che mantiene informazioni su:



- partizioni allocate
- partizioni libere (buchi)

7.7.3 Problema dell'allocazione dinamica della memoria

Come soddisfare una richiesta di dimensione n da una lista di buchi liberi?

- **First-fit:** Alloca il primo buco sufficientemente grande

Può produrre molti piccoli buchi residui

- **Best-fit:** Alloca il buco più piccolo sufficientemente grande; deve scorrere l'intera lista, salvo che sia ordinata per dimensione

Produce il buco residuo più piccolo, che potrebbe non essere più riempito.

- **Worst-fit:** Alloca il buco più grande; deve anch'esso scorrere l'intera lista

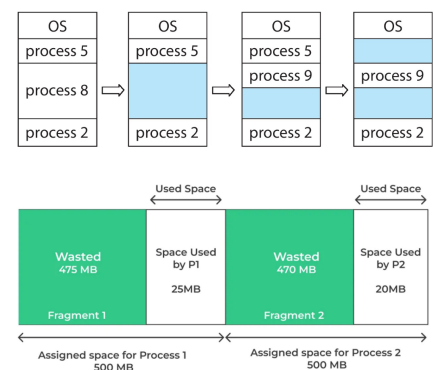
Produce il buco residuo più grande, che può essere riempito da altri processi.

First-fit e best-fit sono migliori di worst-fit in termini di velocità e utilizzo della memoria. Per decidere tra Best e Worst fit andrebbero fatti test sul campo.

7.7.4 Frammentazione

Frammentazione esterna - esiste sufficiente spazio di memoria totale per soddisfare una richiesta, ma non è contiguo. Sia la strategia first-fit che best-fit causano frammentazione esterna.

Frammentazione interna - la memoria allocata può essere leggermente più grande di quella richiesta; la dimensione minima è definita dal SO e la differenza tra la dimensione del programma C e questa dimensione minima è detta frammentazione interna. L'uso della partizione dinamica per assegnare spazio al processo può risolvere questo problema.



È sensato pensare che allocando N blocchi, la metà sono frammentati. E.g. 100 blocchi, 50 frammentati, 50 utilizzati, frammentazione del 50% (thumb rule).

7.7.5 Compattazione

Si riduce la frammentazione esterna tramite la **compattazione**: il contenuto della memoria viene riorganizzato in modo da raccogliere tutta la memoria libera in un unico blocco contiguo. La compactazione è possibile solo se la rilocalizzazione è dinamica e viene eseguita al tempo di esecuzione.

Esempio: La cosa migliore sarebbe trasferire tutti i processi da un'altra parte, questo implica avere altra RAM la quale avrei utilizzato se l'avessi avuta. Posso utilizzare il disco ma questo significa che non potrei utilizzare il pc.

Inoltre si presenta il problema dell'I/O: se un processo è "waiting for IO", non posso spostarlo perché altrimenti l'I/O non saprebbe dove inviare i dati.

Soluzioni: 1. Evitare che il processo venga spostato, la cui conseguenza è una deframmentazione non ottimale;

2. Spostare comunque il programma utilizzando un buffer, anche se questo comporta un tempo complessivo di spostamento più lungo. Spostare la memoria da utente a kernel fatto all'inizio di "waiting for IO".

Sebbene la tecnica di compactazione sia molto utile per migliorare l'utilizzo della memoria e ridurre la frammentazione esterna, il problema è che durante il processo viene sprecata una grande quantità di tempo e la CPU rimane inattiva, riducendo l'efficienza complessiva del sistema.

7.8 Paginazione

Supponiamo di consentire che lo spazio di indirizzamento fisico di un processo possa essere non contiguo: il vantaggio è maggiore. In questo modo, al processo viene allocata memoria fisica ogni volta che la memoria principale è disponibile. Evitiamo la frammentazione esterna e blocchi di memoria di dimensioni variabili, sono della stessa dimensione.

Il processo di paginazione consiste nel dividere la memoria fisica in blocchi di dimensione fissa chiamati **frame**; la dimensione è una potenza di 2, compresa tra 512 byte e 16 MiB. Anche la memoria logica viene divisa in blocchi della stessa dimensione chiamati **pagine**.

Nota: a questo punto è indifferente parlare di SSD o RAM, è la stessa cosa; cambiano solo la latenza e la volatilità.

Questo permette di tenere traccia di tutti i frame liberi e consente di eseguire un programma di dimensione N pagine, trovando N frame liberi e caricando il programma. Per fare ciò dobbiamo impostare una tabella delle pagine per tradurre gli indirizzi logici in fisici. La frammentazione interna è ancora presente.

Per trovare i frame liberi possiamo utilizzare una free-list oppure un vettore chiamato bitmap.

7.8.1 Schema di traduzione degli indirizzi

L'indirizzo (virtuale) generato dalla CPU è suddiviso in:

- **Numero di pagina (p)** - usato come indice in una tabella delle pagine che contiene l'indirizzo base di ogni pagina nella memoria fisica;
- **Offset di pagina (d)** - combinato con l'indirizzo base definisce l'indirizzo di memoria fisica inviato all'unità di memoria.

Per un dato spazio di indirizzamento logico 2^m e dimensione di pagina 2^n .

page number	page offset
p	d
m - n	n

7.9 Hardware di paginazione

Abbiamo quasi annullato il problema della frammentazione dato che tutti i processi ora possono prendere la stessa dimensione della pagina, e non importa se sono contigue o no.

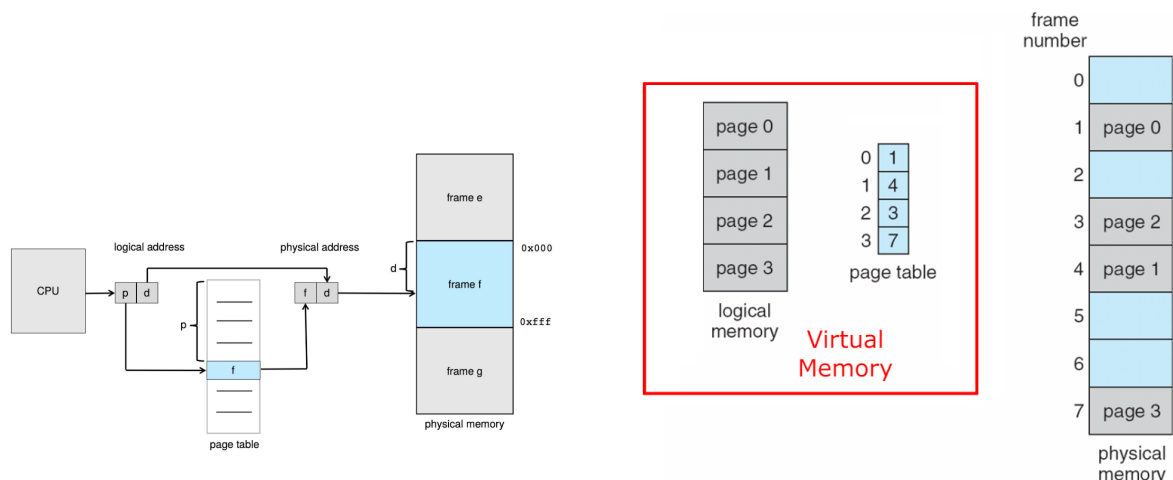
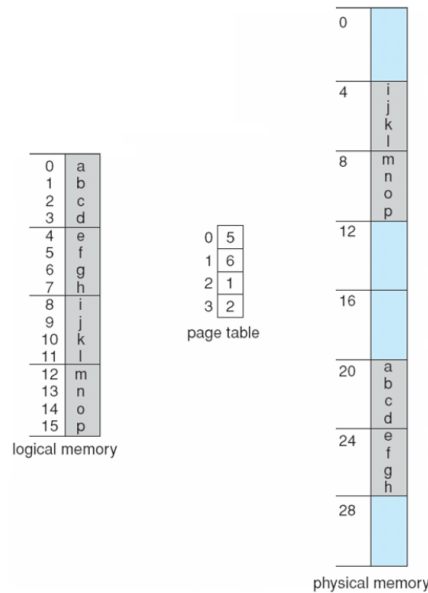


Figura 7.3: Modello di paginazione della memoria logica e fisica

Esempio di paginazione

Indirizzo logico: $n = 2$ (bit di offset), $n-m = 3$ (bit di pagina). Con una dimensione di pagina di 4 byte e una memoria fisica di 32 bit ($2^m = 8$ pagine).



Paginazione - Calcolo della frammentazione interna

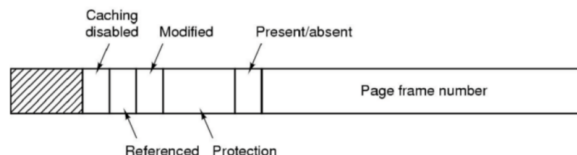
Dimensione pagina = 2.048 byte; dimensione processo = 72.766 byte; si usano 35 pagine + 1.086 byte, la frammentazione interna è $2.048 - 1.086 = 962$ byte.

Caso peggiore = dimensione frame - 1 byte; in media frammentazione = $1/2$ dimensione frame.

Quindi, frame di dimensione ridotta consentono una piccola frammentazione interna... ma la dimensione della tabella delle pagine aumenta perché ogni tabella delle pagine, per processo, occupa memoria: dimensione pagina piccola = moltissime pagine = molta memoria allocata per tracciare le pagine e non disponibile per i task utente/SO.

Le dimensioni delle pagine sono cresciute nel tempo:

- Solaris supporta due dimensioni di pagina - 8 KB e 4 MB
- Win10 supporta due dimensioni di pagina - 4 KB e 2 MB
- Linux supporta due dimensioni di pagina - 4 KB e dimensione huge page personalizzata



Sappiamo, dato l'architettura e il funzionamento della memoria e dei processori, che non possiamo allocare vettori di dimensioni arbitrarie, ma dobbiamo utilizzare potenze di 2.

A questo punto, essendo che abbiamo celle disponibili, possiamo memorizzare dati aggiuntivi. Ad esempio, la protection bit per indicare se il frame è read-only o se è anche permesso fare accessi read-write.

7.9.1 Quante pagine?

Supponiamo una dimensione di pagina di 4KB (2¹² byte) e una CPU hardware a 32 bit. La voce della tabella delle pagine è pari a 32 bit. 2³² pagine, ciascuna da 4KiB, per un totale di memoria indirizzabile pari a 16 TiB (2¹² * 2¹⁰, dove il primo termine rappresenta gli indirizzi di paginazione e il secondo l'offset). Altro esempio:

$$m = \text{lunghezza indirizzo} = 8 \text{ bit.}$$

$$n = \log_2(\text{dimensione pagina}) = \log_2(32 \text{ byte}) = 5 \text{ bit di offset.}$$

$$m - n = 3 = \text{numero di pagine che posso indirizzare.}$$

Riducendo la dimensione della pagina si possono indirizzare più pagine e l'offset diminuisce. Normalmente $m = 32$ e la dimensione della pagina è 4KiB, come nell'esempio precedente.

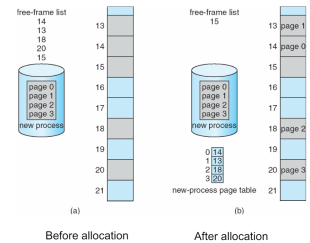


Figura 7.4: Frame liberi

7.9.2 Implementazione della page table

La tabella delle pagine, come detto in precedenza, è mantenuta nella memoria principale. È il principale "artefatto" della gestione della memoria virtuale:

Il registro base della tabella delle pagine (Page-table base register - PTBR) punta alla tabella delle pagine

Il registro di lunghezza della tabella delle pagine (Page-table length register - PTLR) indica la dimensione della tabella delle pagine

In questo schema ogni accesso a dati o istruzioni richiede **due accessi alla memoria!** Uno per la tabella delle pagine e uno per i dati o l'istruzione.

Il problema del doppio accesso alla memoria può essere risolto tramite l'uso di una speciale cache hardware ad accesso rapido chiamata *translation look-aside buffer*, **TLB**.

Sfrutta la **località temporale**: la tendenza dei programmi a utilizzare ripetutamente gli stessi dati nel corso della loro esecuzione.

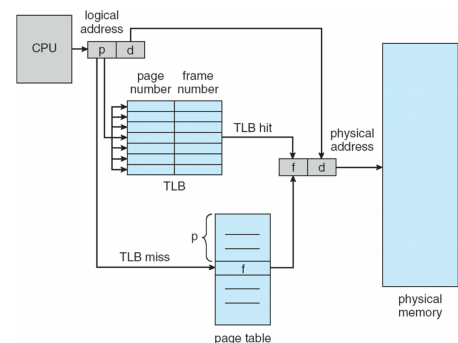
7.9.3 Soluzione hardware - Translation Look-Aside Buffer

La TLB è una cache univoca per tutte le page table. Il primo apporccio potrebbe essere quello di svuotare la TLB ad ogni context switch, ma questo è inefficiente.

Invece di svuotare la TLB, memorizziamo anche **identificatori dello spazio di indirizzamento di un dato processo (ASID)** in ogni voce del TLB.

I TLB sono tipicamente piccoli, da 64 a 1024 voci, ma più veloci; sono quindi una cache completamente associativa.

In caso di TLB miss, il valore viene caricato nel TLB per un accesso più rapido la volta successiva; le politiche di sostituzione devono essere considerate; alcune voci possono essere fissate per un accesso rapido permanente.



Non accelera tutto:

- Caso medio: accesso nel TLB + accesso in memoria per il dato;
- Caso peggiore: nessun dato nel TLB, quindi accesso in memoria per la tabella delle pagine + accesso in memoria per il dato.

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Una volta riempita la TLB, dobbiamo utilizzare algoritmi per decidere quale voce sostituire in caso di TLB miss. Sicuramente l'utilizzo di un valid-invalid bit per ogni voce del TLB è utile per identificare le voci che possono essere sovrascritte.

7.9.4 Tempo di accesso effettivo

Hit ratio - percentuale delle volte in cui un numero di pagina viene trovato nel TLB. Un hit ratio dell'80% significa che troviamo il numero di pagina desiderato nel TLB l'80% delle volte.

Supponiamo che siano necessari 10 nanosecondi per accedere alla memoria. Se troviamo la pagina desiderata nel TLB, un accesso alla memoria mappata richiede 10 ns; altrimenti servono due accessi alla memoria, quindi 20 ns.

Tempo di accesso effettivo (EAT) = $0,80 \times 10 + 0,20 \times 20 = 12$ nanosecondi, il che implica un rallentamento del 20% nel tempo di accesso.

Considerando un hit ratio più realistico del 99%, $EAT = 0,99 \times 10 + 0,01 \times 20 = 10,1$ ns, il che implica solo l'1% di rallentamento nel tempo di accesso.

7.10 Codice C e page fault

Immagina di avere la RAM piena, con solo una pagina libera, e in questa pagina vogliamo fare questo:

```
int main(){
    int[128,128] data;

    ...

    for (j = 0; j < 128; j++)
        for (i = 0; i < 128; i++)
            data[i,j] = 0;

    //quanti page fault? 128 * 128 = 16384

    ...

    for (i = 0; i < 128; i++)
        for (j = 0; j < 128; j++)
            data[i,j] = 0;

    //quanti page fault? solo 128
}
```

Page fault - Un page fault si verifica quando un programma tenta di accedere a dati o codice che si trovano nel suo spazio di indirizzamento, ma non sono attualmente nella RAM del sistema.

Quando si accede a qualcosa nella RAM, il SO carica un blocco di dati dall'SSD nella RAM. Nel primo caso il SO carica un blocco ma il codice C usa solo la prima colonna; nel secondo caso il SO carica un blocco di dati e ne usa tutta la colonna.

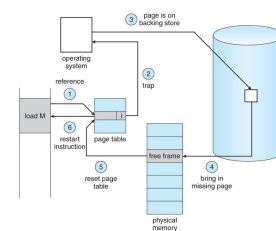


Figura 7.5: Page fault

7.11 Page fault

Cos'è un page fault? I dati sono tipicamente memorizzati in memorie non volatili, grandi e lente, e poi caricati nella RAM; ciò significa che i dati necessari per l'esecuzione (una pagina) possono esistere solo nella memoria secondaria a causa del caricamento dinamico. Devono quindi essere caricati nella RAM, poi nella cache, poi nei registri: questo è un **page fault**! Non possiamo eliminarlo, ma cerchiamo di mitigare questo evento lento.

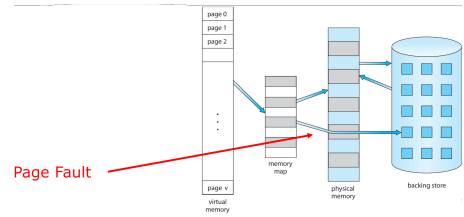


Figura 7.6: Page fault

Background

Il codice deve essere in memoria per essere eseguito, ma l'intero programma è raramente utilizzato interamente per via della gestione delle eccezioni, funzioni non utilizzate e larghe strutture dati.

Tutto il programma non è necessario allo stesso tempo.

Considera la possibilità di eseguire un programma parzialmente caricato: il programma non è più vincolato dai limiti della memoria fisica, ogni programma occupa meno memoria durante l'esecuzione, il che consente a più programmi di essere eseguiti contemporaneamente, aumentando l'utilizzo della CPU e il throughput senza aumentare i tempi di risposta o turnaround. Inoltre, è necessario meno I/O per caricare o scambiare programmi in memoria, il che significa che ogni programma utente viene eseguito più velocemente.

Stiamo parlando di loading dinamico e non abbiamo bisogno che tutto il programma sia tutto in RAM. La ram deve essere capiente abbastanza da contenere il kernel + la parte dei processi che stiamo eseguendo.

Memoria virtuale

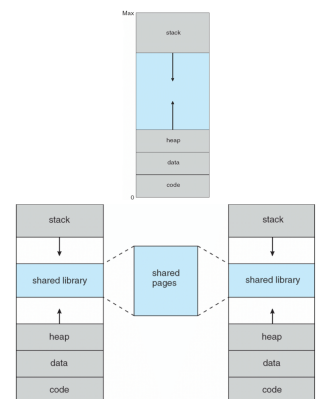
Virtual memory: separazione tra memoria logica utente e memoria fisica; solo una parte del programma deve essere in memoria per l'esecuzione; lo spazio di indirizzamento logico può quindi essere molto più grande dello spazio di indirizzamento fisico (RAM); consente la condivisione degli spazi di indirizzamento da parte di più processi; consente una creazione di processi più efficiente; più programmi in esecuzione contemporaneamente; meno I/O necessario per caricare o scambiare processi in memoria.

Virtual address space: molto maggiore della memoria fisica; visualizzazione logica di come il processo è memorizzato in memoria; di solito inizia all'indirizzo 0 ma non è detto, con indirizzi contigui fino alla fine dello spazio; nel frattempo, la memoria fisica è organizzata in frame di pagina; la MMU deve mappare gli indirizzi logici a quelli fisici.

La virtual memory può essere implementata tramite: demand paging, demand segmentation.

demand paging: quando acceso alle tabelle delle pagine o avviene la traduzione o che devo accedere ad una pagina che è ancora virtuale, non c'è ancora

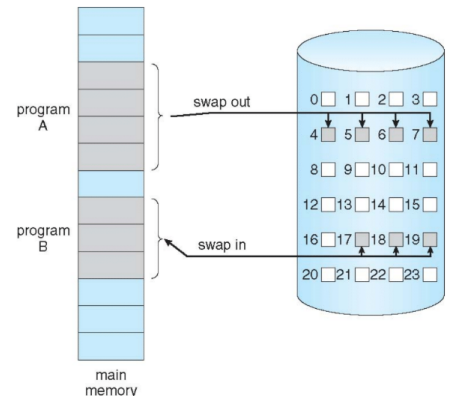
Normalmente lo spazio di indirizzamento logico è progettato in modo che lo stack inizi all'indirizzo logico massimo e cresca "verso il basso", mentre l'heap cresce "verso l'alto". Questo massimizza l'utilizzo dello spazio degli indirizzi, lasciando uno spazio inutilizzato tra i due, noto come "buco". Non è necessaria memoria fisica fino a quando l'heap o lo stack non crescono fino a una nuova pagina. Ciò consente spazi di indirizzamento sparsi con buchi lasciati per la crescita, librerie dinamicamente collegate, ecc. Le librerie di sistema possono essere condivise tramite mappatura nello spazio di indirizzamento virtuale. La memoria condivisa può essere implementata mappando pagine in lettura-scrittura nello spazio di indirizzamento virtuale. Le pagine possono essere condivise durante `fork()`, accelerando la creazione dei processi.



Demand Paging

Posso caricare un processo interamente in memoria al momento del caricamento, oppure posso caricare una pagina solo quando è necessaria. Il demand paging richiede meno I/O, non esegue I/O non necessario, richiede meno memoria, offre una risposta più rapida e consente più utenti. È simile a un sistema di paging con swapping.

Il vero demading dice di caricare solo la prima pagina del processo, ora comportati come hai fatto prima con i processi, swap in e swap out, ma lo fai a livello di pagina page in e page out. Lazy swapper non swappa mai pagine nella memoria fino a che quella pagina non è necessaria. Uno swapper che lavora con le pagine è chiamato pager. Uno swapper che non prende tutto il processo ma solo le pagine, diventa un pager.



Basic Concepts

Con lo swapping, il pager indovina quali pagine saranno utilizzate prima di essere nuovamente scambiate. Invece, il pager porta in memoria solo quelle pagine necessarie. Come determinare quel set di pagine? È necessaria una nuova funzionalità della MMU per implementare il demand paging. Se le pagine necessarie sono già residenti in memoria (**Memory resident**), non c'è differenza rispetto al non demanding paging.

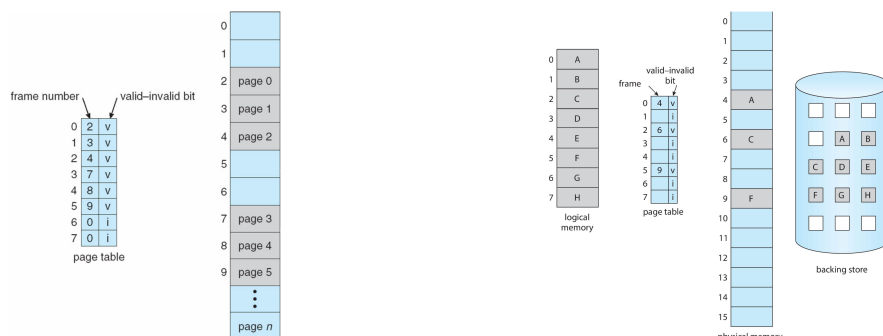
Se la pagina necessaria NON è residente in memoria, è necessario rilevare e caricare la pagina in memoria dallo storage senza modificare il comportamento del programma e senza che il programmatore debba modificare il codice.

7.11.1 Come rilevare i page fault?

Possiamo usare un bit, **valid-invalid**, per tracciare se l'indirizzo è nella RAM o nella memoria secondaria.

Nota: problema dell'allineamento tra la memoria primaria e fisica.

- valido (1): indica che la pagina associata è nello spazio di indirizzamento logico del processo ed esiste già nella RAM;
- non valido (0): indica che la pagina non è nello spazio di indirizzamento logico del processo e non è nella RAM, il che significa che si verificherà un **page fault/trap nel kernel**. Inizialmente, sono tutte non valide.



7.11.2 Passi nella gestione dei page fault

1. Se c'è un riferimento a una pagina (invalida), il primo riferimento a quella pagina genera un trap al sistema operativo = page fault;
2. Il Sistema Operativo guarda un'altra tabella per decidere se:
 - Riferimento non valido → abort
 - Semplicemente non in memoria, deve essere caricata dal disco
3. Trovare un frame libero in memoria;
4. Caricare la pagina nel frame tramite un'operazione disco pianificata, la parte più lenta del processo;
5. Aggiornare le tabelle per indicare che la pagina è ora in memoria; impostare il bit valido = v;
6. Riavviare l'istruzione che ha causato il page fault;

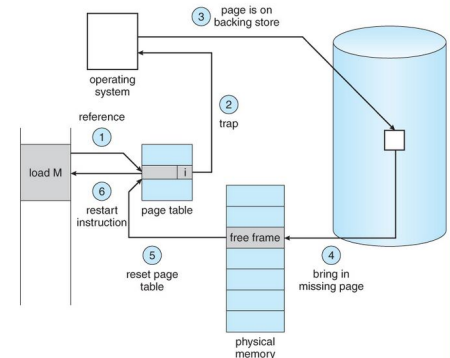


Figura 7.7: Page fault

7.11.3 Costo di un page fault

Supponiamo un tasso di page fault $0 < p < 1$; se $p = 0$ nessun page fault, se $p = 1$ ogni riferimento è un fault.

Tempo di accesso effettivo:

$$\text{EAT} = (1 - p) \times \text{accesso in memoria} + p (\text{overhead page fault} + \text{swap page out} + \text{swap page in})$$

Tempo di accesso in memoria = 200 nanosecondi; tempo medio di gestione del page fault = 8 millisecondi; un accesso su 1.000 causa un page fault. Sembra un numero piccolo, ma non lo è!

Ipotizzando un tempo di accesso RAM = 200 nanosecondi, tempo medio di gestione del page fault = 8 millisecondi:

$$\text{EAT} = (1 - p)200 + p (8 \text{ millisecondi}) = 200 + p7.999.800$$

Se un accesso su 1.000 causa un page fault, allora $\text{EAT} = 8,2$ microsecondi.

Questo è un rallentamento di un fattore $8,2 / 0,2 = 41!!$ Non è per niente buono! Peggio della TLB.

Questa è probabilmente la fonte principale di rallentamenti quando si elabora qualcosa.

7.12 Aspetti nel richiedere paging

Caso estremo - avviare un processo senza pagine in memoria. La prima fetch genererà subito un page fault.

Il sistema operativo imposta il puntatore dell'istruzione alla prima istruzione del processo, che non è residente in memoria, causando un page fault. E questo accade per ogni pagina del processo al primo accesso, il che è noto come **pure demand paging**.

In realtà, una certa istruzione potrebbe accedere a più pagine → più page fault. Considera il fetch e decode di un'istruzione che aggiunge 2 numeri dalla memoria e memorizza il risultato nuovamente in memoria. Il dolore (pain) è diminuito a causa della località di riferimento, Locality of reference.

Il supporto hardware necessario per il demand paging include:

Tabella delle pagine con bit valido/invalido

Memoria secondaria (dispositivo di swap con spazio di swap)

Riavvio dell'istruzione

7.13 Instruction Restart

Considera un'istruzione che può accedere a più locazioni, indirizzi, di memoria. La sorgente e la destinazione in alcuni casi possono sovrapporsi.

- Back move
- Auto incremento/decremento della locazione
- Riavviare l'intera operazione? Cosa succede se l'istruzione destinataria è overlapped?

7.14 Strategia di sostituzione delle pagine

Quando si verifica un page fault, il sistema operativo deve portare la pagina desiderata dalla memoria secondaria nella memoria principale.

La maggior parte dei sistemi operativi mantiene una lista di frame liberi – un pool di frame liberi per soddisfare tali richieste.

Il puntatore è inserito proprio nel frame e quando un frame è libero utilizzala per non allocare un'altra struttura per contenere la lista di frame.



I sistemi operativi tipicamente allocano i frame liberi tramite una tecnica nota come zero-fill-on-demand – il contenuto dei frame viene azzerato prima di essere allocato. All'avvio del sistema, tutta la memoria disponibile viene inserita nella lista dei frame liberi.

7.14.1 Passi nel richiedere Paging - Worse Case

1. Trap all'operating system con conseguente context switch
2. Salva i registri utente e lo stato del processo
3. Determina che l'interruzione è stata causata da un page fault
4. Controlla che il riferimento alla pagina sia legale, altrimenti abort, e determina la posizione della pagina sul disco
5. Emetti una lettura dal disco a un frame libero:
 - Attendi in una coda per questo dispositivo fino a quando la richiesta di lettura non viene soddisfatta
 - Attendi il tempo di seek e/o latenza del dispositivo
 - Inizia il trasferimento della pagina a un frame libero
6. Mentre aspetti, assegna la CPU a qualche altro utente
7. Ricevi un'interruzione dal sottosistema I/O del disco (I/O completato)
8. Salva i registri e lo stato del processo per l'altro utente
9. Determina che l'interruzione è stata causata dal disco
10. Correggi la tabella delle pagine e altre tabelle per mostrare che la pagina è ora in memoria
11. Attendi che la CPU venga assegnata a questo processo di nuovo
12. Ripristina i registri utente, lo stato del processo e la nuova tabella delle pagine, quindi riprendi l'istruzione interrotta

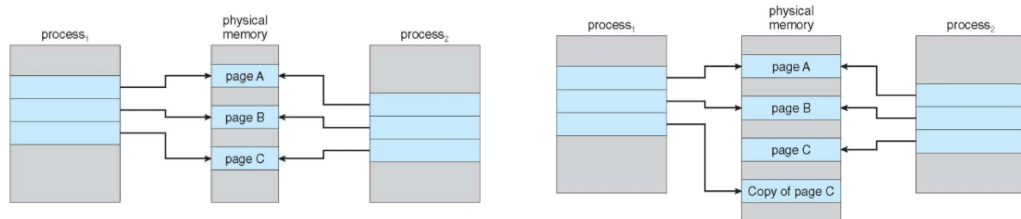
7.14.2 Demand Paging Optimizations

Lo swap space è più veloce dell'I/O del file system anche se sullo stesso dispositivo; lo swap viene allocato in blocchi più grandi, richiedendo meno gestione rispetto al file system; copia l'intera immagine del processo nello spazio di swap (backing store) al momento del caricamento del processo, quindi pagina dentro e fuori dallo spazio di swap; usato in BSD Unix più vecchio; demand page dal programma binario su disco, ma scarta piuttosto che paginare quando si libera un frame; usato in Solaris e BSD attuale; è ancora necessario scrivere nello spazio di swap per le pagine non associate a un file (come stack e heap) - memoria anonima - o per le pagine modificate in memoria ma non ancora scritte indietro al file system; i sistemi mobili tipicamente non supportano lo swapping, invece demand page dal file system e reclamano le pagine di sola lettura (come il codice).

7.14.3 Copy-on-Write

Più efficiente perché è già in memoria. Tutto va bene se leggono e possono rimanere condivise, ma quando iniziano a modificarle devono essere copiate. Permette di efficientare l'utilizzo di memoria, si copia solo quelle che vengono modificate.

Copy-on-Write (COW) consente sia al processo padre che a quello figlio di condividere inizialmente le stesse pagine in memoria. Se uno dei processi modifica una pagina condivisa, solo allora la pagina viene copiata. COW consente una creazione di processi più efficiente poiché vengono copiate solo le pagine modificate. In generale, le pagine libere vengono allocate da un pool di pagine zero-fill-on-demand. Il pool dovrebbe sempre avere frame liberi per un'esecuzione rapida del demand page; non si vuole dover liberare un frame oltre ad altre elaborazioni su un page fault. Perché azzerare una pagina prima di allocarla? La variazione `vfork()` della system call `fork()` sospende il processo padre e il figlio utilizza lo spazio degli indirizzi copy-on-write del padre. Progettato per far sì che il figlio chiami `exec()`, molto efficiente.



7.14.4 Cosa succede se non ci sono frame liberi?

Come per gli algoritmi di scheduling, possiamo implementare la soluzione più semplice usando la casualità: prendere un frame in memoria, metterlo nell'HD e caricare la pagina necessaria nella RAM; questa soluzione non è ottimale!

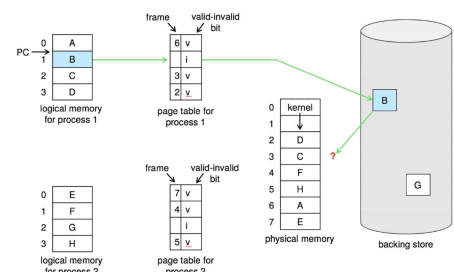
Questa tecnica, indipendentemente dall'algoritmo, si chiama **sostituzione di pagina** - trovare una pagina in memoria non realmente in uso, rimuoverla e caricare la pagina necessaria.

Si previene la sovra-allocazione della memoria modificando la routine di gestione dei page fault in modo da includere la sostituzione delle pagine.

Possiamo farlo usando il **bit di modifica (dirty bit)** per ridurre l'overhead dei trasferimenti di pagina - solo le pagine modificate vengono scritte su disco.

Se il dirty bit è impostato significa che il valore nella RAM è cambiato e prima di caricare la nuova pagina devo scrivere il valore nell'HD.

La sostituzione delle pagine completa la separazione tra memoria logica e fisica - una grande memoria virtuale può essere fornita con una memoria fisica più piccola.



Questo processo significa accedere all'HD **due volte**.

7.14.5 Sostituzione di pagina di base

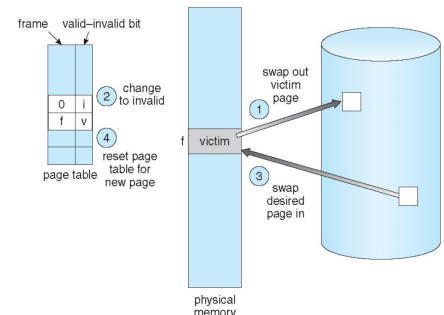
1. Trovare la posizione della pagina desiderata su disco
2. Trovare un frame libero:

Se c'è un frame libero, usarlo

Se non c'è un frame libero, usare un algoritmo di sostituzione delle pagine per selezionare un frame vittima

Scrivere il frame vittima su disco se è sporco (= modificato mentre era in RAM)

3. Caricare la pagina desiderata nel frame (appena) libero; aggiornare le tabelle delle pagine e dei frame
4. Continuare il processo riavviando l'istruzione che ha causato il trap



7.15 Algoritmi di sostituzione di pagine e frame

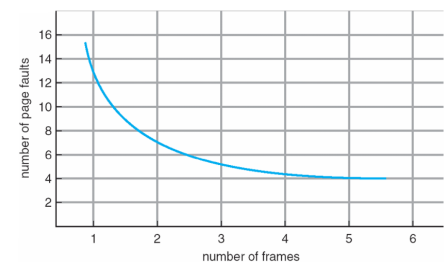
L'algoritmo di allocazione dei frame determina:

Quanti frame assegnare a ciascun processo

Quali frame sostituire; idealmente sostituisco solo la pagina che non mi servirà presto (ma non conosco il futuro)

Algoritmo di sostituzione delle pagine:

Si vuole il tasso di page fault più basso sia al primo accesso che ai successivi

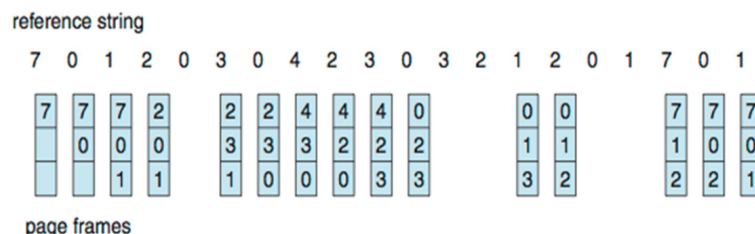


In questa immagine si può vedere che più frame slot sono associati a ogni processo, minore è la probabilità di avere page fault.

7.15.1 Algoritmo FIFO

Stringa di riferimento: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1;

3 frame (3 pagine possono essere in memoria contemporaneamente per processo)



Con questo metodo si hanno 15 page fault; si può fare meglio?

7.15.2 Algoritmo ottimale (?)

Sostituisce la pagina che non verrà usata per il periodo più lungo; 9 page fault è il risultato ottimale per l'esempio. Ma come detto non possiamo leggere il futuro; TUTTAVIA questa è la soluzione ottimale, quindi possiamo usarla per misurare le prestazioni degli altri algoritmi.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

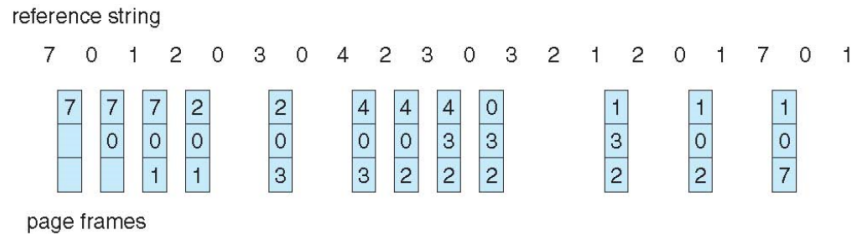
7	7	7	2		2		2		2		7
	0	0	0		0		0		0		0
		1	1		3		3		1		1

page frames

7.15.3 Algoritmo LRU (Least Recently Used)

Abbiamo visto che il programma non può conoscere il futuro, ma possiamo usare la conoscenza del passato!

Sostituisce la pagina che non è stata usata per il periodo di tempo più lungo; sfrutta il principio di località temporale. Si associa a ogni pagina il tempo dell'ultimo utilizzo.



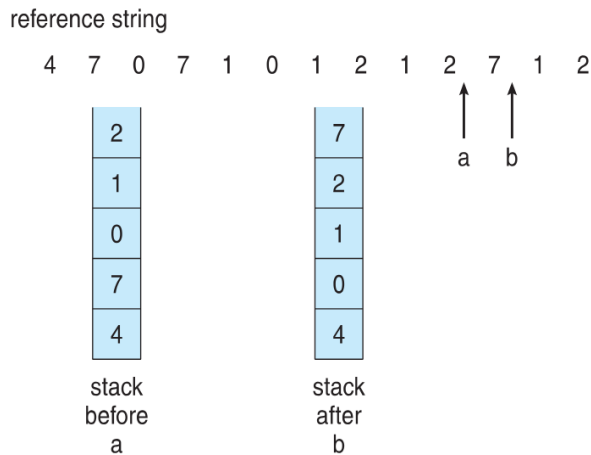
12 fault - meglio di FIFO ma peggio di OPT; generalmente un buon algoritmo e ampiamente usato. Ma come si implementa?

Implementazione con contatore: Ogni voce di pagina ha un contatore; ogni volta che la pagina viene referenziata, si copia il clock nel contatore. Quando una pagina deve essere sostituita, si cerca il contatore con il valore minore; è necessaria una ricerca nella tabella.

Implementazione con stack: Si mantiene uno stack di numeri di pagina in forma di lista doppiamente concatenata. Quando una pagina viene referenziata, viene spostata in cima e sono necessari 6 puntatori da modificare.

Ma ogni aggiornamento è più costoso e non è necessaria una ricerca per la sostituzione.

LRU e OPT sono casi di algoritmi a stack che non hanno l'anomalia di Belady. Si usa uno stack per registrare i riferimenti alle pagine più recenti.



7.15.4 Algoritmi LRU approssimati

LRU richiede hardware speciale ed è comunque lento e costoso in termini di memoria (4 byte per ogni voce).

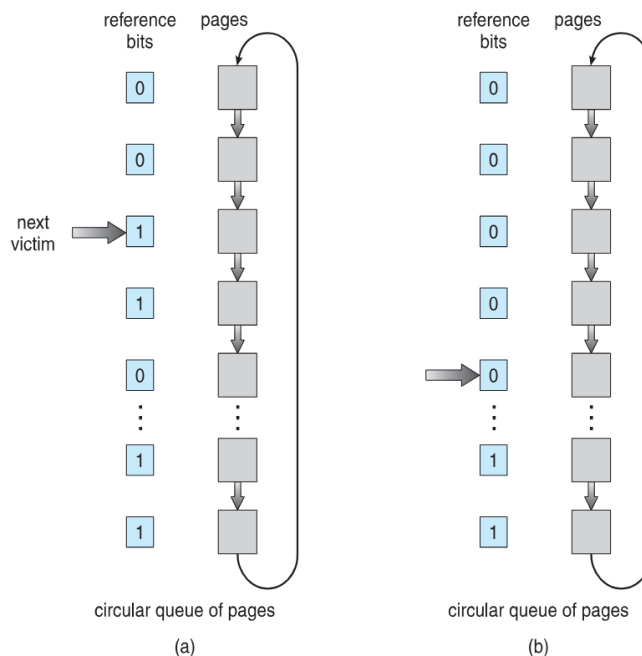
Possiamo semplificare usando un solo bit: il **bit di riferimento**. Inizialmente a ogni pagina viene associato un bit con valore 0; quando la pagina viene referenziata il bit viene impostato a 1. Se il SO necessita di memoria per un processo, può sostituire qualsiasi pagina con il bit a 0 (se ne esiste una).

Nel caso peggiore, l'algoritmo scorre l'intera lista.

Algoritmo second-chance: Generalmente FIFO, più il bit di riferimento fornito dall'hardware con sostituzione **Clock**. Se la pagina da sostituire ha:

- Bit di riferimento = 0 -> sostituirla
- Bit di riferimento = 1 allora:

impostare il bit di riferimento a 0, lasciare la pagina in memoria
sostituire la pagina successiva, con le stesse regole



7.15.5 Miglioramento del Second-Chance Algorithm

Possiamo migliorare l'algoritmo usando il bit di riferimento e il bit di modifica (se disponibile) in concerto. Prendiamo la coppia ordinata (riferimento, modifica):

- (0, 0) né recentemente usata né modificata — la migliore pagina da sostituire
- (0, 1) non recentemente usata ma modificata — non così buona, deve essere scritta prima della sostituzione
- (1, 0) recentemente usata ma pulita — probabilmente sarà usata di nuovo presto
- (1, 1) recentemente usata e modificata — probabilmente sarà usata di nuovo presto e deve essere scritta prima della sostituzione

Quando viene chiamata la sostituzione di pagina, si utilizza lo schema clock ma si usano le quattro classi per sostituire la pagina nella classe non vuota più bassa. Potrebbe essere necessario cercare più volte nella coda circolare.

7.15.6 Algoritmi basati sul conteggio

Si mantiene un contatore del numero di riferimenti effettuati a ogni pagina; non comune.

Algoritmo LFU (Least Frequently Used):

- Sostituisce la pagina con il contatore più basso
- Può avere il problema di sostituire sempre le "pagine appena arrivate" che hanno il contatore impostato a 1

Algoritmo MFU (Most Frequently Used):

Si basa sull'argomento che la pagina con il contatore più basso è stata probabilmente appena caricata e non è ancora stata utilizzata

7.15.7 Page-Buffering Algorithms

Prendiamo un pool di frame liberi, sempre disponibile quando necessario, non trovato al momento del page fault. Leggiamo la pagina in un frame libero e selezioniamo una vittima da espellere e aggiungiamo al pool dei frame liberi. Quando è conveniente, espelliamo la vittima, possibilmente tenendo una lista di pagine modificate. Quando lo storage secondario è altrimenti inattivo, scriviamo le pagine lì e impostiamo a non sporco. Possibilmente, manteniamo intatti i contenuti del frame libero e annotiamo cosa c'è dentro; se viene referenziato di nuovo prima di essere riutilizzato, non c'è bisogno di caricare nuovamente i contenuti da disco. Generalmente utile per ridurre la penalità se viene selezionato il frame vittima sbagliato.

7.15.8 Applications and Page Replacement

Tutti questi algoritmi di sostituzione delle pagine sono basati su ipotesi e non possono prevedere con precisione il futuro accesso alle pagine. Tuttavia, alcune applicazioni, come i database, hanno una migliore conoscenza dei loro modelli di accesso alla memoria e possono utilizzare questa conoscenza per ottimizzare le prestazioni. Le applicazioni che richiedono molta memoria possono causare un doppio buffering, in cui il sistema operativo mantiene una copia della pagina in memoria come buffer I/O, mentre l'applicazione mantiene la pagina in memoria per il proprio lavoro. In questi casi, il sistema operativo può fornire un accesso diretto al disco, bypassando il buffering e altre funzionalità del sistema operativo, consentendo all'applicazione di gestire direttamente l'I/O del disco.

7.16 Allocazione dei frame ai processi

Ogni processo necessita di un numero minimo di frame; il massimo è ovviamente il numero totale di frame nel sistema.

Due principali schemi di allocazione: allocazione fissa e allocazione a priorità, oltre a molte varianti.

7.16.1 Allocazione fissa

Ad esempio, se ci sono 100 frame (dopo aver allocato i frame per il SO) e 5 processi, si assegnano 20 frame a ciascun processo. Si mantiene una parte come pool di frame liberi.

7.16.2 Allocazione proporzionale

Si alloca in base alla dimensione del processo; dinamica in funzione del grado di multiprogrammazione, le dimensioni dei processi cambiano.

In entrambi i tipi di allocazione non è possibile tener conto dei task ad alta/bassa priorità; idealmente si vorrebbe che quelli ad alta priorità terminassero prima, assegnando loro più memoria/CPU.

7.16.3 Allocazione globale vs. locale

Sostituzione globale - il processo seleziona un frame di sostituzione dall'insieme di tutti i frame.

- un processo può sottrarre un frame a un altro
- il tempo di esecuzione del processo può variare notevolmente
- throughput maggiore, quindi più comune

Sostituzione locale - ogni processo seleziona solo dal proprio insieme di frame allocati

- Prestazioni per processo più consistenti
- possibile sottoutilizzo della memoria
- Quindi, throughput minore

7.16.4 Recupero delle pagine

Una strategia per implementare la politica di sostituzione globale è: tutte le richieste di memoria vengono soddisfatte dalla lista dei frame liberi, invece di aspettare che la lista scenda a zero prima di iniziare a selezionare le pagine da sostituire. Il recupero delle pagine viene attivato dal kernel quando la lista scende sotto una certa soglia.

Il processo kernel che lo gestisce è chiamato **reaper**.

Questa strategia cerca di garantire che ci sia sempre sufficiente memoria libera per soddisfare le nuove richieste, ovvero che la lista dei frame liberi non si svuoti mai.

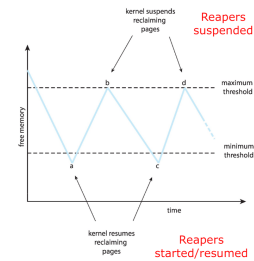
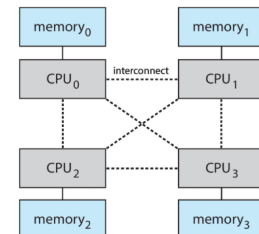


Figura 7.8: Esempio di recupero delle pagine

7.16.5 Non-Uniform Memory Access

Fino ad ora tutta la memoria è stata considerata uguale, ma molti sistemi sono NUMA (Non-Uniform Memory Access) - la velocità di accesso alla memoria varia. Considera le schede di sistema contenenti CPU e memoria, interconnesse tramite un bus di sistema - architettura di multiprocessing NUMA.



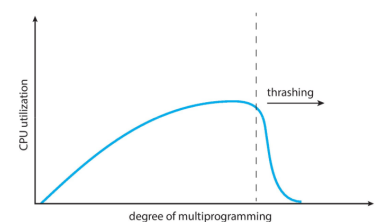
L'ottima prestazione deriva dall'allocare la memoria "vicino" alla CPU su cui è programmato il thread e modificare lo scheduler per programmare il thread sulla stessa scheda di sistema quando possibile. Soluzione adottata da Solaris creando lgroups, una struttura per tracciare i gruppi a bassa latenza CPU/memoria, usata dallo scheduler e dal pager. Quando possibile, schedula tutti i thread di un processo e allocare tutta la memoria per quel processo all'interno dell'lgroup.

7.16.6 Thrashing

Thrashing - un processo è occupato a scambiare continuamente pagine in entrata e in uscita.

Se un processo non ha un numero "sufficiente" di pagine, il tasso di page fault è molto alto: **page fault** per ottenere la pagina, **sostituzione** del frame esistente, ma rapidamente si ha bisogno di nuovo del **frame sostituito**.

Tutti questi cicli portano a:



- Basso utilizzo della CPU
- Il sistema operativo pensa di dover aumentare il grado di multiprogrammazione
- Un altro processo viene aggiunto al sistema

7.16.7 Paging su richiesta e thrashing

Perché il demand paging funziona? Modello di località:

Il processo migra da una località all'altra

Le località possono sovrapporsi

Perché si verifica il thrashing? La somma delle dimensioni delle località è maggiore della dimensione totale della memoria. Limitare gli effetti usando la sostituzione di pagina locale o a priorità.

7.16.8 Working-Set Model

Δ è la finestra di lavoro, un numero fisso di riferimenti. WSS_i è il working set del processo P_i , ovvero il numero totale di pagine referenziate nelle più recenti Δ istruzioni. D è la somma di tutti i WSS_i , ovvero il numero totale di frame richiesti da tutti i processi.

- se Δ è troppo piccolo, non coprirà l'intera località
- se Δ è troppo grande, coprirà diverse località
- se $\Delta = \infty$, coprirà l'intero programma

Se $D > m \implies$ thrashing. Policy se $D > m$, allora sospendi o swap out uno dei processi.

7.16.9 Tenere traccia del Working Set

Approssimativamente con un timer a intervalli + un bit di riferimento. Esempio: $\Delta = 10.000$; l'interruzione del timer avviene ogni 5000 unità di tempo; mantenere in memoria 2 bit per ogni pagina; ogni volta che il timer interrompe, copiare e impostare i valori di tutti i bit di riferimento a 0; se uno dei bit in memoria = 1 $\implies$ pagina nel working set. Perché non è completamente accurato? Miglioramento = 10 bit e interruzione ogni 1000 unità di tempo.

7.16.10 Frequenza dei page fault

Si stabilisce un tasso di page fault (PFF) "accettabile" e si usa la politica di sostituzione locale.

Se il tasso effettivo è troppo basso, il processo perde un frame

Se il tasso effettivo è troppo alto, il processo guadagna un frame

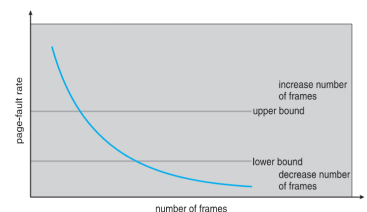


Figura 7.9: PFF

Capitolo 8

Approfondimento sulla Memoria Virtuale

8.1 Swapping delle Pagine

Cosa si intende con swapping? gioco che coinvolge Ram e disco, fino ad ora abbiamo parlato di memoria primaria. Scambio di contenuto tra backing store una parte del disco che non serve ad ospitare il file system ma che serve a fare da memoria aggiuntiva per aiutare la ram.

Un processo può essere fermo per qualche cosa, occupa memoria in ram, Toglilo dalla memoria swap out o roll out e salviamolo sul backing store e in questo modo la ram viene liberata. Usato da altri processi per espandersi o altri programmi di venir eseguito.

Un processo può essere temporaneamente spostato dalla memoria all'HD e poi riportato in memoria per continuare l'esecuzione. Lo spazio di memoria logico totale dei processi può superare la memoria fisica.

HD - Backing store - disco veloce abbastanza grande da contenere copie di tutte le immagini di memoria di tutti gli utenti; deve fornire accesso diretto a queste immagini di memoria.

Roll out, roll in - variante dello swapping usata negli algoritmi di scheduling basati sulla priorità; il processo a bassa priorità viene rimosso dalla memoria (swap out) in modo che il processo a priorità più alta possa essere caricato ed eseguito.

La parte principale del tempo di swap è il tempo di trasferimento; il tempo totale di trasferimento è direttamente proporzionale alla quantità di memoria trasferita. Il sistema mantiene una **ready queue** di processi pronti all'esecuzione che possiedono immagini di memoria su disco.

Il processo che è stato rimosso (swap out) deve tornare agli stessi indirizzi fisici?

Di solito no, ma dipende dal metodo di binding degli indirizzi; occorre anche considerare l'I/O in attesa verso/dalla memoria del processo. Ogni OS può decidere come gestire il metodo di swap:

- Swapping normalmente disabilitato
- Avviato se la quantità di memoria allocata supera una soglia
- Disabilitato nuovamente quando la domanda di memoria scende sotto la soglia

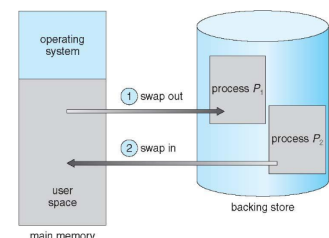


Figura 8.1: Processo di swapping

Quando lo riporto dentro posso portarlo in un posti diverso, è meno vincolato mettendolo in un'altra partizione solo se gli indirizzi sono rilocabili e dipende da come hai fatto binding degli indirizzi.

8.1.1 Tempo di Context Switch con lo Swapping

Se il prossimo processo da eseguire sulla CPU non è in memoria e la RAM è piena, occorre effettuare lo swap out di un processo e lo swap in del processo target. Il tempo di context switch può quindi essere molto elevato, ovvero la latenza per il trasferimento dati verso l'HD.

Esempio: processo da 100MB in swap verso l'hard disk con velocità di trasferimento di 50MB/sec:

Tempo di swap out di 2 secondi

Più swap in di un processo delle stesse dimensioni

Tempo totale del componente di swapping nel context switch: 4 s

È possibile ridurre il tempo sapendo quanta memoria viene effettivamente usata. Chiamate di sistema per informare l'OS sull'uso della memoria tramite `request_memory()` e `release_memory()`.

Altri vincoli: se un task ha I/O in attesa non può essere rimosso (swap out), poiché l'I/O avverrebbe sul processo sbagliato; oppure si trasferisce sempre l'I/O nello spazio kernel e poi al dispositivo I/O, ma questo causa il cosiddetto double buffering, aggiungendo overhead.

Allora come funziona nei **sistemi operativi moderni**?

Lo swap viene attivato solo quando la memoria libera è quasi esaurita

Con la memoria a pagine si scambiano intere pagine dentro e fuori dalla RAM

Le pagine spostate su disco vengono in genere salvate in un "file di paging"

Lo swapping è stato implementato anche sulle pagine. Non tutte le pagine di un processo devono stare in memoria.

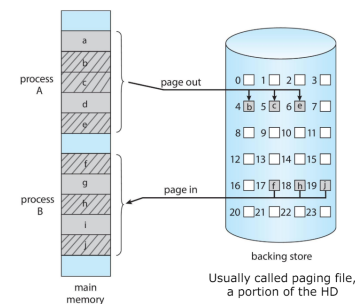


Figura 8.2: Swapping con paginazione

8.1.2 Swapping sui Sistemi Mobili

Non si fa, si fa una cosa più semplice: chiedere all'applicazione di salvarsi i dati altrimenti il kernel la termina.

Lo swapping non viene tipicamente usato perché lo storage è piccolo, il flash drive ha un numero limitato di cicli di scrittura e il bus tra la memoria flash e la CPU è limitato.

Quindi, invece di usare lo swapping quando la memoria libera è bassa:

- **iOS** chiede alle app di cedere volontariamente la memoria allocata; i dati in sola lettura vengono eliminati e ricaricati dal flash se necessario. Il mancato rilascio può portare alla terminazione.
- **Android** termina le app in caso di memoria libera insufficiente, ma prima salva lo stato dell'applicazione nel flash per un riavvio rapido.

Entrambi i sistemi operativi supportano la paginazione come discusso di seguito.

8.2 Struttura della Tabella delle Pagine

Le strutture di memoria per la paginazione possono diventare enormi con metodi diretti. Si consideri uno spazio di indirizzamento logico a 32 bit, con pagine da 4KB: la tabella avrebbe 1 milione di voci e lo spazio totale per OGNI processo è di 4MB solo per ottenere lo spazio di indirizzamento fisico, i quali devono essere contigui.

Una soluzione semplice è dividere la tabella delle pagine in unità più piccole e caricarla solo parzialmente:

1. Shared Pages

2. Paginazione Gerarchica
3. Tabelle delle Pagine con Hashing
4. Tabelle delle Pagine Invertite

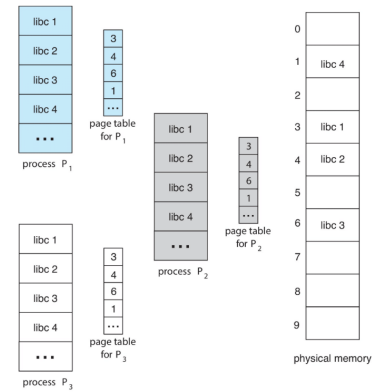
8.2.1 Shared Pages

Shared code

Una sola copia in modalità read-only (reentrant), è condivisa tra i processi (eg. text editors, compilers, windows systems). Il concetto è simile a thread multipli che condividono lo stesso process space. Inoltre è anche molto utile per le comunicazioni interprocesso (IPC), se la condivisione di pagine read-write è consentita.

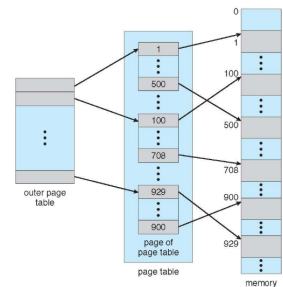
Codice privato e dati

Ogni processo tiene una copia privata del codice e dei dati. Le pagine di codice e dati privati possono apparire ovunque nella logical address space.

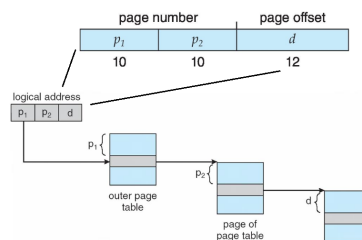


8.2.2 Paginazione gerarchica

Con questo metodo si suddivide lo spazio di indirizzamento logico in più tabelle delle pagine. Una tecnica semplice è la tabella delle pagine a due livelli: in questo modo si carica solo la sezione di interesse. Con indirizzi a 32 bit e pagine da 4 KB, l'indirizzo logico si divide in:



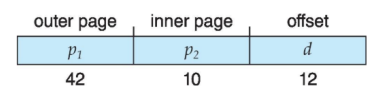
- un numero di pagina di 20 bit, suddiviso in due parti da 10 bit ciascuna:
 - p1 per selezionare la pagina delle page table, cioè la **pagina esterna**
 - p2 per selezionare la riga specifica del “blocco” di page table, la **pagina interna**
- un offset di pagina di 12 bit



ATTENZIONE ERRORE: non è 10 10 12 bensì 10 12 10 ETA da cambiare perché è su 3 livelli e non 2

Spazio di Indirizzamento Logico a 64 bit

Anche uno schema di paginazione a due livelli potrebbe non essere sufficiente; se la dimensione della pagina è ancora 4 KB, la tabella delle pagine ha 2^{52} voci e le tabelle delle pagine interne potrebbero essere 2^{10} , vedi figura affianco.



La tabella delle pagine esterna ha 2^{42} voci, ovvero 4 terabyte per processo! Una soluzione potrebbe essere aggiungere un ulteriore livello di tabella delle pagine esterna, vedi figura affianco.

Il problema qui è che sono necessari 4 accessi in memoria per ottenere una singola posizione di memoria fisica, il che non è una soluzione brillante.

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

8.2.3 Tabelle delle Pagine con Hashing

Gli spazi di indirizzamento comuni sono > 32 bit. Il numero di pagina virtuale viene sottoposto a hashing in una tabella delle pagine; questa tabella contiene un elenco di elementi a cui è stato applicato lo stesso hash.

Ogni elemento contiene:

- il numero di pagina virtuale
- il valore del frame di pagina mappato
- un puntatore all'elemento successivo

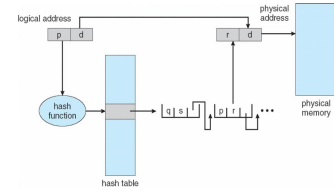


Figura 8.3: Tabella delle pagine con hashing

Se si dispone di uno spazio di indirizzamento a 64 bit, 12 bit sono per l'offset, quindi 52 per l'indirizzo di pagina. Si può usare una tabella hash da 4KB (2^{12}), che tipicamente si adatta a una singola pagina e rimane sempre in memoria per ciascun processo.

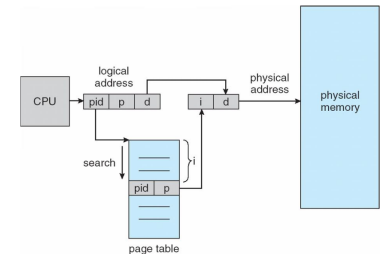
Si accede alla pagina specifica tramite essa. Se tutta la memoria viene usata dal processo, questo non è efficiente. Tuttavia, nella stragrande maggioranza dei casi nessun processo utilizzerà tutti i 2^{52} pagine.

Ovviamente di possono avere collisioni, ma non è un grosso problema se la tabella hash è abbastanza grande. Altrimenti, possono utilizzare **open addressing** (cerca un altro posto) o **linear chaining** (una lista).

8.2.4 Tabella delle Pagine Invertita

Invece di avere una tabella delle pagine per ogni processo che tiene traccia di tutte le pagine logiche possibili, si ha un'unica voce per ogni pagina reale della memoria!

Ogni voce consiste nell'indirizzo virtuale della pagina memorizzata in quella locazione di memoria reale, con informazioni sul processo che possiede quella pagina.



Riduce la memoria necessaria per memorizzare ogni tabella delle pagine, ma aumenta il tempo necessario per ricercare nella tabella quando si verifica un riferimento a una pagina.

Una soluzione potrebbe essere usare l'hashing per limitare la ricerca a una sola voce; anche il TLB può accelerare l'accesso.

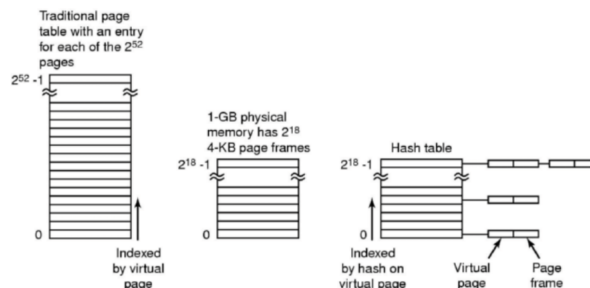


Figura 8.4: Comparativa tra tabella delle pagine tradizionale e tabella delle pagine invertita

8.3 Oracle SPARC Solaris

Oracle SPARC Solaris è un sistema a 64-bit che è molto vicino e integrato con l'hardware. È basato su due hash table:

- Una per il kernel
- Una per i processi utente.

Ogni una mappa memoria virtuale a memoria fisica. Più efficiente che avere due separate hash table per pagina.

Ogni entry ha un address e una span, che indica il numero di pagine che la entry rappresenta.

TLB possiede la translation table entries (TTEs), per un accesso hardware più rapido. La cache di TTEs risiede nel translation storage buffer (TSB), includendo un entry per le pagine recentemente accedute.

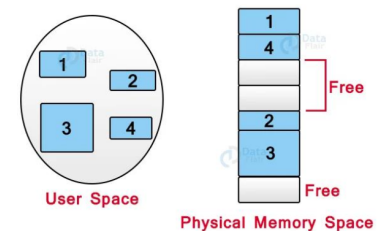
Virtual address reference causa una TLB search:

Se è miss, l'hardware va a cercare nella memoria TSB guardando dentro la TTE corrispondente all'address.

- Se è hit, la CPU copia la TSB entry nella TLB e la traduzione viene completata
- Se no match, il kernel interrompe la ricerca nella hash table. Il kernel crea una TTE dall'appropriata hash table e la memorizza nella TSB. Interrupt handler ritorna il controllo alla MMU, la quale completa la traduzione dell'address.

8.4 Segmentazione

L'uso di pagine di dimensioni fisse apre il problema della frammentazione interna: ogni processo viene abbinato a un certo numero di pagine piuttosto che alla memoria di cui ha bisogno. Esiste quindi un modo per assegnare esattamente la quantità di memoria di cui un processo ha bisogno, evitando così qualsiasi spreco di memoria? La risposta è usare la **segmentazione**.



Un processo è diviso in blocchi; i blocchi in cui viene suddiviso un programma, che non devono essere necessariamente tutti della stessa dimensione, sono chiamati segmenti.

- Un blocco per lo stack
- Un blocco per lo heap
- Un blocco per la libreria 1

Ogni processo è diviso in diversi segmenti, tutti caricati in memoria durante l'esecuzione, anche se non necessariamente in modo contiguo.

Una tabella memorizza le informazioni su tutti questi segmenti: la base e il limite (lunghezza).

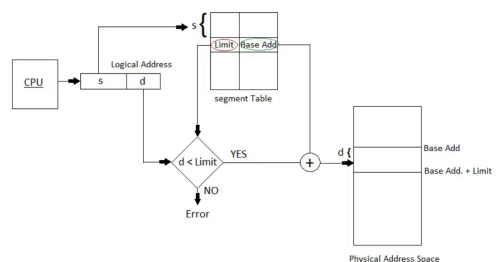
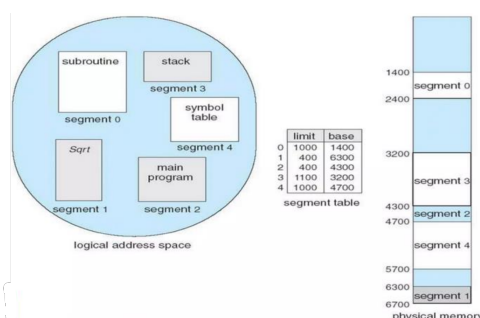
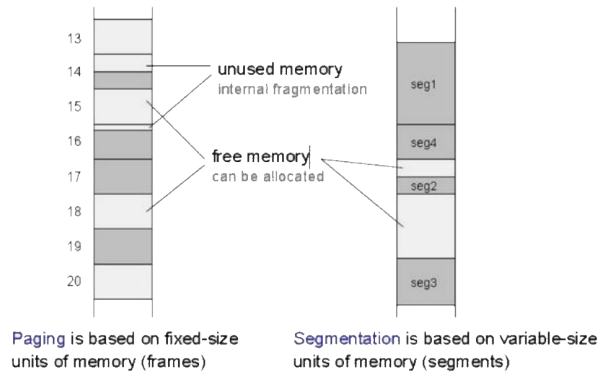


Figura 8.5: Segmentazione: da logico a fisico

8.4.1 Paginazione vs Segmentazione

Feature	Paging	Segmentation	Example
Usage	Systems with a small amount of memory to prevent external fragmentation.	Systems with a large amount of memory to provide better security and flexibility.	A word processing program with a small memory footprint would be a good candidate for paging. A database program with a large memory footprint would be a good candidate for segmentation.
Working	The program is divided into pages and stored in the main memory. When the program needs to access a page, the operating system transfers it from the secondary memory to the main memory.	The program is divided into segments and stored in the secondary memory. When the program needs to access a segment, the operating system transfers it from the secondary memory to the main memory.	A program that is 100 pages long is divided into 4KB pages in paging. In segmentation, the program is divided into 3 segments: code, data, and stack.
Page size	Fixed	Variable	In paging, the page size is typically 4KB. In segmentation, the segment size can be any size.
Loading of pages	Dependent	Independent	In paging, all pages of a process must be loaded into memory before the process can be executed. In segmentation, pages can be loaded into memory independently.
Prevention of internal fragmentation	No	Yes	Paging can lead to internal fragmentation because a free page may not be large enough to accommodate a requesting page. Segmentation can prevent internal fragmentation because segments can be of different sizes.
Prevention of external fragmentation	Yes	No	Paging can prevent external fragmentation because pages are loaded into memory in blocks. Segmentation cannot prevent external fragmentation because segments can be loaded into memory independently.
Memory management overhead	Lower	Higher	The memory management overhead in paging is lower than in segmentation because paging is a simpler technique.
Security	Lower	Higher	Paging cannot provide as much security as segmentation because segments can be loaded into memory independently.
Flexibility	Lower	Higher	Paging is less flexible than segmentation because pages must be loaded into memory in blocks.
Performance	Faster	Slower	Paging is faster than segmentation because it is a simpler technique.



Esempio di Architettura Intel IA-32

Lascia stare

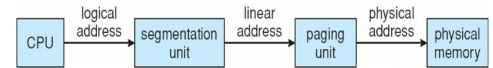
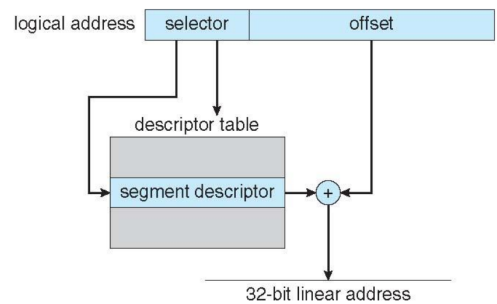


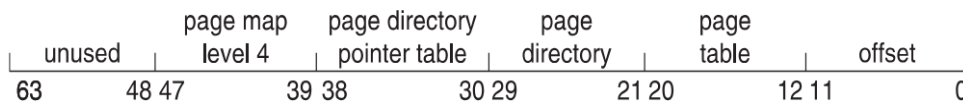
Figura 8.6: Conversione degli indirizzi di memoria

Le unità di paginazione formano l'equivalente dell'MMU; le dimensioni delle pagine possono essere 4 KB o 4 MB, con tabelle delle pagine a due livelli.



Esempio di Intel x86-64

L'attuale generazione dell'architettura CISC ha 64 bit, il che è enorme (> 16 exabyte). In pratica viene implementato solo l'indirizzamento a 48 bit.



8.5 Example: ARM Architecture

Nei moderni processori ARM installati sugli smartphone sono CPU 64bit con particolare attenzione ai consumi energetici. Un livello di paginazione per sezione, due livelli si pagine più piccole.

Due livelli di TLBs

- L'outer ha due micro TLBs, uno per le istruzioni e uno per i dati.
- L'inner è una TLB singola
- Viene controllato il primo interno, successivamente vengono controllati gli esterni mancanti, e in caso di mancata corrispondenza viene eseguita la ricerca nella tabella delle pagine da parte della CPU.

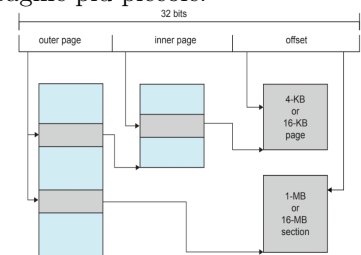


Figura 8.7: Architettura ARM

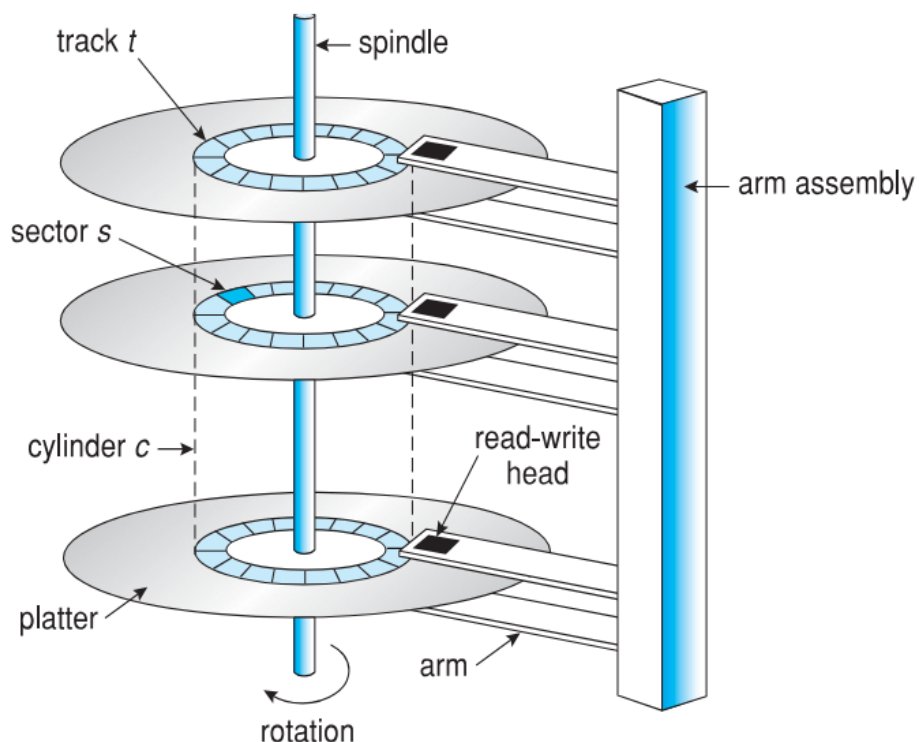
Capitolo 9

Sistemi di Archiviazione di Massa

La maggior parte della memoria secondaria nei computer moderni è costituita da **hard disk drive**, HDD, e dispositivi di **memoria non volatile**, NVM.

HDD fanno ruotare piatti di materiale a rivestimento magnetico sotto testine di lettura-scrittura mobili:

- I dischi ruotano da 60 a 250 volte al secondo
- La velocità di trasferimento è la velocità con cui i dati scorrono tra il disco e il computer, ≈ 1 Gb/sec
- Il tempo di posizionamento (tempo di accesso casuale) è il tempo necessario per spostare il braccio del disco al cilindro desiderato (seek time) e il tempo per far ruotare il settore desiderato sotto la testina (latenza rotazionale), da 3ms a 12ms
- Lo schianto della testina avviene quando la testina del disco entra in contatto con la superficie del disco – molto grave



Latenza di accesso (tempo medio di accesso) = tempo medio di seek + latenza media
Per un disco molto veloce: $3\text{ ms} + 2\text{ ms} = 5\text{ ms}$.

Tempo medio di I/O = tempo medio di accesso + (dati da trasferire / velocità di trasferimento) + overhead del controller

Esempio: trasferire un blocco da 4KB su un disco a 7200 RPM con un seek time medio di 5ms, velocità di trasferimento di 1Gb/sec e overhead del controller di 0.1ms.

A 7200 RPM = 120 giri/sec = 8.333ms/giro; nel caso peggiore bisogna attendere 1 rotazione completa, ma in media si attende mezza rotazione, pari a 4.17ms.

Tempo di trasferimento = $4\text{KB} / 1\text{Gb/s} * 8\text{Gb} / \text{GB} * 1\text{GB} / 1024\text{KB} = 32 / (1024^2) = 0.031\text{ ms}$

Tempo medio di I/O per un blocco da 4KB = $5\text{ms} + 4.17\text{ms} + 0.1\text{ms} + 0.031\text{ms} = 9.301\text{ms}$

9.1 Dispositivi di Memoria Non Volatile

L’NVM include solid-state drive e chiavette USB. Questi dispositivi sono più affidabili e **più veloci** degli HDD, ma più costosi.

La lettura e scrittura avviene in incrementi di pagina (come i settori), ma non si può sovrascrivere in-place: occorre prima cancellare, e le cancellazioni avvengono in incrementi di blocco più grandi. Questi dispositivi possono essere cancellati un numero limitato di volte, espresso come **drive writes per day**, DWPd.

L’Ssd è un dispositivo complesso che integra un processore per verificare lo stato di tutti i settori, cancellare e scrivere blocchi, ecc.

9.2 Scheduling del Disco

Il sistema operativo è responsabile dell’uso efficiente dell’hardware; per il disco ciò significa avere un accesso rapido e una buona **larghezza di banda** del disco. La **bandwidth** è il numero totale di byte trasferiti, diviso per il tempo totale tra la prima richiesta di servizio e il completamento dell’ultimo trasferimento.

Le sorgenti delle richieste di I/O su disco sono molte: OS, processi di sistema, processi utente; includono tutte scrittura/lettura, indirizzo del disco, indirizzo di memoria, numero di settori da trasferire. L’OS mantiene una coda di richieste per dispositivo.

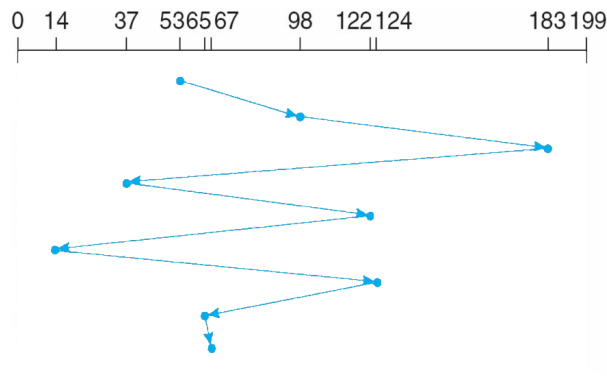
In passato l’HD non era sofisticato come oggi; l’OS doveva occuparsi di tutto: gestione della coda e scheduling della testina del disco. Ora il controllo è affidato all’HD e l’OS deve solo passare gli indirizzi dei blocchi, lasciando che il disco ordini le richieste.

Esempio: vogliamo gestire la seguente lista di richieste usando diversi algoritmi di scheduling:

98, 183, 37, 122, 14, 124, 65, 67

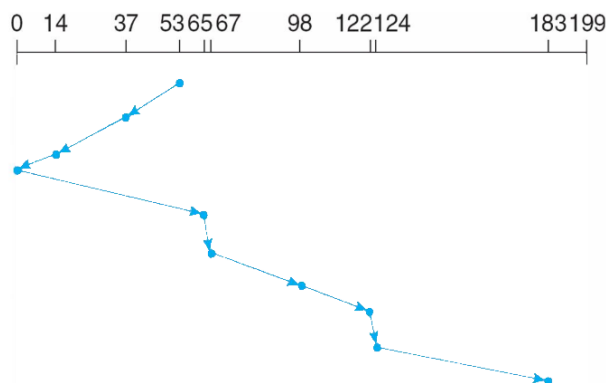
La testina parte dalla posizione 53

9.2.1 FCFS



9.2.2 SCAN - algoritmo dell'ascensore

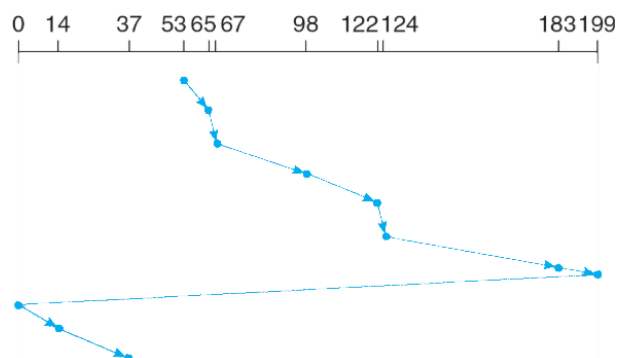
Il braccio del disco parte da un'estremità e si sposta verso l'altra. Questo non è equo: gli ultimi valori devono aspettare molto a lungo.



NOTA: non ci sono richieste da 0 a 65, quindi possiamo saltare quella zona.

9.2.3 C-SCAN

Fornisce un tempo di attesa più uniforme rispetto a SCAN: la testina si sposta da un'estremità del disco all'altra, servendo le richieste man mano che procede. Quando raggiunge l'altra estremità, tuttavia, ritorna immediatamente all'inizio del disco senza servire alcuna richiesta durante il percorso di ritorno.



9.2.4 Scelta dell'Algoritmo di Scheduling del Disco

- FCFS è comune e ha un'attrattiva naturale
- SCAN e C-SCAN hanno prestazioni migliori per sistemi con un carico elevato sul disco

Linux implementa lo scheduler deadline

- Mantiene code separate per lettura e scrittura
- Per impostazione predefinita, i batch di lettura hanno la precedenza sui batch di scrittura, poiché le applicazioni tendono a bloccarsi sulle operazioni di I/O in lettura
- Schedula i batch di lettura e scrittura per l'esecuzione in ordine crescente di indirizzamento logico dei blocchi (LBA)
- Dopo ogni batch, seleziona naturalmente i batch di lettura se presenti, ma controlla anche da quanto tempo le operazioni di scrittura sono in attesa (per evitare la starvation)
- Schedula il successivo batch di lettura o scrittura secondo necessità
- Tipicamente, se un batch di lettura è in attesa da più di 500ms, ottiene la priorità sulla scrittura

Questo scheduler è adatto alla maggior parte dei casi d'uso, in particolare quelli in cui le operazioni di scrittura sono prevalentemente **asincrone**.

9.3 Pianificazione per la NVM

Non vi sono testine del disco né latenza rotazionale, ma c'è ancora margine per l'ottimizzazione. L'NVM è migliore nell'I/O casuale, l'HDD nell'I/O sequenziale; le operazioni di input/output al secondo (IOPS) sono molto più elevate con NVM (centinaia di migliaia vs centinaia).

9.4 Gestione dei Dispositivi

Un aspetto fondamentale di molte parti dell'informatica è il **Rilevamento e Correzione degli Errori**.

Il **rilevamento degli errori** determina se si è verificato un problema; viene spesso implementato tramite bit di parità.

La parità è una forma di checksum; un altro metodo di rilevamento errori è il controllo di ridondanza ciclica (CRC), che usa una funzione hash per rilevare errori su più bit.

Il **codice di correzione degli errori**, ECC, non solo rileva, ma può anche correggere alcuni errori; gli errori soft sono correggibili, quelli hard vengono rilevati ma non corretti.

9.4.1 Gestione dei dispositivi di storage

Il **low-level formatting**, o **formattazione fisica**, divide un disco in settori che il controller può leggere e scrivere. Ogni settore contiene un'intestazione, i dati e il codice di correzione degli errori. In genere ospita 512 byte di dati, ma il valore può essere configurabile.

Per usare un disco per i file, il sistema operativo deve comunque registrare sul disco le proprie strutture dati:

- **Partizionare** il disco in uno o più gruppi di cilindri, ognuno trattato come disco logico
- Eseguire la **formattazione logica**, o “creare un file system”

Per aumentare l'efficienza molti file system raggruppano i blocchi in cluster. L'I/O sul disco avviene in blocchi, quello sui file nei cluster.

La **partizione di boot/root** contiene il sistema operativo; le altre partizioni possono contenere altri sistemi operativi, altri file system, o essere raw. La partizione può essere **montata** al boot, automaticamente o manualmente.

Al momento del montaggio, viene verificata la consistenza del file system e la correttezza di tutti i metadati.

Se non corretta, correggere e riprovare

Se corretta, aggiungere alla tabella di montaggio e consentire l'accesso

Il boot block, o un programma di gestione del boot per sistemi multi-OS, può puntare al volume di boot o al boot loader, ovvero un insieme di blocchi che contengono il codice necessario per caricare il kernel dal file system.

Il boot block inizializza il sistema ed è memorizzato in ROM, firmware. Il programma Bootstrap loader è memorizzato nei boot block della partizione di boot: il master boot record MBR.

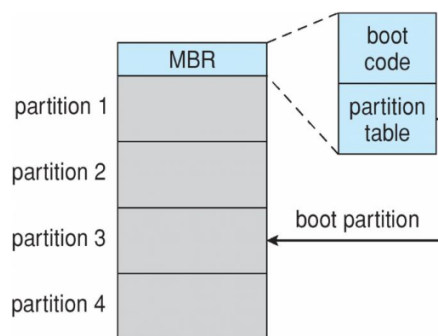


Figura 9.1: Avvio dallo storage secondario in Windows

9.4.2 Gestione dello Spazio di Swap

Utilizzato per spostare interi processi o pagine dalla DRAM all'archiviazione secondaria quando la DRAM non è sufficiente per tutti i processi. Il sistema operativo fornisce la **gestione dello spazio di swap**:

- L'archiviazione secondaria è più lenta della DRAM, quindi è importante ottimizzare le prestazioni
- Di solito sono possibili più spazi di swap, riducendo il carico di I/O su ogni singolo dispositivo
- È preferibile avere dispositivi dedicati
- Può essere in una partizione raw o in un file all'interno di un file system (per comodità)

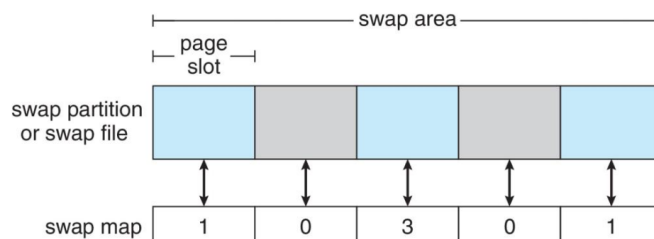


Figura 9.2: Strutture dati per lo swapping nei sistemi Linux

9.4.3 Collegamento dello Storage all'Host

I computer accedono allo storage in tre modi:

- collegato all'host
- collegato in rete
- cloud

L'accesso collegato all'host avviene tramite porte I/O locali, usando una delle varie tecnologie per collegare molti dispositivi (USB, LAN...).

Storage Collegato in Rete: NAS

Il network-attached storage (NAS) è uno storage reso disponibile tramite rete anziché tramite un canale locale (come un bus). **NFS** e **CIFS** sono protocolli comuni, implementati tramite Remote Procedure Call (RPC) tra host e storage, tipicamente su TCP o UDP su rete IP.

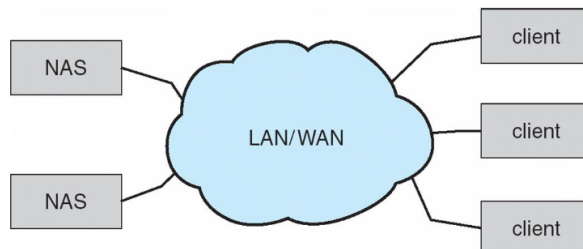


Figura 9.3: NAS

9.4.4 Archiviazione Cloud

Simile al NAS, fornisce accesso allo storage tramite rete. A differenza del NAS, viene acceduto tramite Internet o WAN verso data center remoti.

Il NAS viene presentato come un normale file system, mentre lo storage cloud è basato su API, con programmi che usano le API per fornire l'accesso. Si usano le API a causa della latenza e degli scenari di guasto (i protocolli NAS non funzionerebbero bene).

9.4.5 Array Ridondante di Dischi Indipendenti - RAID

RAID - più dischi forniscono affidabilità tramite ridondanza.

- Aumenta il **tempo medio al guasto**.
- **Tempo medio di riparazione** - periodo di esposizione durante il quale un altro guasto potrebbe causare perdita di dati
- **Tempo medio alla perdita di dati** basato sui fattori precedenti

Se i dischi speculari si guastano indipendentemente, si consideri un disco con 100.000 ore di tempo medio al guasto e 10 ore di tempo medio di riparazione. Il tempo medio alla perdita di dati è:

$MTDL = 100.000^2 / (2 * 10) = 500 * 10^6 \text{ hours or } 57,000 \text{ years!}$

Diversi miglioramenti nelle tecniche di utilizzo dei dischi coinvolgono l'uso di più dischi che lavorano in modo cooperativo.

Il **striping** del disco utilizza un gruppo di dischi come un'unica unità di archiviazione. Il RAID è organizzato in sei diversi livelli.

I sistemi RAID migliorano le prestazioni e l'affidabilità dello storage memorizzando dati ridondanti.

- Il **mirroring** o shadowing (**RAID 1**) mantiene una copia duplicata di ogni disco
- Mirror striped (**RAID 1+0**) o stripe mirror (**RAID 0+1**) offre alte prestazioni e alta affidabilità
- La parità a blocchi interlacciati (**RAID 4, 5, 6**) usa molta meno ridondanza

Il RAID all'interno di un array di storage può comunque fallire se l'array si guasta, quindi la replica automatica dei dati tra array è comune. Spesso un piccolo numero di dischi hot-spare viene lasciato non allocato, pronto a sostituire automaticamente un disco guasto e a ricostruire i dati su di esso.

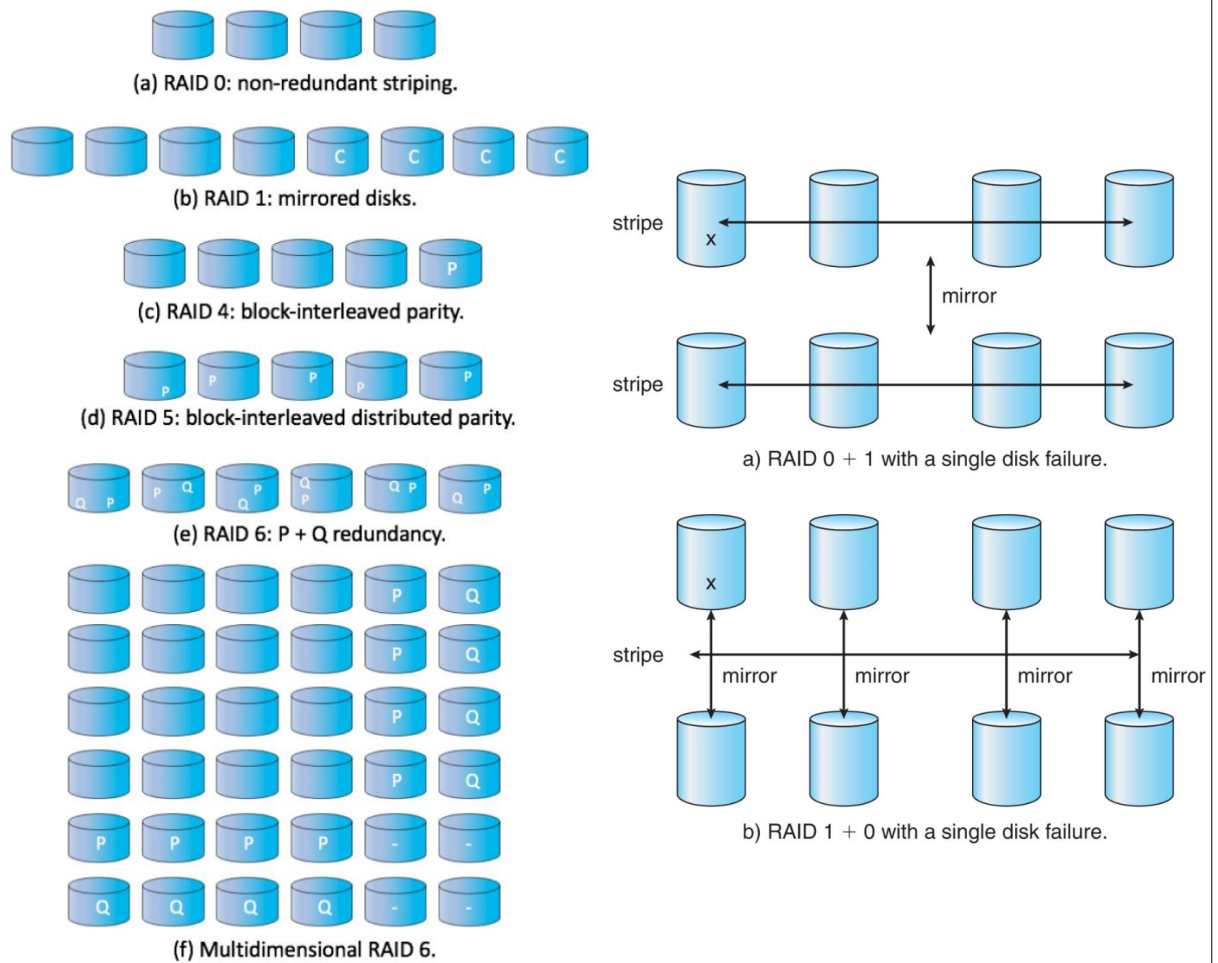


Figura 9.4: Schemi RAID

Capitolo 10

Interfaccia del File System

10.1 Cos'è un File?

Spazio di indirizzamento logico contiguo. Tipi:

- File di Dati
 - Numerico
 - Carattere
 - Binario
- File di Programma

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

Figura 10.1: Tipi di file

10.1.1 Attributi del File

- **Nome** - è la sola informazione mantenuta in forma leggibile dall'uomo
- **Identificatore** - tag unico (numero) che identifica il file all'interno del file system
- **Tipo** - necessario per i sistemi che supportano tipi diversi
- **Posizione** - puntatore alla posizione del file sul dispositivo
- **Dimensione** - dimensione corrente del file
- **Protezione** - controlla chi può leggere, scrivere, eseguire
- **Ora, data e identificazione utente** - dati per protezione, sicurezza e monitoraggio dell'uso

Le informazioni sui file sono conservate nella **struttura delle directory**, mantenuta sul disco. Molte varianti, tra cui attributi estesi del file come il checksum.

10.1.2 Struttura del File

- Nessuna - sequenza di parole, byte
- Struttura semplice - Righe, lunghezza fissa, lunghezza variabile...
- Strutture complesse - Documento formattato, File di caricamento rilocabile

Le ultime due strutture possono essere simulate con la prima inserendo opportuni caratteri di controllo. La decisione sulla struttura spetta all'**OS** o al **Programma**.

10.2 Accesso e Operazioni sui File

Sono necessari diversi dati per gestire i **file aperti**:

- **Tabella dei file aperti**: traccia i file aperti
- **Puntatore al file**: puntatore all'ultima posizione di lettura/scrittura, specifico per ogni processo che ha il file aperto
- **Contatore di aperture del file**: contatore del numero di volte in cui un file è aperto, per consentire la rimozione dei dati dalla tabella dei file aperti quando l'ultimo processo lo chiude
- **Posizione del file su disco**: cache delle informazioni di accesso ai dati
- **Diritti di accesso**: informazioni sulla modalità di accesso per processo

10.2.1 Blocco dei File

Fornito da alcuni **sistemi operativi** e file system. Simile ai lock lettore-scrittore.

Lock condiviso simile al lock lettore - più processi possono acquisirlo contemporaneamente **Lock esclusivo** simile al lock scrittore

Obbligatorio - l'accesso è negato in base ai lock detenuti e richiesti **Consultivo** - i processi possono conoscere lo stato dei lock e decidere come procedere

Esempio: provare a scrivere un file già aperto in Excel genera un errore; aprire un file in gedit di solito non lo fa.

10.2.2 Metodi di Accesso

Un file è un **record logico** di lunghezza fissa:

- Accesso Sequenziale
- Accesso Diretto
- Altri Metodi di Accesso

Accesso Sequenziale

Operazioni: **read next**, **write next**, **reset**; nessuna lettura dopo l'ultima scrittura (**rewrite**) e nessuna operazione di **seek**.

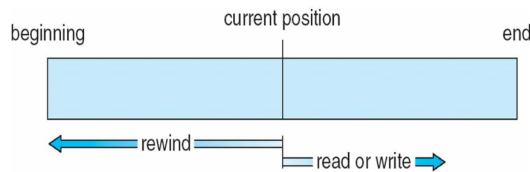


Figura 10.2: Accesso sequenziale

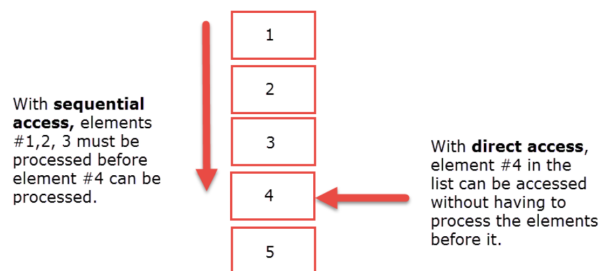
Accesso Diretto

Operazioni: leggi *n*, scrivi *n*, **Reset**, Posiziona (**seek**) a *n*. *n* = numero di blocco relativo

sequential access	implementation for direct access
<i>reset</i>	<i>cp</i> = 0;
<i>read next</i>	<i>read cp</i> ; <i>cp</i> = <i>cp</i> + 1;
<i>write next</i>	<i>write cp</i> ; <i>cp</i> = <i>cp</i> + 1;

Figura 10.3: Accesso diretto

Accesso Sequenziale vs Diretto



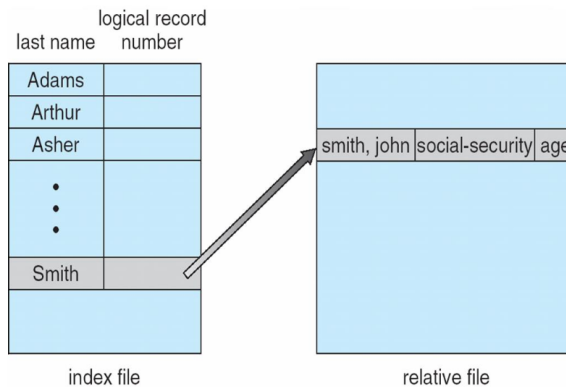
Altri Metodi di Accesso

Possono esistere altri metodi di accesso costruiti sopra i metodi di base. Generalmente comportano la creazione di un indice per il file. L'indice viene mantenuto in memoria per la rapida individuazione della posizione dei dati (si pensi al codice Universal Product Code (UPC) con i relativi record di dati).

Se l'indice è troppo grande, si crea un indice in memoria, che è un indice di un indice su disco.

Il metodo sequenziale ad accesso indicizzato di IBM (ISAM)

- Piccolo indice master, punta ai blocchi su disco dell'indice secondario
- Il file è mantenuto ordinato su una chiave predefinita
- Tutto gestito dall'OS
- Un motore simile, MyISAM, è un modo per indicizzare i DB MySQL



10.3 File Mappati in Memoria

Invece di accedere ai file di dati dall'HD ad ogni accesso, i file di dati possono essere paginati in memoria come i file di processo, risultando in accessi molto più veloci.

Si può vederlo come il "paging" di un file = una tabella delle pagine per file, eccetto ovviamente quando si verificano page-fault. Questo è noto come memory-mapping di un file.

Un file viene mappato su un intervallo di indirizzi all'interno dello spazio di indirizzamento virtuale di un processo e poi paginato secondo necessità usando il normale sistema di demand paging. Può essere usato per caricare un file da 10 GB con soli 2 GB di RAM disponibili!

Le scritture sul file vengono effettuate sui frame di pagina in memoria e non vengono immediatamente scritte su disco. Ecco perché è importante eseguire la `close()` su un file al termine della scrittura.

In questo modo i dati possono essere scritti su disco in sicurezza e i frame di memoria possono essere liberati per altri scopi.

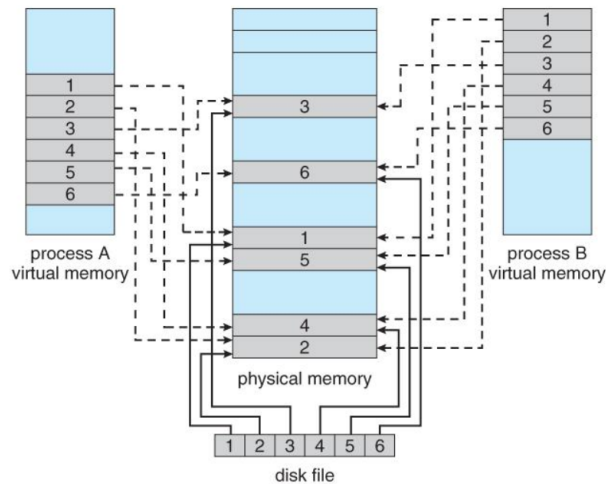


Figura 10.4: File mappati in memoria

10.4 Dischi e File System

Dalla lezione precedente:

- I dischi possono essere suddivisi in partizioni
- I dischi o le partizioni possono essere protetti da RAID contro i guasti
- Le partizioni sono note anche come minidisk, slice

Un file system governa l'organizzazione e l'accesso ai file.

L'entità contenente il file system è nota come volume; un disco o una partizione formattata è un volume.

Il disco o la partizione può essere usato grezzo (raw) - senza file system - oppure formattato con un file system. Ogni volume contenente un file system traccia anche le informazioni di quel file system nella directory del dispositivo o nella tabella dei contenuti del volume. Oltre ai file system di uso generale, esistono molti file system speciali, spesso tutti all'interno dello stesso sistema operativo o computer.

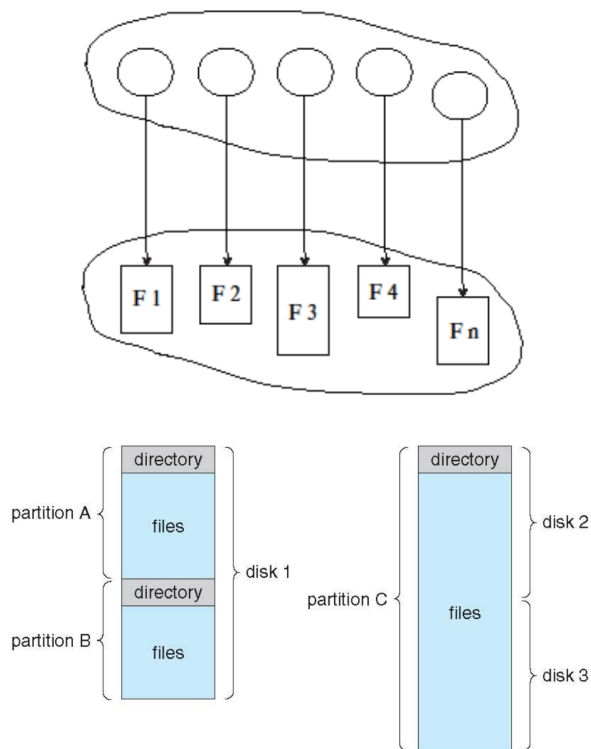
10.4.1 Tipi di File System

Parliamo principalmente di file system di uso generale, ma i sistemi spesso hanno molti file system, sia di uso generale sia speciali.

Si consideri Solaris, che ha:

- tmpfs - FS volatile basato su memoria per I/O rapido e temporaneo
- objfs - interfaccia nella memoria del kernel per ottenere i simboli del kernel per il debugging
- ctfs - file system dei contratti per la gestione dei daemon
- lofs - file system loopback che consente l'accesso a un FS al posto di un altro
- procfs - interfaccia del kernel alle strutture dei processi
- ufs, zfs - file system di uso generale

Anche Linux ha più file system.



10.5 Struttura delle Directory

Una raccolta di nodi contenenti informazioni su tutti i file

Sia la struttura delle directory che i file risiedono sul disco. Le informazioni delle directory si trovano di solito in cima a una partizione.

Le operazioni eseguite sulla Directory sono molte: ricerca di un file, creazione di un file, eliminazione di un file, elenco dei contenuti di una directory, rinomina di un file, navigazione del file system verso sottodirectory o directory padre.

10.5.1 Organizzazione delle Directory

La directory è organizzata logicamente per prestazioni, usabilità e controllo degli accessi. In particolare:

- Efficienza - individuare rapidamente un file
- Denominazione - comoda per gli utenti: due utenti possono avere lo stesso nome per file diversi; lo stesso file può avere più nomi diversi
- Raggruppamento - raggruppamento logico dei file per proprietà (es. tutti i programmi Java, tutti i giochi...)

10.5.2 Directory a Singolo Livello

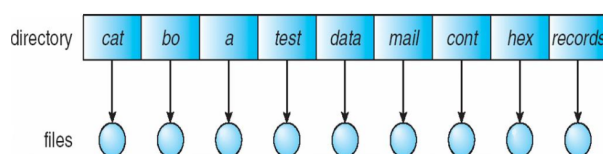


Figura 10.5: Una singola directory per tutti gli utenti

Molti problemi: il problema della denominazione e il problema del raggruppamento ne sono esempi.

10.5.3 Directory a Due Livelli

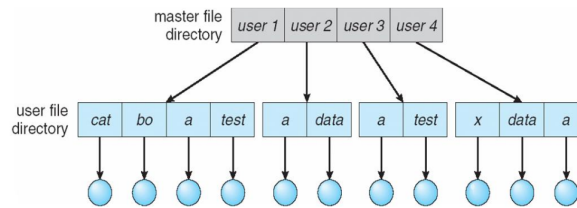
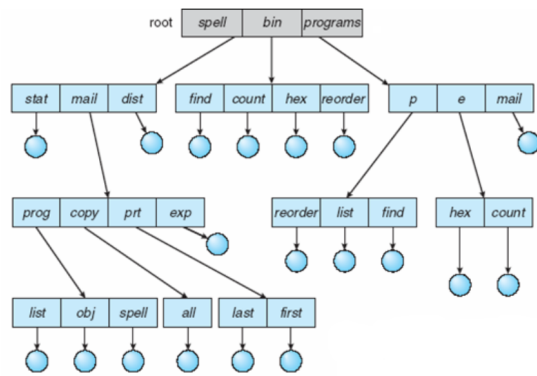


Figura 10.6: Directory separata per ogni utente

Caratteristiche: Percorso lungo (negativo), Possibilità di avere lo stesso nome di file per utenti diversi (neutro), Ricerca efficiente (positivo), Nessuna capacità di raggruppamento (negativo)

10.5.4 Directory ad Albero

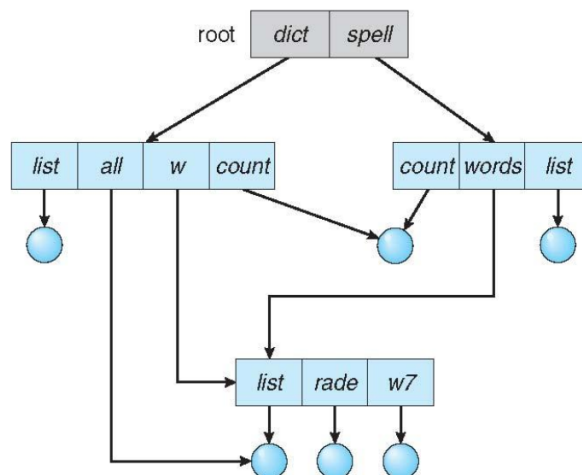


Pro	Contro
Generale	Inefficiente
Scalabile	Nessuna condivisione di file
Facile da cercare	Duplicazione di file in più directory

Figura 10.7: Schemi RAID

10.5.5 Directory a Grafo Aciclico

Hanno sottodirectory e file condivisi con nomi diversi (alias), ma possono fare riferimento agli stessi dati, quindi la condivisione è possibile.



Quando si elimina una directory, si ottiene un puntatore pendente (dangling pointer) che punta a memoria non più allocata; il suo dereferenzamento costituisce un comportamento indefinito.

Soluzioni:

- Backpointer, in modo da poter eliminare tutti i puntatori alla cartella eliminata; records di dimensione variabile sono un problema
- Backpointer usando un'organizzazione a catena (daisy chain)
- Soluzione con contatore di riferimenti (entry-hold-count)

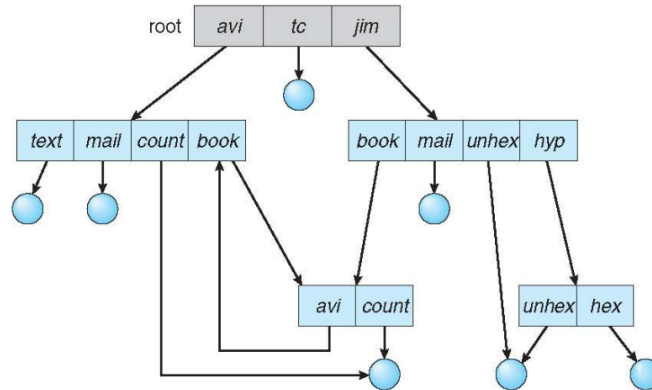
Nuovi tipi di voci nelle directory:

Link - un altro nome (puntatore) a un file esistente

Risolvi il link - segui il puntatore per localizzare il file

10.5.6 Directory a Grafo Generale

Come garantiamo l'assenza di cicli? Consentiamo solo link a file, non a sottodirectory; **Garbage collection**; ogni volta che viene aggiunto un nuovo link, si usa un algoritmo di rilevamento dei cicli per verificare se è accettabile.



10.6 Protezione

Il problema: non si desidera che altri modifichino, leggano o spostino i propri dati/programmi.

Il proprietario/creatore del file deve poter controllare: cosa può essere fatto/visto, da chi. Tipi di accesso: - Lettura - Scrittura - Esecuzione - Aggiunta - Eliminazione - Elenco.

Tutti i sistemi operativi forniscono questa opzione di base.

Capitolo 11

File System Implementation

11.1 Struttura del File System

Struttura del file: unità di archiviazione logica e raccolta di informazioni correlate.

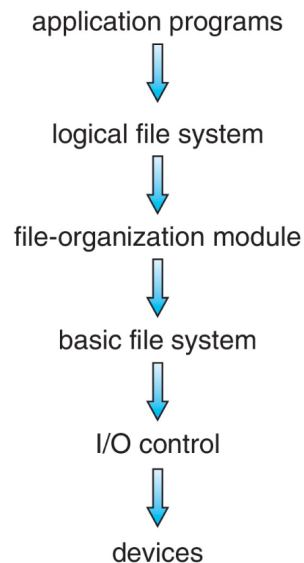
Il file system fornisce:

- interfaccia utente allo storage, mappando logico su fisico
- accesso efficiente e comodo al disco, permettendo di memorizzare, localizzare e recuperare facilmente i dati
- Interfaccia all'HW: il disco fornisce riscrittura in-place e accesso casuale - i trasferimenti I/O avvengono in blocchi o settori (solitamente 512 byte)

File control block (FCB) - struttura di archiviazione contenente informazioni su un file.

Driver del dispositivo controlla il dispositivo fisico - molte attività: un File System può essere organizzato a livelli.

11.2 File System a Livelli



11.2.1 Driver dei Dispositivi

I **driver dei dispositivi** gestiscono i dispositivi I/O al livello di controllo I/O.

Ricevuti comandi come: leggi drive1, cilindro 72, traccia 2, settore 10, in posizione memoria 1060.

Emette comandi hardware di basso livello specifici per il controller hardware

11.2.2 File System di Base

Il **file system di base**, ricevuto un comando come “recupera blocco 123”, lo traduce in comando per il driver del dispositivo.

Gestisce anche i buffer e le cache in memoria (allocazione, liberazione, sostituzione):

- I buffer contengono i dati in transito
- Le cache contengono i dati usati frequentemente

11.2.3 Modulo di Organizzazione dei File

Il **modulo di organizzazione** dei file comprende file, indirizzi logici e blocchi fisici.

Traduce il numero di blocco logico nel numero di blocco fisico.

Gestisce lo spazio libero e l’allocazione del disco.

11.2.4 File System Logico

Il file system logico gestisce le informazioni sui metadati:

- Traduce il nome del file in numero di file, handle e posizione, mantenendo i file control block (inode in UNIX)
- Gestione delle directory
- Protezione

La stratificazione è utile per ridurre complessità e ridondanza, ma aggiunge overhead e può diminuire le prestazioni.

I livelli logici possono essere implementati con qualsiasi metodo di codifica, a discrezione del progettista del OS.

Il file system logico viene chiamato direttamente dalle **applicazioni**, es. C open()

11.3 Tipi di File System

Molti file system, a volte più di uno nello stesso sistema operativo, ciascuno con il proprio formato:

- CD-ROM è ISO 9660;
- Unix ha UFS, FFS;
- Linux ha più di 130 tipi, con ext3 ed ext4 come estensioni del file system principale;
- Windows ha FAT, FAT32, NTFS oltre a floppy, CD, DVD Blu-ray.

Ne continuano ad arrivare di nuovi - ZFS, GoogleFS, Oracle ASM, FUSE

11.4 Dall'API all'Implementazione

11.4.1 Operazioni del File System

Abbiamo system call al livello API: Open, close, read, write, seek... ma come implementiamo le loro funzioni? Mediante strutture su disco e in memoria.

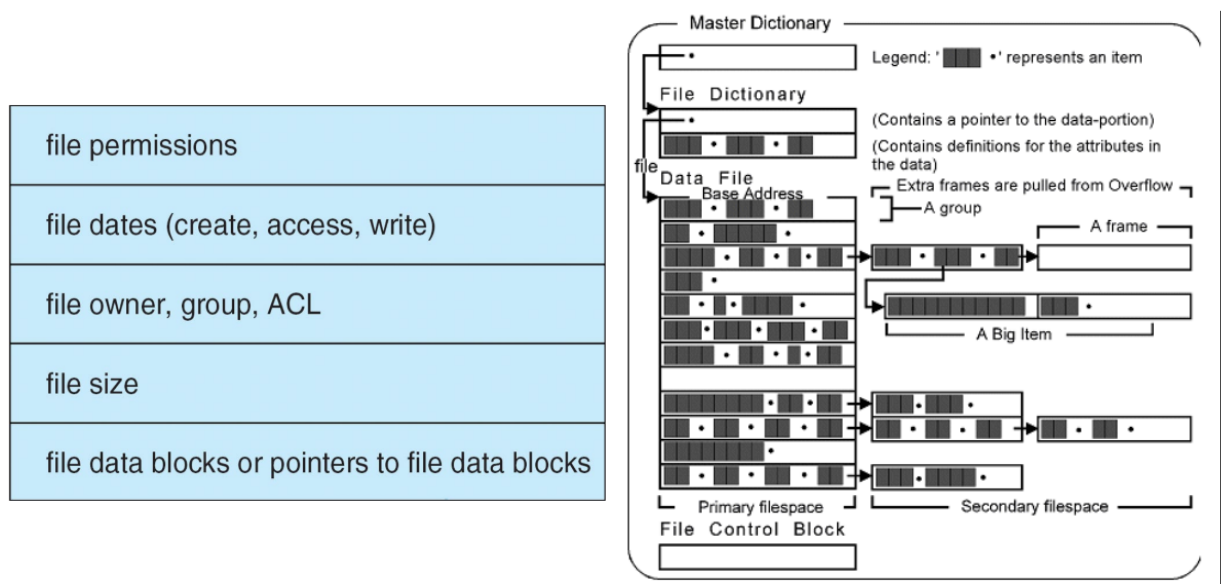
Boot control block (o **MBR**) contiene le informazioni necessarie al sistema per avviare l'OS da quel volume. Necessario se il volume contiene l'OS, di solito il primo blocco del volume.

Volume control block (superblock, master file table) contiene i dettagli del volume: numero totale di blocchi, numero di blocchi liberi, dimensione del blocco, puntatori o array di blocchi liberi.

La struttura delle directory organizza i file: nomi e numeri di inode, master file table.

11.4.2 Blocco di Controllo dei File - FCB

L'OS mantiene un FCB per ogni file, che contiene molti dettagli sul file: tipicamente, numero di inode (se Linux), permessi, dimensione, date.



11.4.3 Il Linux iNode

Un inode (noto anche come index node) è una struttura dati usata da Unix/Linux per descrivere un oggetto all'interno di un file system. Tale oggetto può essere un file o una directory.

Ogni inode memorizza puntatori alle posizioni dei blocchi su disco dei dati e dei metadati dell'oggetto. In generale, i metadati contenuti in un inode sono:

- tipo di file (file normale/directory/link simbolico/file speciale a blocchi/file speciale a caratteri/ecc.),
- permessi,
- ID proprietario/gruppo,
- dimensione,
- ora dell'ultimo accesso/modifica,
- ora di modifica dei metadati
- numero di hard link

filesystem

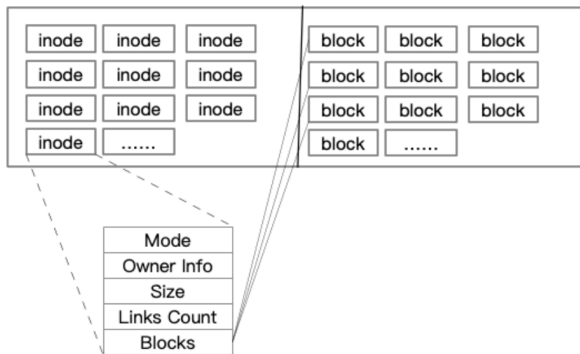


Figura 11.1: Il Linux iNode

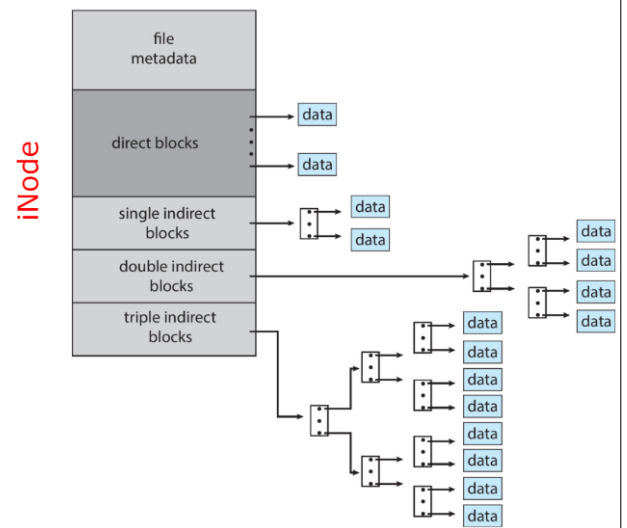


Figura 11.2: UNIX UFS - 4 KB per blocco, indirizzi a 32 bit

11.4.4 Strutture del File System in Memoria

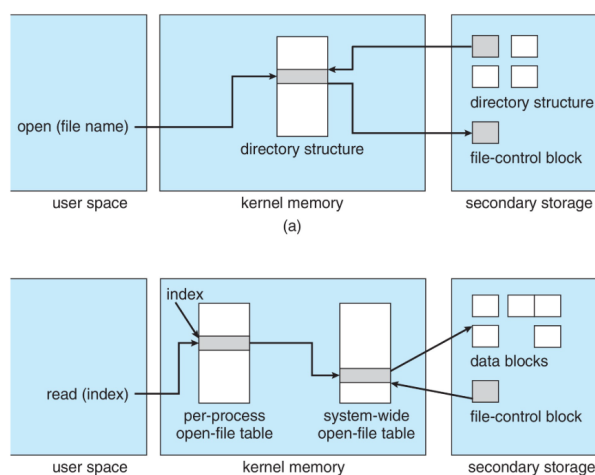
Tabella di montaggio - memorizza i mount dei file system, i punti di montaggio e i tipi di file system.

Tabella globale dei file aperti - contiene una copia del FCB di ogni file e altre informazioni.

Tabella dei file aperti per processo - contiene puntatori alle voci appropriate nella tabella globale e altre informazioni.

Gli elementi della tabella sono chiamati:

- fd descrittori di file in UNIX/Linux
- fh file handler in Windows
- stessa denominazione delle funzioni C

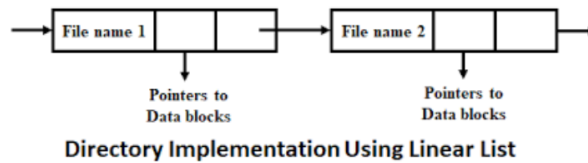


11.5 Implementazione delle Directory

11.5.1 Lista Lineare

Lista lineare di nomi di file con puntatori ai blocchi dati.

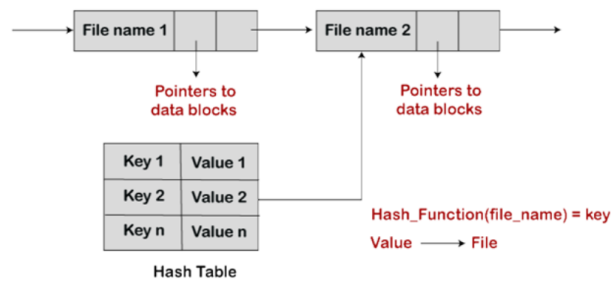
- Semplice da programmare
- Lenta da eseguire: tempo di ricerca lineare; si potrebbe mantenere ordinata alfabeticamente tramite lista collegata o usare un B+ tree



11.5.2 Tabella Hash

Tabella Hash - lista lineare con struttura dati hash:

- Riduce il tempo di ricerca nella directory
- Collisioni - situazioni in cui due nomi di file hanno lo stesso valore hash
- Efficiente solo se le voci hanno dimensione fissa, oppure si usa il metodo a catena di overflow



11.6 Metodo di Allocazione

Un metodo di allocazione definisce come i blocchi del disco vengono assegnati ai file:

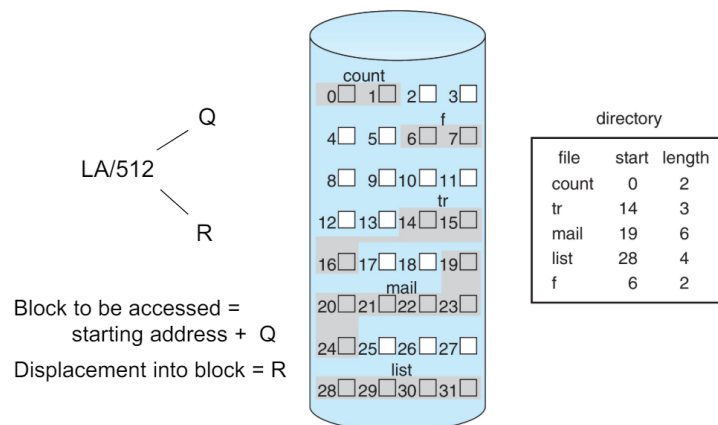
- Contigua
- Collegata
- Tabella di Allocazione dei File (FAT)

11.6.1 Metodo di Allocazione Contigua

Ogni file occupa un insieme di blocchi contigui.

- Migliori prestazioni nella maggior parte dei casi
- Semplice: sono necessari solo la posizione iniziale (numero di blocco) e la lunghezza (numero di blocchi)
- I problemi includono:
 - Trovare spazio sul disco per un file,
 - Conoscere la dimensione del file,
 - Frammentazione esterna nell'HD; necessita di compattazione offline (downtime) o online, detta anche defrag.

Mappatura da logico a fisico (dimensione blocco = 512 byte).



11.6.2 Allocazione Collegata

Ogni file è una lista collegata di blocchi

Il file termina con un puntatore null

Nessuna frammentazione esterna

Ogni blocco contiene un puntatore al blocco successivo

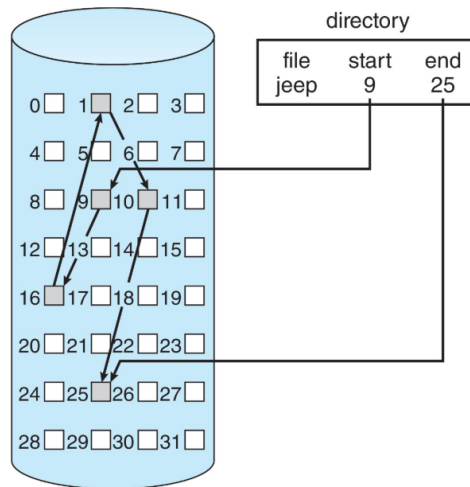
Nessuna compattazione, nessuna frammentazione esterna

Il sistema di gestione dello spazio libero viene chiamato quando è necessario un nuovo blocco

Si migliora l'efficienza raggruppando i blocchi in cluster, ma aumenta la frammentazione interna

Grande problema: localizzare un blocco richiede molti I/O e seek del disco.

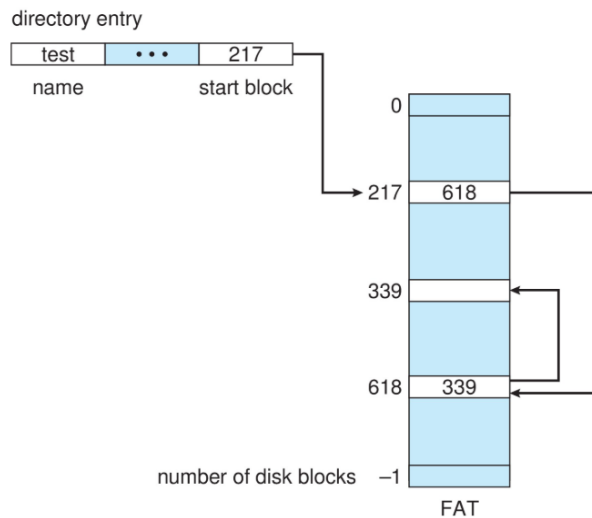
Ogni file è una lista collegata di blocchi su disco: i blocchi possono essere sparsi ovunque sul disco



Il blocco da accedere è il Q^{esimo} blocco nella catena collegata di blocchi che rappresenta il file.
 Spostamento nel blocco = $R + 1$.

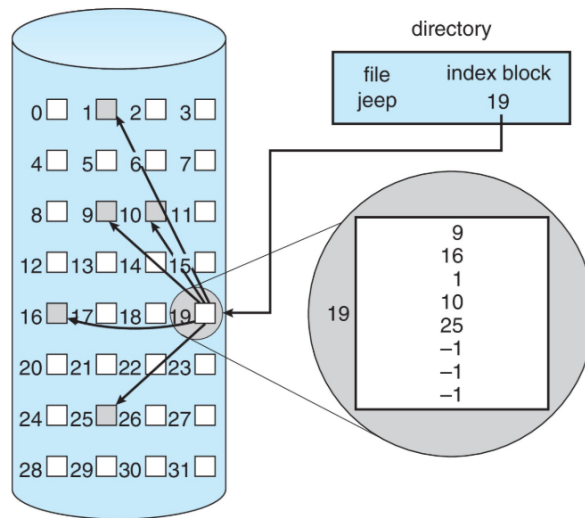
11.6.3 Metodo di Allocazione FAT

L'inizio del volume ha una tabella, indicizzata per numero di blocco. Simile a una lista collegata, ma più veloce su disco e memorizzabile in cache. L'allocazione di nuovi blocchi è semplice.



11.6.4 Metodo di Allocazione Indicizzata

Ogni file ha il proprio/i proprio blocco/i indice con puntatori ai propri blocchi dati.



11.6.5 Prestazioni

Il metodo migliore dipende dal tipo di accesso al file:

- L'allocazione contigua è ottima per accesso sequenziale e casuale
- L'allocazione collegata è buona per l'accesso sequenziale, non per quello casuale

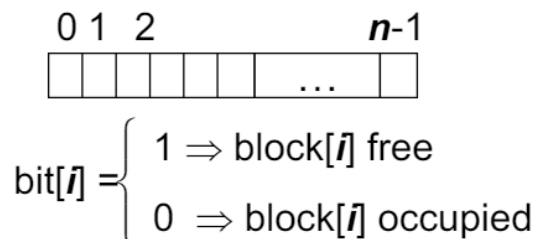
Dichiarare il tipo di accesso alla creazione: scegliere tra contigua o collegata.

L'indicizzata è più complessa: per accedere a un singolo blocco potrebbe essere necessario leggere prima il blocco indice e poi il blocco dati.

11.7 Gestione dello Spazio Libero

Il file system mantiene la **lista dello spazio libero** per tracciare i blocchi disponibili. Simile alla lista delle pagine libere per la gestione della memoria virtuale.

Vettore di bit o bit map (n blocchi)



Calcolo del numero di blocco:

Semplice, $(\text{numero di bit per parola}) * (\text{numero di parole con valore 0}) + \text{offset del primo bit a 1}$
 ma la bit map richiede spazio extra.

Esempio:

$$\text{dimensione blocco} = 4\text{KB} = 2^{12} \text{ byte}$$

dimensione disco = 2^{40} byte (1 TB)

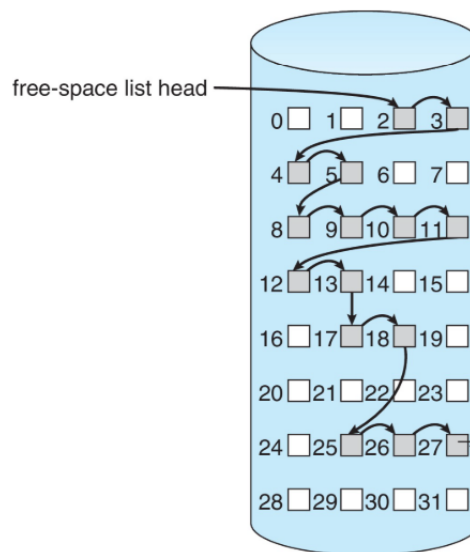
$$n = 2^{40} / 2^{12} = 2^{28} \text{ bit (o 32MB)}$$

Vantaggio: è facile ottenere spazio per file contigui verificando se ci sono abbastanza blocchi adiacenti liberi

11.7.1 Lista Collegata dello Spazio Libero su Disco

Lista collegata (free list):

- **SVANTAGGIO:** non è facile ottenere spazio contiguo
- **VANTAGGIO:** nessuno spreco. Lista collegata dello spazio libero su disco



Non è necessario attraversare l'intera lista (se il numero di blocchi liberi è registrato)

11.7.2 Lista Collegata dello Spazio Libero

Raggruppamento:

- Modificare la lista collegata per memorizzare l'indirizzo degli $n-1$ blocchi liberi successivi nel primo blocco libero, più un puntatore al blocco successivo che contiene puntatori a blocchi liberi (come questo)
- I primi $n-1$ blocchi saranno liberi
- L' n -esimo conterrà puntatori agli $n-1$ blocchi liberi successivi

Conteggio

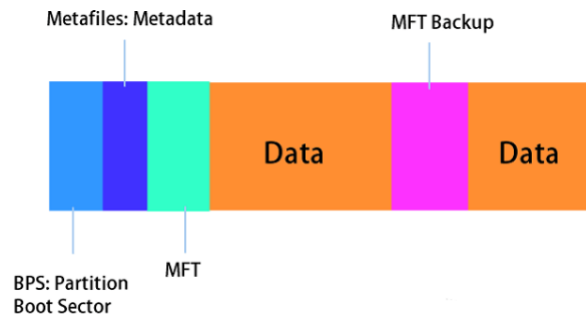
- Poiché lo spazio viene spesso usato e liberato in modo contiguo, con allocazione contigua, estensioni o clustering, si mantiene l'indirizzo del primo blocco libero e il conteggio dei blocchi liberi successivi. La lista dello spazio libero contiene quindi voci con indirizzi e conteggi.
- Il primo blocco della lista libera conterrà l'indirizzo del blocco libero successivo + il numero di blocchi liberi dopo di esso
- Gli altri saranno effettivamente blocchi liberi

11.8 File System in altri OS

11.8.1 File System di Windows

NTFS, detto anche New Technology File System, è un file system proprietario con journaling sviluppato da Microsoft.

A partire da Windows NT 3.1, è il file system predefinito della famiglia Windows NT. Ha soppiantato la File Allocation Table (FAT) come file system preferito su Windows ed è supportato anche da Linux e BSD.



FAT, nota anche come File Allocation Table, è un file system sviluppato per personal computer.

Originariamente sviluppata nel 1977 per l'uso su floppy disk, è stata adattata per dischi rigidi e altri dispositivi. È spesso supportata per motivi di compatibilità dai sistemi operativi attuali per PC e da molti dispositivi mobili e sistemi embedded, consentendo l'interscambio di dati tra sistemi diversi.

FAT32, successore di FAT16, è stata progettata da Microsoft come nuova versione del file system. FAT32 supporta un numero maggiore di cluster possibili e riutilizza la maggior parte del codice esistente.



Figura 11.3: Struttura del file system FAT32

11.8.2 Il File System di Apple

Nel 2017, Apple, Inc., ha rilasciato un nuovo file system per sostituire il suo HFS+ di 30 anni. HFS+ era stato esteso per aggiungere molte nuove funzionalità, ma come di consueto, questo processo ha aggiunto complessità, insieme a righe di codice, e ha reso più difficile aggiungere ulteriori funzionalità. Partire da zero su una pagina bianca consente di avviare una progettazione con le tecnologie e le metodologie attuali e di fornire l'esatto insieme di funzionalità necessarie.

Apple File System (APFS) è un buon esempio di tale progettazione. Il suo obiettivo è funzionare su tutti i dispositivi Apple attuali, dall'Apple Watch all'iPhone fino ai Mac. Creare un file system che funzioni su watchOS, iOS, tvOS e macOS è certamente una sfida. APFS è ricco di funzionalità, tra cui puntatori a 64 bit, cloni per file e directory, snapshot, condivisione dello spazio, calcolo rapido della dimensione delle directory, operazioni atomiche di salvataggio sicuro, design copy-on-write, cifratura (mono- e multi-chiave) e I/O coalescing. Supporta sia l'archiviazione NVM che HDD.

Molte di queste funzionalità sono state già discusse, ma ci sono alcuni nuovi concetti che vale la pena esplorare. La **condivisione dello spazio** è una funzionalità simile a ZFS in cui lo storage è disponibile come uno o più grandi spazi liberi (**container**) da cui i file system possono attingere allocazioni (permettendo ai volumi APFS di crescere e ridursi).

Il **calcolo rapido della dimensione delle directory** fornisce un aggiornamento rapido dello spazio usato. L'**atomic safe-save** è una primitiva (disponibile tramite API, non tramite comandi del file system) che esegue ridenominazioni di file, bundle di file e directory come operazioni atomiche singole. L'I/O coalescing è un'ottimizzazione per i dispositivi NVM in cui più piccole scritture vengono raggruppate in una grande scrittura per ottimizzare le prestazioni di scrittura.

Apple ha scelto di non implementare RAID come parte del nuovo APFS, dipendendo invece dal meccanismo Apple RAID volume per il RAID software. APFS è anche compatibile con HFS+, consentendo una facile conversione delle installazioni esistenti.